

Ejercicios de maratón de programación

Soluciones en Python 3 y C++11 con explicación breve. Material de apoyo para competencia.

#	L	Problema	Evento	Tema	Tiempo	Pág.
1	A	An Easy-Peasy Problem	2024-05-04-RPC04	Ad hoc	$O(1)$	4
2	B	Age Expression	2024-05-04-RPC04	Fuerza bruta / aritmetica	$O(D/A)$	4
3	C	Increasing Sublist	2024-05-04-RPC04	Recorrido lineal	$O(n)$	4
4	D	Let's Portmanteau	2024-05-04-RPC04	Cadenas / simulacion	$O(w1 + w2)$	5
5	E	Weighted Window Sums	2024-05-04-RPC04	Ventana deslizante + ordenamiento	$O(n \log n)$	5
6	F	Close Triangles	2024-05-04-RPC04	Backtracking / geometria	$O((3n)! / (6^n n!))$ con poda	6
7	G	Rightsizing	2024-05-04-RPC04	Heap / set ordenado	$O((n + a) \log(n + a))$	7
8	H	Brightline - Back to the Future	2024-05-04-RPC04	Caminos minimos con pesos negativos (Bellman-Ford/SPFA)	$O(n \cdot m)$ peor caso, tipico $O(m)$	8
9	I	Know your ABC's	2024-05-04-RPC04	Combinatoria	$O(a + b + c)$	9
10	J	Cut the Cake	2024-05-04-RPC04	Geometria (shoelace)	$O(n^3)$	10
11	A	Alphabetical Athletes	2024-08-10-RPC08	Cadenas / ordenamiento	$O(s \log s)$	10
12	B	Bright Beacons	2024-08-10-RPC08	BFS / visibilidad con pendientes	$O(r^*c^*(r+c))$	11
13	C	Cellar Chase	2024-08-10-RPC08	Parsing con pila / grafos serie-paralelo	$O(s)$	12
14	D	Document Dimensions	2024-08-10-RPC08	Greedy + busqueda binaria / poda	$O(\sum_W h(W) \log n)$ con poda	13
15	E	Euroexpress	2024-08-10-RPC08	Ad hoc / geometria	$O(n)$	14
16	F	Football Figurines	2024-08-10-RPC08	DP / conteo (Fibonacci)	$O(n + q)$	14
17	G	Genius Gamer	2024-08-10-RPC08	DP con estados por color	$O(13 * 4^4 * 2^4)$	15
18	H	Haggling over Hours	2024-08-10-RPC08	Flujo maximo / corte minimo de vertices	$O(n^2)$	16
19	I	Inconspicuous Identity	2024-08-10-RPC08	Geometria / trigonometria	$O(1)$	17
20	J	Jog in the Fog	2024-08-10-RPC08	Probabilidad / cota inferior	$O(n)$	18
21	K	Keeping Keys	2024-08-10-RPC08	Greedy / cadenas	$O(s)$	19
22	L	Longbottom Leap	2024-08-10-RPC08	Ad hoc / potencias de 2	$O(s)$	19
23	M	Montage Matrix	2024-08-10-RPC08	Conteo / ordenamiento	$O(n \log n)$	20
24	A	Alien Attack	2024-09-07-RPC09	Grafos / simulación + DSU en reversa	$O((n+m) \log n)$	20
25	B	Broken Borders	2024-09-07-RPC09	Geometría + strings (arreglo de sufijos)	$O((n + \sum k) \log n)$	21
26	C	Christmas Calories	2024-09-07-RPC09	Geometría / probabilidad	$O(1)$	21
27	D	Discus Domination	2024-09-07-RPC09	Ventana deslizante	$O(n)$	21
28	E	Elegant Exterior	2024-09-07-RPC09	Optimización / búsqueda ternaria	$O(\text{iteraciones})$	21
29	F	Fragmented Floor	2024-09-07-RPC09	Geometría ortogonal + emparejamiento bipartito	$O(r \cdot n + H \cdot V \cdot (H+V))$	22
30	G	Gorgeous Garment	2024-09-07-RPC09	Búsqueda binaria + greedy	$O(k \log^2 n)$	22
31	H	Hungry Hunting	2024-09-07-RPC09	DP de cambio de monedas + divide y vencerás	$O(n \cdot w \cdot \log n)$	22
32	I	Icons in the Toolbar	2024-09-07-RPC09	Greedy / sumas alternadas	$O(N)$	23
33	J	Jinxed Jewelry	2024-09-07-RPC09	Greedy / ordenamiento	$O(n \log n)$	23
34	K	K.O. Kids II	2024-09-07-RPC09	Probabilidad / DP	$O(n \cdot k)$	23
35	L	Legendary LAN-Party	2024-09-07-RPC09	Greedy / intercambio (regla de Smith)	$O(n \log n)$	23
36	M	Massive Mountains	2024-09-07-RPC09	Grafos / Dijkstra sobre pares + Floyd	$O(n^4)$	24
37	-	ACM (caminata del equipo)	2024-ACIS-2017	Agrupación / sumas	$O(S \log S)$	24
38	-	Balance	2024-ACIS-2017	Fuerza bruta	$O(C^3)$	25
39	A	Fibonacci divisibility	2024-ejercicios-propios	Fibonacci modular	$O(n \cdot P(M))$	25
40	B	Greatest common divisor	2024-ejercicios-propios	Euclides	$O(n \log V)$	26
41	C	Pitágoras (tipo de triángulo)	2024-ejercicios-propios	Geometría entera	$O(n)$	26
42	D	Sum of digits	2024-ejercicios-propios	Aritmética	$O(n \cdot d)$	27
43	E	Maximum of array	2024-ejercicios-propios	Recorrido lineal	$O(n)$	27
44	F	Median of three	2024-ejercicios-propios	Ordenamiento	$O(n)$	28
45	G	Minimum of two	2024-ejercicios-propios	Comparación	$O(n)$	28

#	L	Problema	Evento	Tema	Tiempo	Pág.
46	H	Triangles	2024-ejercicios-proprios	Desigualdad triangular	$O(n)$	28
47	-	Duel	2025-04-26-maraton	Greedy / dos punteros	$O(n \log n)$	29
48	A	Intentional Blank Page	2025-09-27-RPC11	Ad hoc	$O(p)$	30
49	B	Full House	2025-09-27-RPC11	Ad hoc, conteo	$O(1)$	30
50	C	Duck Stress Balls	2025-09-27-RPC11	Greedy, conteo	$O(c)$	30
51	D	Twin Numbers	2025-09-27-RPC11	Matemáticas, conteo por dígitos	$O(\text{len})$	31
52	E	Simple House	2025-09-27-RPC11	Geometría, búsqueda ternaria	$O(l^2), l = 100$ iteraciones	32
53	F	Pathfinder and Wolves	2025-09-27-RPC11	BFS multifuente, DSU (cuello de botella)	$O(r^c \cdot \alpha)$	33
54	G	Festival of Blossoms	2025-09-27-RPC11	Combinatoria sobre cadenas (bordes, Guibas-Odlyzko)	$O(N \cdot \text{bordes}(S))$	34
55	H	Birthday Bash	2025-09-27-RPC11	Probabilidad (coleccionista de cupones), tabla precalculada	$O(10^6)$	35
56	I	Dark Things	2025-09-27-RPC11	Geometría, búsqueda binaria	$O(k \cdot l \cdot n), l = 55$	35
57	J	Galactic Voyage	2025-09-27-RPC11	Dijkstra sobre residuos, CRT	$O(m \cdot (S + P^2 K) \log m)$	37
58	K	Degenerating Walkways	2025-09-27-RPC11	Probabilidad en árboles, árbol virtual, rerooting	$O((n + m) \log n)$	38
59	L	Copogonia	2025-09-27-RPC11	Grafos, fuerza bruta sobre subconjuntos	$O(2^k \cdot n^2)$	42
60	M	Weekend Gardening	2025-09-27-RPC11	Probabilidad, DP	$O(c \cdot n \cdot e)$	43
61	A	Alto Adaptation	2025-10-11-RPC12	DP	$O(n^2)$	44
62	B	Bottle of New Port	2025-10-11-RPC12	Matemáticas / ad hoc	$O(1)$	44
63	C	Chess	2025-10-11-RPC12	Simulación	$O(N^2 + M)$	44
64	D	Dralinpome	2025-10-11-RPC12	Strings / conteo	$O(s)$	44
65	E	Encrypting	2025-10-11-RPC12	Strings / ad hoc	$O(N)$	45
66	F	Flakes	2025-10-11-RPC12	Simulación / grillas	$O(nm(n+m))$	45
67	G	Genealogy Gumbo	2025-10-11-RPC12	Grafos / alcanzabilidad	$O(n \log n)$	45
68	H	Hidden Sequence	2025-10-11-RPC12	Greedy / punteros	$O(\text{total})$	46
69	I	Item Selection	2025-10-11-RPC12	Greedy	$O(n)$	46
70	J	Judge Meetings	2025-10-11-RPC12	Simulación	$O(n \cdot m)$	46
71	K	Koehandel	2025-10-11-RPC12	Ad hoc	$O(1)$	46
72	L	Luminosity	2025-10-11-RPC12	DP de intervalos / TSP en la recta	$O(K^2), K \leq 2N+1$	47
73	M	Most Scenic Cycle	2025-10-11-RPC12	Grafos serie-paralelo	$O(E \log E)$	47
74	A	Accidental Arithmetic	2025-11-01-RPC13	Probabilidad / linealidad de la esperanza	$O(L)$	47
75	B	Boggle Sort	2025-11-01-RPC13	DP	$O(16 \cdot 6 \cdot 26)$	48
76	C	Coherency	2025-11-01-RPC13	Geometría + rejilla hash + DSU	$O(n \cdot k), k = \text{vecinos por celda}$	48
77	D	Dart Game	2025-11-01-RPC13	Implementación	$O(1)$	48
78	E	Excruciating Elevators	2025-11-01-RPC13	Residuos modulares + enumeración de fases ancladas	$O(n^3 \cdot n \cdot 4) \approx 1e8$ (C++ 0.6 s)	48
79	F	Fences Make Good Neighbors	2025-11-01-RPC13	DP de intervalos (triangulación de polígono convexo)	$O(n^3)$	49
80	G	Geometry Rush	2025-11-01-RPC13	Barrido por columnas / intervalos	$O(w + n + m)$	49
81	H	Honest (or dishonest) Lottery	2025-11-01-RPC13	Conteo	$O(n)$	49
82	I	Intermill Logistics	2025-11-01-RPC13	Greedy + ordenamiento	$O(n \log n)$	50
83	J	Jump ... Triple Jump	2025-11-01-RPC13	Fuerza bruta	$O(1000)$	50
84	K	Knowing the Clock	2025-11-01-RPC13	Matemática	$O(1)$	50
85	L	Linguistic Labyrinth	2025-11-01-RPC13	Conteo en retícula 3D	$O(n^3 \cdot (2n)^3)$	50
86	M	Marching Orders	2025-11-01-RPC13	CRT generalizado	$O(n^2)$	51
87	-	Borrar	2025-maraton-interna-UTP	Cadenas	$O(s)$	51
88	-	Guanex (cola de prioridad)	2025-maraton-interna-UTP	Heap	$O(n \log n)$	52
89	-	K cuadrado	2025-maraton-interna-UTP	Aritmética modular	$O(q)$	52
90	-	Bubble sort	2025-practica-codeforces	Ordenamiento	$O(n^2)$	53
91	-	Maximum subarray	2025-practica-codeforces	Kadane / DP	$O(n)$	53
92	A	A Little Leftover Pizza	2026-04-11-RPC03	Implementación / aritmética	$O(n)$	54
93	B	Andor Strikes Again	2026-04-11-RPC03	DP en árboles	$O(N)$	54
94	C	AROD	2026-04-11-RPC03	Conteo / geometría en retícula	$O(m_x \cdot m_y + V \log V + m_x^2 + m_y^2)$	54
95	D	But I Want to Win	2026-04-11-RPC03	Greedy / simulación	$O(c \log c)$	55

#	L	Problema	Evento	Tema	Tiempo	Pág.
96	E	Chess Solitaire	2026-04-11-RPC03	Backtracking con memoización	$O(\text{estados} \cdot m^2)$	55
97	F	Fractional Sequence	2026-04-11-RPC03	Matemáticas	$O(1)$	55
98	G	How Many Balls?	2026-04-11-RPC03	Matemáticas / ecuación cuadrática	$O(R)$, $R = 10^4$	55
99	H	Move It, Slowpoke!	2026-04-11-RPC03	Dijkstra sobre estados	$O(m \cdot d \cdot \text{grado} \cdot \log)$	56
100	I	Number Pyramid	2026-04-11-RPC03	Propagación de restricciones	$O(n^2)$	56
101	J	Polyomino Tiling	2026-04-11-RPC03	Cadenas / palíndromos con hueco + bitsets	$O(k^3/64)$ peor caso teórico; $\sim k^2/8$ arcos $\times k/64$ en la práctica	56
102	K	Triptych	2026-04-11-RPC03	DP de conteo	$O(w^3)$	57
103	L	Valley Gulls	2026-04-11-RPC03	Simulación por eventos	$O(p^2 \cdot (p + m))$	57
104	A	Alejandro, lee por favor	RCP09-UTP	Simulación / cadenas	$O(L)$	57
105	B	Betito el viajero	RCP09-UTP	Grafos / flood fill	$O(R \cdot C)$ por caso	58
106	C	Company	RCP09-UTP	Árboles / DP	$O(N)$	58
107	D	Dangerous odyssey	RCP09-UTP	BFS en grilla / sumas de prefijos	$O(N \cdot M)$	58
108	E	Enarmonía	RCP09-UTP	Programación dinámica	$O(T^2 \cdot K)$	58
109	F	Flipando colores con la DIAN	RCP09-UTP	Greedy / scheduling en árboles (Horn)	$O(n \log n)$	59
110	G	Guanex y el diámetro con actualizaciones	RCP09-UTP	Árboles / LCA (binary lifting)	$O((n+q) \log V)$	59
111	H	Humbertov y su taza de café	RCP09-UTP	Geometría / fórmula cerrada	$O(1)$ por caso	59
112	I	Internal triangles	RCP09-UTP	Combinatoria / aritmética modular	$O(1)$ por consulta	60
113	J	Juan y sus ovejas	RCP09-UTP	Union-Find	$O((N+P) \alpha(N))$	60
114	K	K-th shortest path	RCP09-UTP	Dijkstra	$O(K \cdot M \log N)$	60
115	L	Locate Bobby's nest	RCP09-UTP	Árboles / centro por BFS	$O(H \cdot W)$ por caso	60
116	M	Marble tilt maze	RCP09-UTP	BFS en espacio de estados	$O((NM)^2)$	61

1. A. An Easy-Peasy Problem

Evento: RPC 2024 - 4th Activity (Competitive Programming Network) **Tema:** Ad hoc **Tiempo:** O(1) **Memoria:** O(1) **Archivo:** Peasy.py / .cpp
Estado: local-ok

Enunciado: Dados los aciertos s1 al final de la primera mitad y s2 al final del concurso, decidir si al menos la mitad de quienes resolvieron lo hicieron en la primera mitad.

Solución: El problema es facil si al menos la mitad de los que lo resolvieron lo hicieron en la primera mitad, es decir $s1 \geq s2/2$. Para evitar division y decimales se compara en enteros: $2*s1 \geq s2$. Si se cumple se imprime E, si no H. Como s2 ya incluye a s1, no hay que restar nada. Revisando los ejemplos: 10 de 20 es exactamente la mitad (E), 6 de 13 queda por debajo de 6.5 (H) y 15 de 22 supera 11 (E). Solo es lectura de dos enteros y una comparacion, por lo que el tiempo y la memoria son constantes.

Python 3

```
# Facil si al menos la mitad resolvio en la primera mitad: 2*s1 >= s2.
s1, s2 = map(int, input().split())
print("E" if 2 * s1 >= s2 else "H")
```

C++11

```
// Facil si al menos la mitad resolvio en la primera mitad: 2*s1 >= s2.
#include <bits/stdc++.h>
int main() { int a, b; std::cin >> a >> b; std::cout << (2 * a >= b ? "E" : "H") << "\n"; }
```

2. B. Age Expression

Evento: RPC 2024 - 4th Activity (Competitive Programming Network) **Tema:** Fuerza bruta / aritmetica **Tiempo:** O(D/A) **Memoria:** O(1) **Archivo:** Age.py / .cpp **Estado:** local-ok

Enunciado: Dada la edad D de Dr. O y las de sus nietas A y K, decidir si existen enteros positivos a, k con $a*A + k*K = D$.

Solución: Las edades son a lo sumo 150, asi que basta probar todos los valores de a desde 1 mientras el resto $D - a*A$ siga siendo positivo. Para cada uno, existe k positivo si y solo si ese resto es divisible por K (el resto positivo garantiza $k \geq 1$). Si algun a funciona se imprime 1, si no 0. Es una ecuacion diofantica lineal, pero con limites tan pequenos la fuerza bruta es mas simple y segura que usar el algoritmo extendido de Euclides y ajustar las soluciones a valores positivos. El ciclo hace como mucho unas 150 iteraciones.

Python 3

```
# Probar todos los a >= 1: (D - a*A) debe ser positivo y divisible por K.
d, A, K = map(int, input().split())
print(1 if any((d - a * A) % K == 0 for a in range(1, d // A + 1) if d - a * A > 0) else 0)
```

C++11

```
// Probar todos los a >= 1: (D - a*A) debe ser positivo y divisible por K.
#include <bits/stdc++.h>
int main() {
    int d, A, K, ok = 0; std::cin >> d >> A >> K;
    for (int a = 1; d - a * A > 0; a++) if ((d - a * A) % K == 0) ok = 1;
    std::cout << ok << "\n";
}
```

3. C. Increasing Sublist

Evento: RPC 2024 - 4th Activity (Competitive Programming Network) **Tema:** Recorrido lineal **Tiempo:** O(n) **Memoria:** O(n) **Archivo:** Increasing.py / .cpp **Estado:** local-ok

Enunciado: Hallar la longitud de la sublistas contigua estrictamente creciente mas larga de una lista de hasta 30 numeros.

Solución: Se recorre la lista una vez manteniendo la longitud de la racha creciente que termina en la posicion actual. Si el elemento es estrictamente mayor que el anterior, la racha crece en uno; si no (menor o igual), se reinicia en 1, porque el elemento actual empieza una nueva sublistas. En cada paso se actualiza el maximo visto. Los elementos repetidos rompen la racha porque se exige orden estricto, como muestran los tres 5 seguidos del segundo ejemplo. La respuesta minima es 1 porque cualquier elemento solo es una sublistas creciente valida. Es un solo recorrido lineal sin estructuras extra.

Python 3

```
# Recorrido lineal: longitud de la racha estrictamente creciente actual.
import sys
v = list(map(int, sys.stdin.read().split()))[1:]
best = cur = 1
for i in range(1, len(v)):
    cur = cur + 1 if v[i] > v[i - 1] else 1
```

```

    best = max(best, cur)
print(best)

```

C++11

```

// Recorrido lineal: longitud de la racha estrictamente creciente actual.
#include <bits/stdc++.h>
int main() {
    int n; std::cin >> n; std::vector<int> v(n);
    for (auto &x : v) std::cin >> x;
    int best = 1, cur = 1;
    for (int i = 1; i < n; i++) { cur = v[i] > v[i - 1] ? cur + 1 : 1; best = std::max(best, cur); }
    std::cout << best << "\n";
}

```

4. D. Let's Portmanteau

Evento: RPC 2024 - 4th Activity (Competitive Programming Network) **Tema:** Cadenas / simulación **Tiempo:** $O(|w1| + |w2|)$ **Memoria:** $O(|w1| + |w2|)$
Archivo: Portmanteau.py / .cpp **Estado:** local-ok

Enunciado: Combinar dos palabras: prefijo de la primera hasta su primera vocal (desde la segunda letra), una vocal de union y el sufijo de la segunda hasta su primera vocal hacia la izquierda.

Solución: Se simulan las reglas literalmente. En la primera palabra se avanza desde el índice 1 mientras no haya vocal; el prefijo es $w1[i:]$ y, si se encontro una, esa es $v1$. En la segunda se retrocede desde el penultimo caracter mientras no haya vocal; el sufijo es $w2[j+1:]$ y, si se encontro, esa es $v2$. La vocal de union es $v2$ si existe, si no $v1$, y si ninguna existe la letra 'o'. La primera letra de $w1$ y la ultima de $w2$ se toman siempre, aunque sean vocales, por eso los recorridos empiezan en la segunda y penultima posicion. Es igual a la solucion previa del usuario, solo que compactada.

Python 3

```

# Simulacion directa de las reglas: prefijo hasta la primera vocal (desde la 2a letra),
# sufijo hasta la primera vocal hacia la izquierda, y vocal de union v2, v1 u 'o'.
w1, w2 = input().strip(), input().strip()
V = "aeiou"
i = 1
while i < len(w1) and w1[i] not in V:
    i += 1
v1 = w1[i] if i < len(w1) else ""
j = len(w2) - 2
while j >= 0 and w2[j] not in V:
    j -= 1
v2 = w2[j] if j >= 0 else ""
print(w1[:i] + (v2 or v1 or "o") + w2[j + 1:])

```

C++11

```

// Simulacion directa de las reglas: prefijo hasta la primera vocal (desde la 2a letra),
// sufijo hasta la primera vocal hacia la izquierda, y vocal de union v2, v1 u 'o'.
#include <bits/stdc++.h>
using namespace std;
bool isv(char c) { return string("aeiou").find(c) != string::npos; }
int main() {
    string a, b; cin >> a >> b;
    int i = 1, j = (int)b.size() - 2;
    while (i < (int)a.size() && !isv(a[i])) i++;
    while (j >= 0 && !isv(b[j])) j--;
    char m = j >= 0 ? b[j] : (i < (int)a.size() ? a[i] : 'o');
    cout << a.substr(0, i) + m + b.substr(j + 1) << "\n";
}

```

5. E. Weighted Window Sums

Evento: RPC 2024 - 4th Activity (Competitive Programming Network) **Tema:** Ventana deslizante + ordenamiento **Tiempo:** $O(n \log n)$ **Memoria:** $O(n)$
Archivo: Weighted.py / .cpp **Estado:** local-ok

Enunciado: Para cada ventana de tamaño k calcular su suma ponderada $1 \cdot a_1 + \dots + k \cdot a_k$ e imprimir las ventanas ordenadas por esa suma, desempataando por índice de inicio.

Solución: Calcular cada suma ponderada desde cero costaría $O(nk)$, demasiado para $n = 2 \cdot 10^5$. Al desplazar la ventana un lugar, cada elemento que permanece baja su peso en uno y el primero sale: la nueva suma es $W - S + k \cdot a[i+k]$, donde S es la suma simple de la ventana actual. S también se actualiza en $O(1)$ sumando el que entra y restando el que sale. Así se obtienen todas las sumas en $O(n)$. Luego se ordenan los pares (suma, índice), y el orden lexicográfico ya resuelve los empates. Las sumas llegan a unos $2 \cdot 10^{18}$, por eso en C++ se usa long long.

Python 3

```
# Ventana deslizante:  $W(i+1) = W(i) - \text{suma}(\text{ventana } i) + k \cdot a[i+k]$ . Ordenar por (W, i).
import sys
d = sys.stdin.buffer.read().split()
n, k = int(d[0]), int(d[1])
a = list(map(int, d[2:2 + n]))
w = sum((j + 1) * a[j] for j in range(k))
s = sum(a[:k])
res = [(w, 1)]
for i in range(n - k):
    w += k * a[i + k] - s
    s += a[i + k] - a[i]
    res.append((w, i + 2))
res.sort()
sys.stdout.write("\n".join("%d %d" % (i, w) for w, i in res) + "\n")
```

C++11

```
// Ventana deslizante:  $W(i+1) = W(i) - \text{suma}(\text{ventana } i) + k \cdot a[i+k]$ . Ordenar por (W, i).
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, k; cin >> n >> k; vector<long long> a(n);
    for (auto &x : a) cin >> x;
    long long w = 0, s = 0;
    for (int j = 0; j < k; j++) w += (j + 1) * a[j], s += a[j];
    vector<pair<long long, int>> r{{w, 1}};
    for (int i = 0; i + k < n; i++) { w += k * a[i + k] - s; s += a[i + k] - a[i]; r.push_back({w, i + 2}); }
    sort(r.begin(), r.end());
    for (auto &p : r) cout << p.second << ' ' << p.first << '\n';
}
```

6. F. Close Triangles

Evento: RPC 2024 - 4th Activity (Competitive Programming Network) **Tema:** Backtracking / geometría **Tiempo:** $O((3n)! / (6^n n!))$ con poda
Memoria: $O(n^3)$ **Archivo:** Triangles.py / .cpp **Estado:** local-ok

Enunciado: Dados $3n$ puntos ($n \leq 5$), formar n triángulos usando cada punto una vez y minimizar la diferencia entre el área mayor y la menor.

Solución: Con $n \leq 5$ hay a lo sumo 15 puntos y unas 1.4 millones de particiones en triángulos, así que se enumeran todas con backtracking sobre una máscara de bits. Para no contar la misma partición varias veces, el punto libre de menor índice siempre se empareja con dos libres mayores. Se trabaja con el doble del área (entero, producto cruz), llevando el mínimo y el máximo de la partición parcial; si su diferencia ya no mejora la mejor respuesta, se poda la rama. Al final la diferencia doble D es entera, y la respuesta $D/2$ termina en .0 o .5, que se imprime exacta sin problemas de redondeo.

Python 3

```
# Backtracking: el punto libre de menor índice forma triángulo con otros dos libres.
# Áreas dobles enteras; se poda si (max - min) ya no mejora. Respuesta = D/2 exacta.
import sys
d = list(map(int, sys.stdin.read().split()))
n = d[0]; N = 3 * n
X = d[1::2]; Y = d[2::2]
A = [[abs((X[j] - X[i]) * (Y[k] - Y[i]) - (X[k] - X[i]) * (Y[j] - Y[i])) for k in range(N)]
      for j in range(N)] for i in range(N)]
best = 10 ** 18
def go(used, lo, hi):
    global best
    if hi - lo >= best:
        return
    if used == (1 << N) - 1:
        best = hi - lo
        return
    i = 0
    while used >> i & 1:
        i += 1
    free = [j for j in range(i + 1, N) if not used >> j & 1]
    Ai = A[i]
    for x in range(len(free)):
        j = free[x]; Aij = Ai[j]
        for k in free[x + 1:]:
            a = Aij[k]
            go(used | 1 << i | 1 << j | 1 << k, min(lo, a), max(hi, a))
go(0, 10 ** 18, -1)
print("%d.%d" % (best // 2, 5 * (best % 2)))
```

C++11

```
// Backtracking: el punto libre de menor índice forma triángulo con otros dos libres.
// Áreas dobles enteras; se poda si (max - min) ya no mejora. Respuesta = D/2 exacta.
```

```

#include <bits/stdc++.h>
using namespace std;
int N;
long long X[15], Y[15], best = LLONG_MAX;
long long ar(int i, int j, int k) { return llabs((X[j] - X[i]) * (Y[k] - Y[i]) - (X[k] - X[i]) * (Y[j] - Y[i])); }
void go(int used, long long lo, long long hi) {
    if (hi - lo >= best) return;
    if (used == (1 << N) - 1) { best = hi - lo; return; }
    int i = 0;
    while (used >> i & 1) i++;
    for (int j = i + 1; j < N; j++) if (!(used >> j & 1))
        for (int k = j + 1; k < N; k++) if (!(used >> k & 1)) {
            long long a = ar(i, j, k);
            go(used | 1 << i | 1 << j | 1 << k, min(lo, a), max(hi, a));
        }
}
int main() {
    int n; cin >> n; N = 3 * n;
    for (int i = 0; i < N; i++) cin >> X[i] >> Y[i];
    go(0, LLONG_MAX / 4, -LLONG_MAX / 4);
    cout << best / 2 << '.' << 5 * (best % 2) << "\n";
}

```

7. G. Rightsizing

Evento: RPC 2024 - 4th Activity (Competitive Programming Network) **Tema:** Heap / set ordenado **Tiempo:** $O((n + a) \log(n + a))$ **Memoria:** $O(n + a)$
Archivo: Rightsizing.py / .cpp **Estado:** local-ok

Enunciado: Con empleados y salarios, procesar aumentos y despidos; en cada despido sale el de mayor salario (desempate alfabético) y se imprime su nombre y salario.

Solución: Se necesita el máximo por salario con desempate por nombre, bajo actualizaciones. En C++ se usa un set de pares (-salario, nombre): el primer elemento es el despido, y un aumento borra la entrada vieja y mete la nueva, todo en $O(\log n)$. En Python se usa un heap con borrado perezoso: cada aumento empuja una entrada nueva sin borrar la vieja, y al despedir se sacan entradas hasta encontrar una cuyo salario coincida con el actual del empleado y que siga contratado. Cada entrada sale a lo sumo una vez, así que el costo total es $O((n + a) \log)$.

Python 3

```

# Max-heap con borrado perezoso de (-salario, nombre); cada aumento inserta una entrada nueva
# y al despedir se descartan las entradas obsoletas (salario distinto o ya despido).
import sys, heapq
d = sys.stdin.buffer.read().split()
n, a = int(d[0]), int(d[1]); p = 2
sal = {}; h = []
for _ in range(n):
    e = d[p].decode(); s = int(d[p + 1]); p += 2
    sal[e] = s; h.append((-s, e))
heapq.heapify(h)
out = []
for _ in range(a):
    if d[p] == b"1":
        e = d[p + 1].decode(); sal[e] += int(d[p + 2]); p += 3
        heapq.heappush(h, (-sal[e], e))
    else:
        p += 1
        while True:
            s, e = heapq.heappop(h)
            if sal.get(e) == -s:
                break
        del sal[e]
        out.append("%s %d" % (e, -s))
if out:
    sys.stdout.write("\n".join(out) + "\n")

```

C++11

```

// Set ordenado por (-salario, nombre); un aumento borra y reinserta, despedir toma el primero.
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, a; cin >> n >> a;
    map<string, long long> sal;
    set<pair<long long, string>> st;
    for (int i = 0; i < n; i++) { string e; long long s; cin >> e >> s; sal[e] = s; st.insert({-s, e}); }
    while (a--) {
        int t; cin >> t;
        if (t == 1) {
            string e; long long r; cin >> e >> r;
            st.erase({-sal[e], e}); sal[e] += r; st.insert({-sal[e], e});
        }
    }
}

```

```

    } else {
        auto it = st.begin();
        cout << it->second << ' ' << -it->first << '\n';
        sal.erase(it->second); st.erase(it);
    }
}
}
}

```

8. H. Brightline - Back to the Future

Evento: RPC 2024 - 4th Activity (Competitive Programming Network) **Tema:** Caminos minimos con pesos negativos (Bellman-Ford/SPFA) **Tiempo:** $O(n \cdot m)$ peor caso, típico $O(m)$ **Memoria:** $O(n + m)$ **Archivo:** Brightline.py / .cpp **Estado:** local-ok

Enunciado: Grafo dirigido con aristas azules (peso positivo) y rojas (peso negativo), sin ciclos negativos; listar las ciudades alcanzables desde 1 llegando mas joven.

Solución: Llegar mas joven a una ciudad equivale a que exista un camino desde la ciudad 1 con peso total negativo, donde las lineas rojas pesan -a y las azules +a. Como no hay ciclos negativos, la distancia minima esta bien definida y basta calcularla y quedarse con las ciudades donde es menor que cero. Dijkstra no sirve por los pesos negativos, así que se usa Bellman-Ford en su variante con cola (SPFA): solo se reprocesan los vertices cuya distancia mejoro. Con $n = 5000$ y $m = 10^4$ el peor caso es $5 \cdot 10^7$ relajaciones, aceptable. La ciudad 1 nunca sale porque su distancia es 0.

Python 3

```

# Camino minimo desde 1 con pesos negativos (sin ciclos negativos): SPFA (Bellman-Ford con cola).
# Se imprimen las ciudades con distancia minima < 0.
import sys
from collections import deque
d = sys.stdin.buffer.read().split()
n, m = int(d[0]), int(d[1])
g = [[] for _ in range(n + 1)]
for i in range(m):
    s, e, t, w = int(d[2 + 4 * i]), int(d[3 + 4 * i]), d[4 + 4 * i], int(d[5 + 4 * i])
    g[s].append((e, -w if t == b"r" else w))
INF = float("inf")
dist = [INF] * (n + 1); dist[1] = 0
inq = [False] * (n + 1); inq[1] = True
q = deque([1])
while q:
    u = q.popleft(); inq[u] = False; du = dist[u]
    for v, w in g[u]:
        if du + w < dist[v]:
            dist[v] = du + w
            if not inq[v]:
                inq[v] = True; q.append(v)
res = [str(v) for v in range(1, n + 1) if dist[v] < 0]
if res:
    print("\n".join(res))

```

C++11

```

// Camino minimo desde 1 con pesos negativos (sin ciclos negativos): SPFA (Bellman-Ford con cola).
// Se imprimen las ciudades con distancia minima < 0.
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m; cin >> n >> m;
    vector<vector<pair<int, long long>>> g(n + 1);
    for (int i = 0; i < m; i++) {
        int s, e; char t; long long a; cin >> s >> e >> t >> a;
        g[s].push_back({e, t == 'r' ? -a : a});
    }
    const long long INF = LLONG_MAX / 4;
    vector<long long> dist(n + 1, INF);
    vector<bool> inq(n + 1, false);
    deque<int> q{1}; dist[1] = 0; inq[1] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop_front(); inq[u] = false;
        for (auto &p : g[u]) if (dist[u] + p.second < dist[p.first]) {
            dist[p.first] = dist[u] + p.second;
            if (!inq[p.first]) { inq[p.first] = true; q.push_back(p.first); }
        }
    }
    for (int v = 1; v <= n; v++) if (dist[v] < 0) cout << v << '\n';
}

```

9. I. Know your ABC's

Evento: RPC 2024 - 4th Activity (Competitive Programming Network) **Tema:** Combinatoria **Tiempo:** $O(a + b + c)$ **Memoria:** $O(a + b + c)$ **Archivo:** ABC.py / .cpp **Estado:** local-ok

Enunciado: Contar, modulo 10^9+7 , las secuencias con a letras A, b B y c C sin dos letras iguales seguidas.

Solución: Al quitar las A, las B y C quedan en m bloques no vacíos, cada uno alternante. Como dos A no pueden ir juntas, A y bloques se intercalan: m vale a-1, a o a+1 (2 formas si $m = a$, 1 en los otros casos). Cada bloque es de tipo B extra (BCB..B), C extra o balanceado (BC..BC o CB..CB, 2 formas, al menos un par). Con p, q y r bloques de cada tipo, $p - q = b - c$ y $r = m - p - q$. Se multiplica el multinomial $m!/(p!q!r!)$, 2^r y las formas de repartir $T = b - p$ pares entre los m bloques, $C(T - r + m - 1, m - 1)$. Verificado con una DP de fuerza bruta.

Python 3

```
# Las B/C forman m segmentos alternantes separados por las A (|m - a| <= 1).
# Segmentos: p con una B extra, q con una C extra, r balanceados (2 formas, >= 1 par).
# Formas = sum m!/(p!q!r!) * 2^r * C(T - r + m - 1, m - 1) * (2 si m == a), T = b - p pares.
a, b, c = map(int, input().split())
M = 10 ** 9 + 7
F = [1] * 700
for i in range(1, 700):
    F[i] = F[i - 1] * i % M
I = [pow(f, M - 2, M) for f in F]
def comb(x, y):
    return 0 if y < 0 or x < y or x < 0 else F[x] * I[y] * I[x - y] % M
ans = 0
for m in range(max(1, a - 1), a + 2):
    mult = 2 if m == a else 1
    for p in range(m + 1):
        q = p - (b - c); r = m - p - q; T = b - p
        if q < 0 or r < 0 or T < r:
            continue
        ways = F[m] * I[p] * I[q] * I[r] * pow(2, r, M) * comb(T - r + m - 1, m - 1)
        ans = (ans + ways * mult) % M
print(ans)
```

C++11

```
// Las B/C forman m segmentos alternantes separados por las A (|m - a| <= 1).
// Segmentos: p con una B extra, q con una C extra, r balanceados (2 formas, >= 1 par).
// Formas = sum m!/(p!q!r!) * 2^r * C(T - r + m - 1, m - 1) * (2 si m == a), T = b - p pares.
#include <bits/stdc++.h>
using namespace std;
const long long M = 1000000007;
long long F[700], I[700];
long long pw(long long b, long long e) {
    long long r = 1; b %= M;
    for (; e >= 1; b = b * b % M) if (e & 1) r = r * b % M;
    return r;
}
long long comb(int x, int y) { return (y < 0 || x < 0 || x < y) ? 0 : F[x] * I[y] % M * I[x - y] % M; }
int main() {
    int a, b, c; cin >> a >> b >> c;
    F[0] = 1;
    for (int i = 1; i < 700; i++) F[i] = F[i - 1] * i % M;
    for (int i = 0; i < 700; i++) I[i] = pw(F[i], M - 2);
    long long ans = 0;
    for (int m = max(1, a - 1); m <= a + 1; m++)
        for (int p = 0; p <= m; p++) {
            int q = p - (b - c), r = m - p - q, T = b - p;
            if (q < 0 || r < 0 || T < r) continue;
            long long w = F[m] * I[p] % M * I[q] % M * I[r] % M * pw(2, r) % M * comb(T - r + m - 1, m - 1) % M;
            ans = (ans + w * (m == a ? 2 : 1)) % M;
        }
    cout << ans << "\n";
}
```

10. J. Cut the Cake

Evento: RPC 2024 - 4th Activity (Competitive Programming Network) **Tema:** Geometria (shoelace) **Tiempo:** $O(n^3)$ **Memoria:** $O(n)$ **Archivo:** Cake.py / .cpp **Estado:** local-ok

Enunciado: Cortar un poligono convexo con una recta que pase por dos vertices y minimizar la diferencia de areas entre las dos piezas.

Solución: Un corte por los vertices i y j no adyacentes divide el poligono convexo en el subpoligono $i..j$ y el resto. Con la formula del zapato (shoelace) se calcula el doble del area total S y el doble del area P de la pieza $i..j$; la diferencia real es $|S - 2P| / 2$. Se prueban todos los pares no adyacentes (excluyendo tambien el par primero-ultimo, que es un lado) y se toma el minimo. Con $n \leq 100$, recalculando cada pieza desde cero da $O(n^3)$, unas 10^6 operaciones. Como todo es entero, la respuesta termina en .0 o .5 y se imprime exacta, sin problemas de redondeo.

Python 3

```
# Area doble total S (shoelace). Para cada diagonal (i, j) no adyacente, la pieza i..j tiene
# area doble P; diferencia = |S - 2P| / 2.  $O(n^3)$  con  $n \leq 100$  basta.
import sys
d = list(map(int, sys.stdin.read().split()))
n = d[0]; X = d[1::2]; Y = d[2::2]
def area(i, j):
    idx = list(range(i, j + 1))
    return abs(sum(X[idx[t]] * Y[idx[t - 1]] - X[idx[t - 1]] * Y[idx[t]] for t in range(len(idx))))
S = area(0, n - 1)
best = min(abs(S - 2 * area(i, j)) for i in range(n) for j in range(i + 2, n) if not (i == 0 and j == n - 1))
print("%d.%d" % (best // 2, 5 * (best % 2)))
```

C++11

```
// Area doble total S (shoelace). Para cada diagonal (i, j) no adyacente, la pieza i..j tiene
// area doble P; diferencia = |S - 2P| / 2.  $O(n^3)$  con  $n \leq 100$  basta.
#include <bits/stdc++.h>
using namespace std;
int n;
long long X[100], Y[100];
long long area(int i, int j) {
    long long s = 0;
    for (int t = i; t <= j; t++) { int u = t == i ? j : t - 1; s += X[t] * Y[u] - X[u] * Y[t]; }
    return llabs(s);
}
int main() {
    cin >> n;
    for (int i = 0; i < n; i++) cin >> X[i] >> Y[i];
    long long S = area(0, n - 1), best = LLONG_MAX;
    for (int i = 0; i < n; i++)
        for (int j = i + 2; j < n; j++)
            if (!(i == 0 && j == n - 1)) best = min(best, llabs(S - 2 * area(i, j)));
    cout << best / 2 << "." << 5 * (best % 2) << "\n";
}
```

11. A. Alphabetical Athletes

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** Cadenas / ordenamiento **Tiempo:** $O(|s| \log |s|)$ **Memoria:** $O(|s|)$
Archivo: Athletes.py / .cpp **Estado:** local-ok

Enunciado: Decidir si una palabra (ignorando mayusculas) tiene sus letras en orden alfabetico, directo o inverso.

Solución: Se pasa la palabra a minusculas, porque solo la primera letra puede venir en mayuscula y no se distinguen. Luego basta comparar la palabra con su version ordenada ascendente y con su version ordenada descendente: si coincide con alguna, las letras aparecen en orden alfabetico (se permiten letras repetidas seguidas, como en Rommee). Ordenar 60 caracteres es instantaneo; tambien se podria revisar en un solo recorrido que todas las diferencias consecutivas tengan el mismo signo, pero la comparacion con sorted es mas corta y menos propensa a errores. Ejemplo: beklopt ordenada da la misma cadena, Ente no coincide con ninguno de los dos ordenes.

Python 3

```
# La palabra (en minusculas) debe ser no decreciente o no creciente.
s = input().strip().lower()
print("yes" if list(s) == sorted(s) or list(s) == sorted(s, reverse=True) else "no")
```

C++11

```
// La palabra (en minusculas) debe ser no decreciente o no creciente.
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s; cin >> s;
```

```

    for (char &c : s) c = tolower(c);
    string a = s, b = s;
    sort(a.begin(), a.end()); sort(b.rbegin(), b.rend());
    cout << (s == a || s == b ? "yes" : "no") << "\n";
}

```

12. B. Bright Beacons

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** BFS / visibilidad con pendientes **Tiempo:** $O(r*c*(r+c))$ **Memoria:** $O(r*c)$
Archivo: Beacons.py / .cpp **Estado:** local-ok

Enunciado: Cuadrícula de alturas; una hoguera ve a otra en la misma fila o columna si ningún pico intermedio queda estrictamente por encima del segmento. Mínimo de hogueras extra de la esquina NO a la SE.

Solución: Es un camino mínimo sin pesos: cada celda es un nodo y hay arista hacia las celdas visibles de su fila y su columna; la respuesta es la distancia BFS menos uno (la hoguera final ya existe). Para hallar las celdas visibles desde una celda se avanza en cada dirección guardando la pendiente máxima (altura relativa / distancia) de las celdas ya vistas. Una celda es visible si su pendiente es mayor o igual que esa máxima, porque entonces ningún pico intermedio queda por encima de la recta. Las pendientes se comparan con productos cruzados para evitar decimales. Cada celda revisa $r+c$ celdas, a lo sumo 2 millones de pasos.

Python 3

```

# BFS sobre celdas; desde cada celda se recorre cada dirección manteniendo la
# pendiente máxima vista: j es visible si su pendiente >= esa máxima.
import sys
from collections import deque
def main():
    d = sys.stdin.read().split()
    r, c = int(d[0]), int(d[1])
    h = [list(map(int, d[2 + i])) for i in range(r)]
    dist = [[-1] * c for _ in range(r)]
    dist[0][0] = 0
    q = deque([(0, 0)])
    while q:
        y, x = q.popleft()
        if y == r - 1 and x == c - 1:
            break
        nd = dist[y][x] + 1
        h0 = h[y][x]
        for dy, dx in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            bn, bd = 0, 0 # pendiente máxima bn/bd (bd=0: ninguna)
            yy, xx, t = y + dy, x + dx, 1
            while 0 <= yy < r and 0 <= xx < c:
                n = h[yy][xx] - h0
                if bd == 0 or n * bd >= bn * t:
                    if dist[yy][xx] < 0:
                        dist[yy][xx] = nd
                        q.append((yy, xx))
                    bn, bd = n, t
                yy += dy; xx += dx; t += 1
        print(dist[r - 1][c - 1] - 1)
    main()

```

C++11

```

// BFS sobre celdas; desde cada celda se recorre cada dirección manteniendo la
// pendiente máxima vista: j es visible si su pendiente >= esa máxima.
#include <bits/stdc++.h>
using namespace std;
int main() {
    int r, c; cin >> r >> c;
    vector<string> h(r);
    for (auto &s : h) cin >> s;
    vector<vector<int>> dist(r, vector<int>(c, -1));
    dist[0][0] = 0;
    queue<pair<int, int>> q; q.push({0, 0});
    int dy[] = {1, -1, 0, 0}, dx[] = {0, 0, 1, -1};
    while (!q.empty()) {
        int y = q.front().first, x = q.front().second; q.pop();
        for (int d = 0; d < 4; d++) {
            int bn = 0, bd = 0; // pendiente máxima bn/bd (bd=0: ninguna)
            for (int yy = y + dy[d], xx = x + dx[d], t = 1; yy >= 0 && yy < r && xx >= 0 && xx < c; yy += dy[d], xx += dx[d], t++) {
                int n = h[yy][xx] - h[y][x];
                if (bd == 0 || n * bd >= bn * t) {
                    if (dist[yy][xx] < 0) { dist[yy][xx] = dist[y][x] + 1; q.push({yy, xx}); }
                    bn = n; bd = t;
                }
            }
        }
    }
}

```

```

    }
    cout << dist[r - 1][c - 1] - 1 << "\n";
}

```

13. C. Cellar Chase

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** Parsing con pila / grafos serie-paralelo **Tiempo:** $O(|s|)$ **Memoria:** $O(\text{profundidad})$ **Archivo:** Cellar.py / .cpp **Estado:** local-ok

Enunciado: Un sistema de corredores serie-paralelo descrito con $()$, $(A+B)$ y $(A*B)$. Los profesores entran por la entrada, solo avanzan hacia la salida y nunca deben dejar un camino libre entre la entrada y el troll. Cuantos se necesitan.

Solución: Se define $f(\text{sistema})$. Un corredor simple necesita 1. En serie $(A+B)$ todos barren A, se reunen en la union (que bloquea el paso) y luego barren B, asi que basta $\max(f(A), f(B))$. En paralelo $(A*B)$ los profesores no pueden regresar: quien entra a una rama nunca pasa a la otra, y mientras una rama este sucia alguien debe custodiar la entrada; por eso se necesita $f(A)+f(B)$. Esto reproduce ambos ejemplos (2 y 5). La expresion se evalua con una pila de valores y operadores sin recursion, porque la cadena mide hasta un millon de caracteres y la anidacion puede ser muy profunda.

Python 3

```

# Serie (A+B) necesita max(A,B) profesores; paralelo (A*B) necesita A+B; () necesita 1.
import sys
def main():
    s = sys.stdin.readline().strip()
    val = [None]; op = [None]
    for ch in s:
        if ch == '(':
            val.append(None); op.append(None)
        elif ch == ')':
            v = val.pop(); op.pop()
            if v is None:
                v = 1
            if val[-1] is None:
                val[-1] = v
            elif op[-1] == '+':
                val[-1] = max(val[-1], v)
            else:
                val[-1] += v
        else:
            op[-1] = ch
    print(val[0])
main()

```

C++11

```

// Serie (A+B) necesita max(A,B) profesores; paralelo (A*B) necesita A+B; () necesita 1.
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s; cin >> s;
    vector<long long> val(1, 0); vector<char> op(1, 0); // val 0 = aun vacio
    for (char ch : s) {
        if (ch == '(') { val.push_back(0); op.push_back(0); }
        else if (ch == ')') {
            long long v = val.back(); val.pop_back(); op.pop_back();
            if (v == 0) v = 1;
            if (val.back() == 0) val.back() = v;
            else if (op.back() == '+') val.back() = max(val.back(), v);
            else val.back() += v;
        } else op.back() = ch;
    }
    cout << val[0] << "\n";
}

```

14. D. Document Dimensions

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** Greedy + búsqueda binaria / poda **Tiempo:** $O(\sum W \log n)$ con poda **Memoria:** $O(n)$ **Archivo:** Document.py / .cpp **Estado:** local-ok

Enunciado: Colocar n palabras en líneas (separadas por un espacio) eligiendo los saltos de línea para minimizar alto + ancho del papel.

Solución: Para un ancho W fijo, el alto mínimo lo da el greedy: meter en cada línea tantas palabras como quepan. Con prefijos $q[j]$ = suma de (largo+1), la última palabra de una línea que empieza en i se encuentra con búsqueda binaria ($q[j] \leq q[i] + W + 1$). Se prueban los anchos desde la palabra más larga, pero primero se evalúa $W = \text{raíz}(S)$, cerca del óptimo, para tener una buena cota. Luego se descarta cualquier W cuya cota inferior $W + (S+1)/(W+1)$ no mejore, se corta la simulación cuando el número de líneas ya no puede mejorar y se termina cuando $W+1$ alcanza la mejor respuesta. Así solo se simulan unos pocos anchos.

Python 3

```
# Probar cada ancho W >= palabra mas larga; alto por greedy saltando con bisect
# sobre prefijos. Se empieza en W=sqrt(S) y se poda con la cota alto >= (S+1)/(W+1) y cortando si ya no mejora.
import sys
from bisect import bisect_right
def main():
    d = sys.stdin.buffer.read().split()
    n = int(d[0])
    q = [0] * (n + 1) # q[j] = suma de (largo+1) de las primeras j palabras
    for i in range(n):
        q[i + 1] = q[i] + len(d[1 + i]) + 1
    S = q[n] - 1
    mx = max(len(w) for w in d[1:n + 1])
    best = S + 1 # W = S, una línea
    W0 = max(mx, int(S ** 0.5)) # se evalua primero cerca del optimo para podar mejor
    for W in [W0] + list(range(mx, S)):
        if W + 1 >= best: break
        if W + (S + W + 1) // (W + 1) < best: # cota inferior
            lim = best - W
            i = lines = 0
            while i < n and lines < lim:
                i = bisect_right(q, q[i] + W + 1, i) - 1
                lines += 1
            if i == n and W + lines < best:
                best = W + lines
    print(best)
main()
```

C++11

```
// Probar cada ancho W >= palabra mas larga; alto por greedy saltando con búsqueda binaria
// sobre prefijos. Se empieza en W=sqrt(S) y se poda con la cota alto >= (S+1)/(W+1) y cortando si ya no mejora.
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<long long> q(n + 1, 0); long long mx = 0; string w;
    for (int i = 0; i < n; i++) { cin >> w; q[i + 1] = q[i] + w.size() + 1; mx = max(mx, (long long)w.size()); }
    long long S = q[n] - 1, best = S + 1;
    long long W0 = max(mx, (long long)sqrt((double)S)); // primero cerca del optimo para podar mejor
    for (long long it = mx - 1; it < S; it++) {
        long long W = it < mx ? W0 : it;
        if (W + 1 >= best) break;
        if (W + (S + W + 1) / (W + 1) >= best) continue;
        long long lim = best - W, lines = 0; int i = 0;
        while (i < n && lines < lim) {
            i = upper_bound(q.begin() + i, q.end(), q[i] + W + 1) - q.begin() - 1;
            lines++;
        }
        if (i == n && W + lines < best) best = W + lines;
    }
    cout << best << "\n";
}
```

15. E. Euroexpress

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** Ad hoc / geometría **Tiempo:** O(n) **Memoria:** O(1) **Archivo:** Euroexpress.py / .cpp **Estado:** local-ok

Enunciado: Una maleta $p \leq q \leq r$ es válida si cada una de sus caras cabe en alguna de las n restricciones $a \times b$. Maximizar el volumen.

Solución: Ordenando las dimensiones de la maleta $p \leq q \leq r$, la cara más grande es $q \times r$, y las otras dos caras ($p \times q$ y $p \times r$) son componente a componente menores o iguales que ella. Por tanto, si $q \times r$ cabe en alguna restricción, todas las caras caben en esa misma. Dada una restricción $a \leq b$, lo mejor es $q=a$, $r=b$ y p tan grande como permite $p \leq q$, o sea $p=a$. El volumen es $a \times a \times b$ y la respuesta es el máximo sobre todas las restricciones. Llega hasta 10^{18} y cabe en long long. En el ejemplo 2, la restricción 3×8 da $3 \times 3 \times 8 = 72$.

Python 3

```
# Basta que la cara mayor (q,r) quepa en una restricción (a<=b); entonces p=q=a, r=b.
import sys
d = sys.stdin.buffer.read().split()
n = int(d[0]); best = 0
for i in range(n):
    a, b = int(d[1 + 2 * i]), int(d[2 + 2 * i])
    if a > b: a, b = b, a
    best = max(best, a * a * b)
print(best)
```

C++11

```
// Basta que la cara mayor (q,r) quepa en una restricción (a<=b); entonces p=q=a, r=b.
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; scanf("%d", &n); long long best = 0;
    for (int i = 0; i < n; i++) {
        long long a, b; scanf("%lld %lld", &a, &b);
        if (a > b) swap(a, b);
        best = max(best, a * a * b);
    }
    printf("%lld\n", best);
}
```

16. F. Football Figurines

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** DP / conteo (Fibonacci) **Tiempo:** O(n + q) **Memoria:** O(n) **Archivo:** Football.py / .cpp **Estado:** local-ok

Enunciado: Desde cada piso salen escaleras al piso +1 y al +2. Para cada consulta (s,t), sumar el número de escaleras de todas las rutas distintas de s a t, módulo $1e9+7$.

Solución: La respuesta solo depende de $k = t - s$. Sea $N(k)$ el número de rutas que suben k pisos: $N(0) = N(1) = 1$ y $N(k) = N(k-1) + N(k-2)$, como Fibonacci, según el primer paso. Sea $E(k)$ la suma de escaleras de todas esas rutas. Si la ruta empieza con un paso de 1, aporta $E(k-1)$ más una escalera por cada una de sus $N(k-1)$ rutas, y análogamente con el paso de 2. Entonces $E(k) = E(k-1) + E(k-2) + N(k-1) + N(k-2) = E(k-1) + E(k-2) + N(k)$. Se precalcula hasta n en O(n) y cada consulta se responde en O(1). Ejemplo: $E(4) = 7 + 3 + 5 = 15$.

Python 3

```
# N(k): rutas de longitud k (Fibonacci); E(k) = E(k-1) + E(k-2) + N(k) escalones totales.
import sys
def main():
    d = sys.stdin.buffer.read().split()
    n, q = int(d[0]), int(d[1])
    M = 10**9 + 7
    N = [1] * (n + 1); E = [0] * (n + 1)
    for k in range(2, n + 1):
        N[k] = (N[k - 1] + N[k - 2]) % M
    if n >= 1: E[1] = 1
    for k in range(2, n + 1):
        E[k] = (E[k - 1] + E[k - 2] + N[k]) % M
    out = [str(E[int(d[3 + 2 * i]) - int(d[2 + 2 * i])]) for i in range(q)]
    print("\n".join(out))
main()
```

C++11

```
// N(k): rutas de longitud k (Fibonacci); E(k) = E(k-1) + E(k-2) + N(k) escalones totales.
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    const long long M = 1000000007;
    int n, q; scanf("%d %d", &n, &q);
    vector<long long> N(n + 2, 1), E(n + 2, 0);
    E[1] = 1;
    for (int k = 2; k <= n; k++) { N[k] = (N[k - 1] + N[k - 2]) % M; E[k] = (E[k - 1] + E[k - 2] + N[k]) % M; }
    while (q--) { int s, t; scanf("%d %d", &s, &t); printf("%lld\n", E[t - s]); }
}
```

17. G. Genius Gamer

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** DP con estados por color **Tiempo:** $O(13 \cdot 4^4 \cdot 2^4)$ **Memoria:** $O(4^4)$
Archivo: Gamer.py / .cpp **Estado:** local-ok

Enunciado: Rummikub sin comodines: decidir si un conjunto de fichas (color, valor) se puede particionar en grupos (mismo valor, al menos 3 colores distintos) y escaleras (mismo color, al menos 3 valores consecutivos).

Solución: Se recorren los valores de 1 a 13. El estado guarda, por cada color, el largo de la escalera abierta que llega al valor anterior: 0, 1, 2 o 3+ (256 estados). En cada valor, cada ficha presente va a su escalera (el largo sube, con tope 3) o al grupo de ese valor. Una escalera solo puede cerrarse si su largo es 0 o 3+, y eso pasa cuando el color no tiene ficha o la ficha se va al grupo. El grupo del valor debe tener 0, 3 o 4 fichas, porque solo cabe un grupo por valor. Si una escalera de 3+ termina y luego empieza otra, el caso equivale a extenderla. Al final todos los colores deben quedar en 0 o 3+.

Python 3

```
# DP por valor 1..13; estado = largo de la escalera abierta por color (0,1,2,3+).
# Cada ficha va a escalera o al grupo del valor; el grupo debe tener 0, 3 o 4 fichas.
import sys
from itertools import product
def main():
    d = sys.stdin.read().split()
    n = int(d[0]); col = {"Red": 0, "Yellow": 1, "Blue": 2, "Black": 3}
    have = [[False] * 4 for _ in range(14)]
    for i in range(n):
        have[int(d[2 + 2 * i])][col[d[1 + 2 * i]]] = True
    states = {(0, 0, 0, 0)}
    for v in range(1, 14):
        nxt = set()
        for st in states:
            opts = [] # por color: lista de (nuevo_estado, va_al_grupo)
            for c in range(4):
                p = st[c]; o = []
                if have[v][c]:
                    o.append((min(p + 1, 3), 0))
                    if p in (0, 3): o.append((0, 1))
                elif p in (0, 3):
                    o.append((0, 0))
                opts.append(o)
            for ch in product(*opts):
                g = sum(x[1] for x in ch)
                if g in (0, 3, 4):
                    nxt.add(tuple(x[0] for x in ch))
        states = nxt
    ok = any(all(x in (0, 3) for x in st) for st in states)
    print("possible" if ok else "impossible")
main()
```

C++11

```
// DP por valor 1..13; estado = largo de la escalera abierta por color (0,1,2,3+), en base 4.
// Cada ficha va a escalera o al grupo del valor; el grupo debe tener 0, 3 o 4 fichas.
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    map<string,int> col = {"Red",0}, {"Yellow",1}, {"Blue",2}, {"Black",3};
    bool have[14][4] = {};
    for (int i = 0; i < n; i++) { string c; int v; cin >> c >> v; have[v][col[c]] = true; }
    vector<bool> cur(256, false); cur[0] = true;
    for (int v = 1; v <= 13; v++) {
        vector<bool> nxt(256, false);
        for (int st = 0; st < 256; st++) if (cur[st])
            for (int mask = 0; mask < 16; mask++) { // bit c: la ficha de color c va al grupo
                int g = __builtin_popcount(mask), ns = 0; bool ok = (g == 0 || g >= 3);
                for (int c = 0; c < 4 && ok; c++) {
                    int p = (st >> (2 * c)) & 3, np = 0;
                    bool closed = (p == 0 || p == 3);
                    if (mask >> c & 1) { if (!have[v][c] || !closed) ok = false; }
                    else if (have[v][c]) np = min(p + 1, 3);
                    else if (!closed) ok = false;
                    ns |= np << (2 * c);
                }
            }
        }
    }
```

```

    }
    if (ok) nxt[ns] = true;
}
cur = nxt;
}
bool ok = false;
for (int st = 0; st < 256; st++) if (cur[st]) {
    bool e = true;
    for (int c = 0; c < 4; c++) { int p = (st >> (2 * c)) & 3; if (p == 1 || p == 2) e = false; }
    if (e) ok = true;
}
cout << (ok ? "possible" : "impossible") << "\n";
}

```

18. H. Haggling over Hours

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** Flujo maximo / corte minimo de vertices **Tiempo:** $O(n^2)$ **Memoria:** $O(n)$
Archivo: Haggling.py / .cpp **Estado:** local-ok

Enunciado: Dados n intervalos, k es la cadena mas larga en la que cada inicio es al menos el fin anterior + 1. Quitar la minima cantidad de intervalos para que k baje.

Solución: Primero se calcula con DP $O(n^2)$ el largo $L(i)$ de la mejor cadena que termina en i y $R(i)$ el de la mejor que empieza en i . Solo importan los intervalos con $L+R-1 = k$, organizados en capas segun L . Hay que destruir todas las cadenas de largo k , que es un corte minimo de vertices entre la primera y la ultima capa, y por Menger es igual al flujo maximo con capacidad 1 por vertice. Para que haya $O(n)$ aristas, cada capa se ordena por inicio y se encadena con nodos auxiliares: desde i se apunta al primer intervalo de la capa siguiente con inicio $\geq \text{fin}+1$ y la cadena alcanza todo el sufijo. El flujo es a lo sumo n y se calcula con BFS de caminos aumentantes.

Python 3

```

# k = cadena mas larga (siguiente inicio >= fin+1). Respuesta = corte minimo de vertices
# que toque todas las cadenas de largo k: flujo maximo en el DAG por capas, con nodos
# "cadena" por capa (ordenados por inicio) para que las aristas sean  $O(n)$ .
import sys
from collections import deque
def main():
    d = sys.stdin.buffer.read().split()
    n = int(d[0])
    iv = sorted((int(d[1 + 2 * i]), int(d[2 + 2 * i])) for i in range(n))
    L = [1] * n; R = [1] * n
    for i in range(n):
        for j in range(n):
            if iv[j][1] + 1 <= iv[i][0] and L[j] + 1 > L[i]: L[i] = L[j] + 1
    for i in range(n - 1, -1, -1):
        for j in range(n):
            if iv[j][0] >= iv[i][1] + 1 and R[j] + 1 > R[i]: R[i] = R[j] + 1
    k = max(L)
    layer = [[] for _ in range(k + 2)]
    for i in range(n): # iv ya esta ordenado por inicio
        if L[i] + R[i] - 1 == k: layer[L[i]].append(i)
    V = 2 + 3 * n; INF = n + 1
    g = [[] for _ in range(V)]; to = []; cap = []
    def add(u, v, c):
        g[u].append(len(to)); to.append(v); cap.append(c)
        g[v].append(len(to)); to.append(u); cap.append(0)
    chain = {}
    for l in range(1, k + 1):
        lst = layer[l]
        for p, j in enumerate(lst):
            chain[j] = 2 + 2 * n + j
            add(chain[j], 2 + j, INF)
            if p + 1 < len(lst): add(chain[j], 2 + 2 * n + lst[p + 1], INF)
            add(2 + j, 2 + n + j, 1)
            if l == 1: add(0, 2 + j, 1)
            if l == k: add(2 + n + j, 1, 1)
    for l in range(1, k):
        nx = layer[l + 1]
        for i in layer[l]:
            b = iv[i][1] + 1
            for j in nx: # primer j de la capa siguiente con inicio >= b
                if iv[j][0] >= b:
                    add(2 + n + i, chain[j], INF); break
    flow = 0
    while True:
        par = [-1] * V; par[0] = -2; dq = deque([0])
        while dq and par[1] == -1:
            u = dq.popleft()
            for e in g[u]:
                if cap[e] and par[to[e]] == -1:

```

```

        par[to[e]] = e; dq.append(to[e])
    if par[1] == -1: break
    v = 1
    while v != 0:
        e = par[v]; cap[e] -= 1; cap[e ^ 1] += 1; v = to[e ^ 1]
        flow += 1
    print(flow)
main()

```

C++11

```

// k = cadena mas larga (siguiente inicio >= fin+1). Respuesta = corte minimo de vertices
// que toque todas las cadenas de largo k: flujo maximo en el DAG por capas, con nodos
// "cadena" por capa (ordenados por inicio) para que las aristas sean O(n).
#include <bits/stdc++.h>
using namespace std;
vector<vector<int>> g; vector<int> to, cap;
void add(int u, int v, int c) {
    g[u].push_back(to.size()); to.push_back(v); cap.push_back(c);
    g[v].push_back(to.size()); to.push_back(u); cap.push_back(0);
}
int main() {
    int n; scanf("%d", &n);
    vector<pair<long long, long long>> iv(n);
    for (auto &p : iv) scanf("%lld %lld", &p.first, &p.second);
    sort(iv.begin(), iv.end());
    vector<int> L(n, 1), R(n, 1);
    for (int i = 0; i < n; i++) for (int j = 0; j < n; j++)
        if (iv[j].second + 1 <= iv[i].first) L[i] = max(L[i], L[j] + 1);
    for (int i = n - 1; i >= 0; i--) for (int j = 0; j < n; j++)
        if (iv[j].first >= iv[i].second + 1) R[i] = max(R[i], R[j] + 1);
    int k = *max_element(L.begin(), L.end());
    vector<vector<int>> layer(k + 2);
    for (int i = 0; i < n; i++) if (L[i] + R[i] - 1 == k) layer[L[i]].push_back(i);
    int V = 2 + 3 * n, INF = n + 1; // 0=s, 1=t, 2+i entrada, 2+n+i salida, 2+2n+i cadena
    g.assign(V, {});
    for (int l = 1; l <= k; l++) {
        auto &lst = layer[l];
        for (size_t p = 0; p < lst.size(); p++) {
            int j = lst[p];
            add(2 + 2 * n + j, 2 + j, INF);
            if (p + 1 < lst.size()) add(2 + 2 * n + j, 2 + 2 * n + lst[p + 1], INF);
            add(2 + j, 2 + n + j, 1);
            if (l == 1) add(0, 2 + j, 1);
            if (l == k) add(2 + n + j, 1, 1);
        }
    }
    for (int l = 1; l < k; l++)
        for (int i : layer[l])
            for (int j : layer[l + 1])
                if (iv[j].first >= iv[i].second + 1) { add(2 + n + i, 2 + 2 * n + j, INF); break; }
    int flow = 0;
    while (true) {
        vector<int> par(V, -1); par[0] = -2; queue<int> q; q.push(0);
        while (!q.empty() && par[1] == -1) {
            int u = q.front(); q.pop();
            for (int e : g[u]) if (cap[e] && par[to[e]] == -1) { par[to[e]] = e; q.push(to[e]); }
        }
        if (par[1] == -1) break;
        for (int v = 1; v != 0; v = to[par[v] ^ 1]) { cap[par[v]]--; cap[par[v] ^ 1]++; }
        flow++;
    }
    printf("%d\n", flow);
}

```

19. I. Inconspicuous Identity

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** Geometría / trigonometría **Tiempo:** O(1) **Memoria:** O(1) **Archivo:** Identity.py / .cpp **Estado:** local-ok

Enunciado: Un paraguas de 8 varillas de largo x ; la tela entre varillas son triangulos isosceles y se dispone de $a \text{ m}^2$ de tela. Maximizar el area protegida (proyeccion vertical).

Solución: Si el angulo entre dos varillas vecinas es t , la tela usada es $8 \cdot (1/2) x^2 \sin t = 4x^2 \sin t$, que debe ser $\leq a$. Las puntas forman un octagono regular de radio R con lado $2x \sin(t/2) = 2R \sin(\pi/8)$. El area protegida es la del octagono, $2 \cdot \sqrt{2} \cdot R^2$, que crece con t . El paraguas totalmente plano tiene $t = \pi/4$ ($R = x$), asi que $t = \min(\pi/4, \arcsin(a/(4x^2)))$: si alcanza la tela se abre del todo, y si no se usa toda la tela disponible. En el ejemplo 1 sobra tela y la respuesta es $2 \cdot \sqrt{2} \cdot 0.25 = 0.7071$; en el ejemplo 2 el angulo es $\arcsin(0.1)$.

Python 3

```
# Tela:  $4x^2 \sin(t) \leq a$ , con  $t \leq \pi/4$  (paraguas plano). Las puntas forman un octagono
# regular de radio  $R = x \sin(t/2)/\sin(\pi/8)$ ; area seca =  $2*\sqrt{2}*R^2$ .
import math
a, x = map(float, input().split())
s = a / (4 * x * x)
t = math.pi / 4 if s >= math.sin(math.pi / 4) else math.asin(s)
R = x * math.sin(t / 2) / math.sin(math.pi / 8)
print("%.10f" % (2 * math.sqrt(2) * R * R))
```

C++11

```
// Tela:  $4x^2 \sin(t) \leq a$ , con  $t \leq \pi/4$  (paraguas plano). Las puntas forman un octagono
// regular de radio  $R = x \sin(t/2)/\sin(\pi/8)$ ; area seca =  $2*\sqrt{2}*R^2$ .
#include <bits/stdc++.h>
using namespace std;
int main() {
    double a, x, PI = acos(-1.0);
    scanf("%lf %lf", &a, &x);
    double s = a / (4 * x * x);
    double t = s >= sin(PI / 4) ? PI / 4 : asin(s);
    double R = x * sin(t / 2) / sin(PI / 8);
    printf("%.10f\n", 2 * sqrt(2.0) * R * R);
}
```

20. J. Jog in the Fog

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** Probabilidad / cota inferior **Tiempo:** O(n) **Memoria:** O(1) **Archivo:** Jog.py / .cpp **Estado:** local-ok

Enunciado: Jesse recorre en bucle una ruta de n celdas y empieza en una fase uniforme al azar. Sin ver nada, minimizar el tiempo esperado hasta encontrarlo (vale cruzarse a mitad de paso).

Solución: Sea d la distancia Manhattan minima de tu posicion a una celda de la ruta: ninguna fase se puede encontrar antes del tiempo d . Despues, cada medio segundo se encuentra como mucho una fase nueva (una al cruzarse a mitad de paso y otra al llegar a la celda). Eso se logra exactamente si, al llegar, se corre por la ruta en sentido contrario a Jesse: se encuentran fases distintas en $d, d+0.5, \dots, d+(n-1)/2$. Como la cota inferior se alcanza, la respuesta es el promedio $d + (n-1)/4$. Se calcula en enteros $(4d+n-1)/4$ y se imprime con dos decimales exactos. Ejemplos: $5 + 1/4$ y $2 + 5/4$.

Python 3

```
# Ir a la celda de la ruta mas cercana (Manhattan d) y correr en contra de Jesse:
# se encuentra una fase nueva cada medio segundo. Esperado =  $d + (n-1)/4$ .
import sys
d = sys.stdin.buffer.read().split()
x, y, n = int(d[0]), int(d[1]), int(d[2])
m = min(abs(int(d[3 + 2 * i]) - x) + abs(int(d[4 + 2 * i]) - y) for i in range(n))
print("%.2f" % ((4 * m + n - 1) / 4))
```

C++11

```
// Ir a la celda de la ruta mas cercana (Manhattan d) y correr en contra de Jesse:
// se encuentra una fase nueva cada medio segundo. Esperado =  $d + (n-1)/4$ .
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long x, y, n, m = LLONG_MAX;
    scanf("%lld %lld %lld", &x, &y, &n);
    for (int i = 0; i < n; i++) {
        long long a, b; scanf("%lld %lld", &a, &b);
        m = min(m, llabs(a - x) + llabs(b - y));
    }
    long long v = 4 * m + n - 1; // respuesta * 4, exacta
    printf("%lld.%02lld\n", v / 4, v % 4 * 25);
}
```

21. K. Keeping Keys

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** Greedy / cadenas **Tiempo:** $O(|s|)$ **Memoria:** $O(1)$ **Archivo:** Keys.py / .cpp **Estado:** local-ok

Enunciado: Se paga 1 centavo por tecla pulsada; mantener una tecla escribe repeticiones gratis y las mayusculas usan shift. Costo minimo de escribir el texto.

Solución: Una tecla de letra mantenida sigue escribiendo aunque se pulse o se suelte shift, asi que cada bloque maximo de caracteres consecutivos con la misma tecla (sin importar mayusculas) cuesta 1, y la barra espaciadora igual. Shift se puede mantener mientras se escriben espacios, y solo una minuscula obliga a soltarlo. Por eso se suma 1 por cada bloque maximo de mayusculas, donde los espacios no rompen el bloque. Ejemplos: en LI se pulsa shift, se mantiene l y se suelta shift, total 2. En A AaA las teclas son a, espacio, a (3) y hay dos bloques de shift (A A y la ultima A), total 5.

Python 3

```
# Una tecla por cada cambio de letra (sin importar mayuscula) o espacio, mas un shift
# por cada bloque de mayusculas (los espacios no rompen el bloque, las minusculas si).
s = input().rstrip("\n")
cost = 0; prev = None; shift = False
for ch in s:
    k = ch.lower()
    if k != prev:
        cost += 1
    prev = k
    if ch.isupper():
        if not shift: cost += 1; shift = True
    elif ch.islower():
        shift = False
print(cost)
```

C++11

```
// Una tecla por cada cambio de letra (sin importar mayuscula) o espacio, mas un shift
// por cada bloque de mayusculas (los espacios no rompen el bloque, las minusculas si).
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s; getline(cin, s);
    if (!s.empty() && s.back() == '\r') s.pop_back();
    int cost = 0; char prev = 0; bool shift = false;
    for (char ch : s) {
        char k = tolower(ch);
        if (k != prev) cost++;
        prev = k;
        if (isupper(ch)) { if (!shift) { cost++; shift = true; } }
        else if (islower(ch)) shift = false;
    }
    cout << cost << "\n";
}
```

22. L. Longbottom Leap

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** Ad hoc / potencias de 2 **Tiempo:** $O(|s|)$ **Memoria:** $O(|s|)$ **Archivo:** Longbottom.py / .cpp **Estado:** local-ok

Enunciado: El hechizo con k palabras long cubre $32 \cdot 2^k$ escalones consecutivos. Imprimir el hechizo mas corto que cubra toda la cadena (sin ceros al inicio ni al final).

Solución: Como la cadena no tiene ceros al inicio ni al final, el primer y el ultimo escalon necesitan arreglo, y el hechizo, lanzado una sola vez desde el primer escalon, debe cubrir exactamente |s| escalones. La version mas corta (long) cubre 32 y cada long extra duplica la capacidad. Basta duplicar desde 32 hasta alcanzar |s| y contar las palabras. Con $|s| \leq 10^6$ salen a lo sumo 16 palabras. Se imprime long repetido, separado por espacios. Ejemplos: una cadena de 32 caracteres sigue siendo long, y una de 33 necesita long long.

Python 3

```
# Menor k con 32*2^k >= |s|; se imprime "long" k+1 veces.
import sys
m = len(sys.stdin.readline().strip()); c, k = 32, 1
while c < m: c *= 2; k += 1
print(" ".join(["long"] * k))
```

C++11

```
// Menor k con 32*2^k >= |s|; se imprime "long" k+1 veces.
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    string s; cin >> s;
    long long c = 32; int k = 1;
    while (c < (long long)s.size()) { c *= 2; k++; }
    for (int i = 0; i < k; i++) cout << (i ? " long" : "long");
    cout << "\n";
}
```

23. M. Montage Matrix

Evento: RPC 2024 - 8th Activity (Competitive Programming Network) **Tema:** Conteo / ordenamiento **Tiempo:** $O(n \log n)$ **Memoria:** $O(n)$ **Archivo:** Montage.py / .cpp **Estado:** local-ok

Enunciado: Acomodar n personas en filas de a lo sumo w de forma que delante de cada uno solo haya personas estrictamente mas bajas.

Solución: Cada columna de la foto es una secuencia estrictamente decreciente de atras hacia adelante, y como cada fila tiene a lo sumo w personas, hay a lo sumo w columnas. Dos personas de igual altura no pueden compartir columna, así que si una altura aparece mas de w veces es imposible. Si ninguna aparece mas de w veces, se reparten las personas ordenadas por altura de forma ciclica en w columnas: las alturas iguales terminan en columnas distintas y cada columna queda estricta. Se ordena y se revisa si $h[i] == h[i+w]$ para algun i (en Python, con Counter).

Python 3

```
# Posible si y solo si ninguna altura se repite mas de w veces (w columnas estrictas).
import sys
from collections import Counter
d = sys.stdin.buffer.read().split()
n, w = int(d[0]), int(d[1])
print("yes" if max(Counter(d[2:2 + n]).values()) <= w else "no")
```

C++11

```
// Posible si y solo si ninguna altura se repite mas de w veces (w columnas estrictas).
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, w; scanf("%d %d", &n, &w);
    vector<int> h(n);
    for (auto &x : h) scanf("%d", &x);
    sort(h.begin(), h.end());
    bool ok = true;
    for (int i = 0; i + w < n; i++) if (h[i] == h[i + w]) ok = false;
    puts(ok ? "yes" : "no");
}
```

24. A. Alien Attack

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** Grafos / simulación + DSU en reversa **Tiempo:** $O((n+m) \log n)$ **Memoria:** $O(n+m)$ **Archivo:** Alien.py / Alien.cpp.py / .cpp **Estado:** local-ok

Solución: La clave es que las ciudades perdidas forman componentes sin aristas hacia las sobrevivientes, así que nunca alteran los grados de éstas. Por eso el orden en que se atacan las sobrevivientes coincide con la simulación pura que ignora la ETO: un heap de máximos por (grado, -índice) con borrado perezoso, excluyendo a NY. Con ese orden fijo, el resto es conectividad decremental offline: se insertan las ciudades en orden inverso uniendo con DSU a los vecinos ya presentes (NY está desde el inicio). Una ciudad cuenta como un día si al insertarla queda en la componente de NY, es decir, si seguía conectada cuando la atacaron. La respuesta es ese conteo más uno.

Python 3

(no disponible)

C++11

(no disponible)

25. B. Broken Borders

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** Geometría + strings (arreglo de sufijos) **Tiempo:** $O((n + \Sigma k) \log n)$ **Memoria:** $O(n + \Sigma k)$ **Archivo:** Broken.py / Broken.cpp.py / .cpp **Estado:** local-ok

Solución: Como no hay tres vértices consecutivos colineales, una pieza colocada debe coincidir con una cadena de aristas consecutivas del polígono. Cada par de aristas consecutivas se codifica con el token $(|e_1|^2, e_1 \cdot e_2, e_1 \times e_2, |e_2|^2)$, invariante a rotación y traslación. El texto es el polígono duplicado ($2n$ tokens) y se construye su arreglo de sufijos. Cada pieza, en sus 4 variantes (espejo $x \rightarrow -x$ y reversa), se busca con búsqueda binaria y da un rango de sufijos; cada rango se pinta con la mayor longitud usando un DSU de 'siguiente sin pintar'. Con diferencias se marcan las aristas cubiertas; las piezas de una sola arista cubren toda arista de igual longitud. YES si todo queda cubierto.

Python 3

(no disponible)

C++11

(no disponible)

26. C. Christmas Calories

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** Geometría / probabilidad **Tiempo:** $O(1)$ **Memoria:** $O(1)$ **Archivo:** Christmas.py / Christmas.cpp.py / .cpp **Estado:** local-ok

Solución: Si el ángulo central entre los dos puntos es ϕ , la distancia en línea recta es la cuerda $2r \cdot \sin(\phi/2)$. Se ve a Caro si esa cuerda es a lo sumo l , es decir si $\phi \leq 2 \cdot \text{asin}(l/(2r))$. Por simetría ϕ es uniforme en $[0, \pi]$, así que la probabilidad de verla es $2 \cdot \text{asin}(l/(2r))/\pi$ y la pedida es su complemento. Si $l \geq 2r$ se ve toda la pista y la respuesta es 0. Se imprime con 10 decimales en ambas versiones; la precisión doble basta incluso con $r = 10^9$ y $l = 1$.

Python 3

(no disponible)

C++11

(no disponible)

27. D. Discus Domination

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** Ventana deslizante **Tiempo:** $O(n)$ **Memoria:** $O(n)$ **Archivo:** Discus.py / Discus.cpp.py / .cpp **Estado:** local-ok

Solución: Para cada destino j el mejor punto de partida es el mínimo de a en la ventana $[j-m, j]$, que incluye a j (lanzar al mismo cuadro da 0, así que la respuesta nunca es negativa). Ese mínimo de ventana deslizante se mantiene con una deque monótona de índices con valores crecientes: al entrar j se sacan por detrás los índices con valor mayor o igual, y por delante los que salen de la ventana. La respuesta es el máximo de a_j menos el valor del frente de la deque. Cada índice entra y sale una sola vez, por lo que el total es lineal.

Python 3

(no disponible)

C++11

(no disponible)

28. E. Elegant Exterior

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** Optimización / búsqueda ternaria **Tiempo:** $O(\text{iteraciones})$ **Memoria:** $O(1)$ **Archivo:** Elegant.py / Elegant.cpp.py / .cpp **Estado:** local-ok

Solución: La madera usada es $2w + 2h + 2 \cdot \text{sqrt}(w^2 + h^2) + 2 \cdot \text{sqrt}((w/2)^2 + h^2)$ y el área es $wh + wh/2 = 1.5wh$. Con $t = h/w$ la madera es $w \cdot f(t)$ y el área $1.5 \cdot t \cdot w^2$. Conviene usar toda la madera, luego $w = n/f(t)$ y el área es $1.5 \cdot t \cdot n^2/f(t)^2$, que depende solo de t y es unimodal. Una búsqueda ternaria de 200 iteraciones en $t \in [0, 10]$ da el óptimo con precisión de sobra. Como el área escala con n^2 , también bastaría hallar la constante una vez, pero la ternaria es igual de corta.

Python 3

(no disponible)

C++11

(no disponible)

29. F. Fragmented Floor

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** Geometría ortogonal + emparejamiento bipartito **Tiempo:** $O(r \cdot n + H \cdot V \cdot (H+V))$ **Memoria:** $O(H \cdot V)$ **Archivo:** Fragmented.py / Fragmented.cpp.py / .cpp **Estado:** local-ok

Solución: Resultado clásico: mínimo = $r - L + 1$, donde r es el número de vértices reflejos y L el máximo de cuerdas 'buenas' (segmentos axiales internos entre dos vértices reflejos) que no se cruzan ni se tocan. Las cuerdas horizontales y verticales forman un grafo bipartito de cruces y, por König, $L = H + V$ - emparejamiento máximo. Las cuerdas se hallan lanzando desde cada vértice reflejo un rayo en la dirección opuesta a su arista horizontal (o vertical) y buscando la primera pared que toca; si el punto de impacto es otro vértice reflejo, hay cuerda. El emparejamiento se calcula con Kuhn (caminos aumentantes).

Python 3

(no disponible)

C++11

(no disponible)

30. G. Gorgeous Garment

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** Búsqueda binaria + greedy **Tiempo:** $O(k \log^2 n)$ **Memoria:** $O(n + k)$ **Archivo:** Gorgeous.py / Gorgeous.cpp.py / .cpp **Estado:** local-ok

Solución: Si se pueden tejer P vueltas también se pueden $P-1$: se quita la primera vuelta y todo se corre a vueltas más baratas. Así que se busca P binariamente. Para un P fijo se recorre desde el último color hacia atrás con el estado (vueltas que faltan R , ancho máximo permitido W = ancho del color siguiente). Que un color tome el mayor ancho posible ($\leq W$ y que su hilo alcance) deja R más chico y W más grande, ambos mejores, así que el greedy es óptimo. Ese ancho máximo se halla con búsqueda binaria sobre sumas prefijas. Es factible si R llega a 0.

Python 3

(no disponible)

C++11

(no disponible)

31. H. Hungry Hunting

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** DP de cambio de monedas + divide y vencerás **Tiempo:** $O(n \cdot w \cdot \log n)$ **Memoria:** $O(w \log n)$ **Archivo:** Hungry.py / Hungry.cpp.py / .cpp **Estado:** local-ok

Solución: Es el problema de mínimo de monedas ilimitadas con el conjunto $\{c_j : j \neq i\} \cup \{2c_i\}$. Rehacer la DP para cada i cuesta $n^2 w$, demasiado. Se usa divide y vencerás sobre los platos: solve(l, r, dp) recibe la DP con todos los platos fuera de $[l, r]$; para la mitad izquierda se agregan los platos de la derecha y viceversa. En la hoja solo falta agregar $2c_i$ y leer $dp[w]$. Cada plato se agrega $O(\log n)$ veces. Agregar la moneda v es $dp[x] = \min(dp[x], dp[x-v]+1)$.

Python 3

(no disponible)

C++11

(no disponible)

32. I. Icons in the Toolbar

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** Greedy / sumas alternadas **Tiempo:** $O(N)$ **Memoria:** $O(N)$
Archivo: Icons.py / Icons.cpp.py / .cpp **Estado:** local-ok

Solución: La fila 1 contiene al mayor, s_1 , y su alto es s_1 . Si la fila 2 tiene alto s_j , los íconos $s_1..s_{j-1}$ van en la fila 1, cada uno en su propia columna, y lo mejor es emparejarlos con $s_j..s_{2j-2}$, que no suman ancho. Los restantes $s_{2j-1}..s_{2N}$ se emparejan de a dos consecutivos y aportan $s_{2j-1} + s_{2j+1} + \dots$ al ancho. Se prueba cada j entre 2 y $N+1$ con una suma prefija y una suma alternada desde el final, y se toma el mínimo de $(s_1+s_j) \cdot \text{ancho}(j)$. Todo cabe en long long.

Python 3

(no disponible)

C++11

(no disponible)

33. J. Jinxed Jewelry

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** Greedy / ordenamiento **Tiempo:** $O(n \log n)$ **Memoria:** $O(n)$
Archivo: Jinxed.py / Jinxed.cpp.py / .cpp **Estado:** local-ok

Solución: Cada eslabón abierto sirve para unir dos extremos. Si se abren x eslabones tomándolos de los extremos de las cadenas, quedan q piezas, y para cerrarlas en círculo hacen falta q uniones, así que se necesita $x \geq q$. Para reducir q conviene consumir por completo las cadenas más cortas. Si se consumen las j más cortas (P_j eslabones) quedan $n-j$ piezas y el costo es $\max(P_j, n-j)$; los eslabones que falten se sacan del extremo de otra cadena sin crear piezas nuevas. Se ordena y se toma el mínimo para $0 \leq j < n$.

Python 3

(no disponible)

C++11

(no disponible)

34. K. K.O. Kids II

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** Probabilidad / DP **Tiempo:** $O(n \cdot k)$ **Memoria:** $O(k)$ **Archivo:** Kids.py / Kids.cpp.py / .cpp **Estado:** local-ok

Solución: Estado: $\text{dist}[j]$ = probabilidad de que nadie haya ganado aún y el progreso global sea j obstáculos. Un niño parte de j , supera gratis esos y luego cada obstáculo i con probabilidad a_i . Se recorre con un acumulador $\text{acc}(i) = \text{acc}(i-1) \cdot a_i + \text{dist}[i]$: el niño termina con probabilidad $\text{acc}(k)$ y el nuevo progreso queda en i con probabilidad $\text{acc}(i) \cdot (1 - a_{i+1})$. Para la posición p , la probabilidad de que Glen gane es $\text{acc}(k)$ calculado sobre la distribución tras $p-1$ niños, así que basta simular los n niños y tomar el máximo. Son $n \cdot k = 10^7$ operaciones.

Python 3

(no disponible)

C++11

(no disponible)

35. L. Legendary LAN-Party

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** Greedy / intercambio (regla de Smith) **Tiempo:** $O(n \log n)$ **Memoria:** $O(n)$ **Archivo:** Legendary.py / Legendary.cpp.py / .cpp **Estado:** local-ok

Solución: El switch k -ésimo paga $c_k \cdot (1 + \text{suma de } m \text{ de los anteriores})$. Por el argumento de intercambio entre vecinos, i debe ir antes que j cuando $m_i \cdot c_j < m_j \cdot c_i$, es decir, se ordena por m/c ascendente (regla de Smith de scheduling con pesos). La comparación se hace con productos enteros para evitar errores de punto flotante. Después se recorre el orden acumulando la posición y sumando $c \cdot \text{posición}$. Los valores llegan a unos $2 \cdot 10^{17}$, que caben en long long.

Python 3

(no disponible)

C++11

(no disponible)

36. M. Massive Mountains

Evento: RPC 09 - 2024 (Competitive Programming Network, 9th Activity) **Tema:** Grafos / Dijkstra sobre pares + Floyd **Tiempo:** $O(n^4)$ **Memoria:** $O(n^2)$ **Archivo:** Massive.py / Massive.cpp.py / .cpp **Estado:** local-ok

Solución: Entre dos instantes en que ambos están quietos, si uno viaja con la compañía X el otro solo puede usar Y, y así sigue hasta que ambos queden quietos a la vez. Entonces cada fase es: A recorre un camino solo de X y B uno solo de Y (o al revés), y la fase dura el máximo de los dos. Con Floyd se calculan DX y DY (distancia 0 a sí mismo). Luego se corre un Dijkstra denso sobre pares (u, v) con transición $(u,v) \rightarrow (u',v')$ de costo $\min(\max(DX[u][u'], DY[v][v']), \max(DY[u][u'], DX[v][v']))$. Son 5625 estados con 5625 transiciones cada uno.

Python 3

(no disponible)

C++11

(no disponible)

37. -. ACM (caminata del equipo)

Evento: Maratón ACIS 2017 (práctica 2024) **Tema:** Agrupación / sumas **Tiempo:** $O(S \log S)$ **Memoria:** $O(S)$ **Archivo:** Acm.py / .cpp **Estado:** local-ok

Solución: Siguiendo el código del usuario, los tramos se agrupan por su punto de partida y se suma la longitud total de cada grupo. Como el equipo debe ir junto, su velocidad es la mínima de la lista. Para cada grupo el tiempo es la suma dividida entre esa velocidad, redondeado hacia arriba; se calcula en enteros como $(t + v - 1) / v$ para evitar errores de punto flotante. La respuesta es el mínimo entre todos los grupos. El original arrancaba el mínimo en 200 (podía quedarse corto) y tenía un while True sin manejo de EOF; ambas cosas se corrigieron.

Python 3

```
# Sumar las longitudes de los tramos de cada punto de partida; el equipo camina a la
# velocidad del mas lento. Respuesta: min sobre puntos de ceil(suma / vmin). Varios casos hasta EOF.
import sys
lines = sys.stdin.read().split("\n")
out, p = [], 0
while p < len(lines) and lines[p].strip():
    n, s = map(int, lines[p].split()); p += 1
    tot = {}
    for _ in range(s):
        a, b, d = map(int, lines[p].split()); p += 1
        tot[a] = tot.get(a, 0) + d
    v = min(map(int, lines[p].split())); p += 1
    out.append(str(min((t + v - 1) // v for t in tot.values())))
print("\n".join(out))
```

C++11

```
// Sumar las longitudes de los tramos de cada punto de partida; el equipo camina a la
// velocidad del mas lento. Respuesta: min sobre puntos de ceil(suma / vmin). Varios casos hasta EOF.
#include <bits/stdc++.h>
using namespace std;
int main() {
    string line;
    while (getline(cin, line)) {
        istream h(line); long long n, s;
        if (!(h >> n >> s)) break;
        map<long long, long long> tot;
        for (int i = 0; i < s; i++) {
            getline(cin, line); istream r(line);
            long long a, b, d; r >> a >> b >> d; tot[a] += d;
        }
        getline(cin, line); istream r(line);
        long long v = LLONG_MAX, x;
        while (r >> x) v = min(v, x);
        long long best = LLONG_MAX;
        for (auto &e : tot) best = min(best, (e.second + v - 1) / v);
        cout << best << "\n";
    }
}
```

38. -. Balance

Evento: Maratón ACIS 2017 (práctica 2024) **Tema:** Fuerza bruta **Tiempo:** $O(C^3)$ **Memoria:** $O(1)$ **Archivo:** Balance.py / .cpp **Estado:** local-ok

Solución: Se conservan las cuatro condiciones del código original. Si $a = i \cdot t_1$, $b = j \cdot t_2$ y $d = k \cdot t_3$, la balanza queda equilibrada cuando $a + b + d = W$ (todas las pesas en el otro platillo) o cuando un tipo va con el objeto: $W + a = b + d$, $W + b = a + d$ o $W + d = a + b$. Se recorren todas las ternas con tres ciclos anidados y se cuenta cada terna una sola vez aunque cumpla varias condiciones. Con C pequeño (unas decenas) la fuerza bruta $O(C^3)$ es suficiente. Si C fuera grande, se podría fijar (i, j) y despejar k en $O(1)$.

Python 3

```
# Fuerza bruta sobre (i, j, k) copias de cada pesa: la balanza queda equilibrada si todas
# van al otro platillo o si una sola clase acompaña al objeto.  $O(C^3)$ ,  $C$  pequeño.
import sys
c, w, t1, t2, t3 = map(int, sys.stdin.read().split())
r = 0
for i in range(c + 1):
    for j in range(c + 1):
        for k in range(c + 1):
            a, b, d = i * t1, j * t2, k * t3
            if a + b + d == w or w + a == b + d or w + b == a + d or w + d == a + b:
                r += 1
print(r)
```

C++11

```
// Fuerza bruta sobre (i, j, k) copias de cada pesa: la balanza queda equilibrada si todas
// van al otro platillo o si una sola clase acompaña al objeto.  $O(C^3)$ ,  $C$  pequeño.
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long c, w, t1, t2, t3, r = 0;
    cin >> c >> w >> t1 >> t2 >> t3;
    for (int i = 0; i <= c; i++)
        for (int j = 0; j <= c; j++)
            for (int k = 0; k <= c; k++) {
                long long a = i * t1, b = j * t2, d = k * t3;
                if (a + b + d == w || w + a == b + d || w + b == a + d || w + d == a + b) r++;
            }
    cout << r << "\n";
}
```

39. A. Fibonacci divisibility

Evento: Ejercicios propios 2024 **Tema:** Fibonacci modular **Tiempo:** $O(n \cdot P(M))$ **Memoria:** $O(1)$ **Archivo:** FibonacciDivisibility.py / .cpp **Estado:** local-ok

Solución: El código original usaba una lista fija de 29 Fibonacci, que falla para M grandes. Basta con generar la sucesión módulo M : guardamos el par $(F(i), F(i+1))$ reducido módulo M y avanzamos hasta que $F(i) \equiv 0$. Trabajar módulo M mantiene los números pequeños y es correcto porque la divisibilidad solo depende del residuo. La sucesión módulo M es periódica (período de Pisano) y siempre contiene un cero, así que el ciclo termina, a lo sumo en unas $6M$ iteraciones. Para cada consulta el costo es lineal en el índice encontrado, con memoria constante. Las respuestas se imprimen en una línea separadas por espacio.

Python 3

```
# Para cada M, avanzamos Fibonacci modulo M hasta encontrar  $F(i) \% M == 0$  con  $i > 0$ .
import sys
d = sys.stdin.read().split()
out = []
for m in map(int, d[1:1 + int(d[0])]):
    a, b, i = 1 % m, 1 % m, 1
    while a:
        a, b, i = b, (a + b) % m, i + 1
    out.append(str(i))
print(" ".join(out))
```

C++11

```
// Para cada M, avanzamos Fibonacci modulo M hasta encontrar  $F(i) \% M == 0$  con  $i > 0$ .
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    for (int k = 0; k < n; k++) {
        long long m; cin >> m;
        long long a = 1 % m, b = 1 % m, i = 1;
        while (a) { long long c = (a + b) % m; a = b; b = c; i++; }
        cout << (k ? " " : "") << i;
    }
}
```

```

    }
    cout << "\n";
}

```

40. B. Greatest common divisor

Evento: Ejercicios propios 2024 **Tema:** Euclides **Tiempo:** $O(n \log V)$ **Memoria:** $O(1)$ **Archivo:** GreatestCommonDivisor.py / .cpp **Estado:** local-ok

Solución: El usuario usaba el algoritmo de Euclides por restas, que puede ser muy lento si un número es mucho mayor que el otro. Aquí se usa la versión con módulo: $\text{mcd}(a, b) = \text{mcd}(b, a \bmod b)$, que termina en $O(\log)$ pasos. El mínimo común múltiplo se obtiene con la identidad $\text{mcd} \cdot \text{mcm} = a \cdot b$; para evitar desbordes en C++ se calcula como $a / \text{mcd} \cdot b$, dividiendo primero. En Python se usa `math.gcd`. Cada par se imprime entre paréntesis y los pares se separan con un espacio, todo en una sola línea de salida.

Python 3

```

# Euclides para el MCD; MCM = a / mcd * b.
import sys
from math import gcd
d = list(map(int, sys.stdin.read().split()))
out = []
for k in range(d[0]):
    a, b = d[1 + 2 * k], d[2 + 2 * k]
    g = gcd(a, b)
    out.append("(%d %d)" % (g, a // g * b))
print(" ".join(out))

```

C++11

```

// Euclides para el MCD; MCM = a / mcd * b.
#include <bits/stdc++.h>
using namespace std;
long long g(long long a, long long b) { return b ? g(b, a % b) : a; }
int main() {
    int n; cin >> n;
    for (int k = 0; k < n; k++) {
        long long a, b; cin >> a >> b;
        long long d = g(a, b);
        cout << (k ? " " : "") << "(" << d << " " << a / d * b << ")";
    }
    cout << "\n";
}

```

41. C. Pitágoras (tipo de triángulo)

Evento: Ejercicios propios 2024 **Tema:** Geometría entera **Tiempo:** $O(n)$ **Memoria:** $O(1)$ **Archivo:** Pitagoras.py / .cpp **Estado:** local-ok

Solución: El código original usaba `sqrt` y `pow` en punto flotante y truncaba a entero, lo que puede clasificar mal. La versión nueva compara en enteros c^2 con $a^2 + b^2$: si son iguales el ángulo opuesto a c es recto (R), si c^2 es menor el ángulo es agudo (A) y si es mayor es obtuso (O), por la ley de los cosenos. Usamos `long long` para que los cuadrados no desborden. Se quitaron los mensajes de entrada ("Ingresa...") porque un juez solo espera la salida pedida. Las letras se imprimen separadas por espacios en una línea.

Python 3

```

# Comparar c^2 con a^2 + b^2 en enteros: igual -> R, menor -> A (agudo), mayor -> O (obtuso).
import sys
d = list(map(int, sys.stdin.read().split()))
out = []
for k in range(d[0]):
    a, b, c = d[1 + 3 * k:4 + 3 * k]
    s = a * a + b * b
    out.append("R" if c * c == s else "A" if c * c < s else "O")
print(" ".join(out))

```

C++11

```

// Comparar c^2 con a^2 + b^2 en enteros: igual -> R, menor -> A (agudo), mayor -> O (obtuso).
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    for (int k = 0; k < n; k++) {
        long long a, b, c; cin >> a >> b >> c;
        long long s = a * a + b * b;
        cout << (k ? " " : "") << (c * c == s ? 'R' : (c * c < s ? 'A' : 'O'));
    }
}

```

```
    cout << "\n";
}
```

42. D. Sum of digits

Evento: Ejercicios propios 2024 **Tema:** Aritmética **Tiempo:** $O(n \cdot d)$ **Memoria:** $O(1)$ **Archivo:** SumOfDigits.py / .cpp **Estado:** local-ok

Solución: Es simulación directa. Se calcula $v = a \cdot b + c$ y se suman sus dígitos tomando $v \bmod 10$ y dividiendo entre 10 hasta llegar a cero, igual que en el código original. En Python basta con convertir el número a cadena y sumar cada carácter como entero. En C++ se usa long long para que el producto no desborde. Si v es 0 la suma es 0, y ambos métodos lo manejan sin caso especial. La complejidad es lineal en la cantidad de dígitos de cada número. Los resultados van en una línea separados por espacio.

Python 3

```
# Calcular a*b+c y sumar sus digitos.
import sys
d = list(map(int, sys.stdin.read().split()))
out = []
for k in range(d[0]):
    a, b, c = d[1 + 3 * k:4 + 3 * k]
    out.append(str(sum(map(int, str(a * b + c)))))
print(" ".join(out))
```

C++11

```
// Calcular a*b+c y sumar sus digitos.
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    for (int k = 0; k < n; k++) {
        long long a, b, c; cin >> a >> b >> c;
        long long v = a * b + c, s = 0;
        for (; v > 0; v /= 10) s += v % 10;
        cout << (k ? " " : "") << s;
    }
    cout << "\n";
}
```

43. E. Maximum of array

Evento: Ejercicios propios 2024 **Tema:** Recorrido lineal **Tiempo:** $O(n)$ **Memoria:** $O(1)$ en C++ **Archivo:** MaximumOfArray.py / .cpp **Estado:** local-ok

Solución: Un solo recorrido basta: mantenemos el mayor y el menor visto hasta el momento y los actualizamos con cada número leído. En C++ se lee hasta EOF con while ($\text{cin} \gg x$), sin guardar el arreglo, así que la memoria es constante; los valores iniciales son LLONG_MIN y LLONG_MAX para que el primer número los reemplace. En Python se leen todos los tokens y se usan max y min, que también son lineales. Es la misma idea que el código original del usuario. La salida es "max min" en una línea.

Python 3

```
# Recorrer todos los numeros guardando maximo y minimo.
import sys
v = list(map(int, sys.stdin.read().split()))
print(max(v), min(v))
```

C++11

```
// Recorrer todos los numeros guardando maximo y minimo.
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long x, mx = LLONG_MIN, mn = LLONG_MAX;
    while (cin >> x) { mx = max(mx, x); mn = min(mn, x); }
    cout << mx << " " << mn << "\n";
}
```

44. F. Median of three

Evento: Ejercicios propios 2024 **Tema:** Ordenamiento **Tiempo:** $O(n)$ **Memoria:** $O(1)$ **Archivo:** MedianOfThree.py / .cpp **Estado:** local-ok

Solución: Con solo tres números lo más simple es ordenarlos y tomar el del centro. Ordenar tres elementos cuesta tiempo constante, así que el total es lineal en la cantidad de tríos. Se podría resolver con comparaciones ($\max(\min(a,b), \min(\max(a,b), c))$), pero ordenar es más claro y difícil de equivocar. Los valores repetidos no causan problemas: si dos son iguales, uno de ellos queda en el medio. Las medianas se imprimen separadas por espacios en una sola línea, igual que en la solución original del usuario.

Python 3

```
# Ordenar los tres numeros y tomar el del medio.
import sys
d = list(map(int, sys.stdin.read().split()))
print(" ".join(str(sorted(d[1 + 3 * k:4 + 3 * k])[1]) for k in range(d[0])))
```

C++11

```
// Ordenar los tres numeros y tomar el del medio.
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    for (int k = 0; k < n; k++) {
        long long v[3]; cin >> v[0] >> v[1] >> v[2];
        sort(v, v + 3);
        cout << (k ? " " : "") << v[1];
    }
    cout << "\n";
}
```

45. G. Minimum of two

Evento: Ejercicios propios 2024 **Tema:** Comparación **Tiempo:** $O(n)$ **Memoria:** $O(1)$ **Archivo:** MinimumOfTwo.py / .cpp **Estado:** local-ok

Solución: Basta comparar los dos números de cada par y quedarse con el menor (min). El código original tenía dos problemas para un juez: imprimía mensajes como "Ingresa el número..." y, si los números eran iguales, escribía un texto en vez del valor. Si $a = b$ el mínimo es ese mismo valor, así que no hace falta un caso especial. Se quitaron los mensajes y las respuestas se imprimen en una línea separadas por espacios. Se usa long long para soportar valores grandes y negativos.

Python 3

```
# Imprimir el menor de cada par.
import sys
d = list(map(int, sys.stdin.read().split()))
print(" ".join(str(min(d[1 + 2 * k], d[2 + 2 * k])) for k in range(d[0])))
```

C++11

```
// Imprimir el menor de cada par.
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    for (int k = 0; k < n; k++) {
        long long a, b; cin >> a >> b;
        cout << (k ? " " : "") << min(a, b);
    }
    cout << "\n";
}
```

46. H. Triangles

Evento: Ejercicios propios 2024 **Tema:** Desigualdad triangular **Tiempo:** $O(n)$ **Memoria:** $O(1)$ **Archivo:** Triangles.py / .cpp **Estado:** local-ok

Solución: Tres segmentos forman un triángulo si y solo si cada uno es estrictamente menor que la suma de los otros dos (desigualdad triangular). El código original solo revisaba dos de las tres condiciones (faltaba $a + c > b$), así que fallaba con entradas como 1 10 1. Aquí se revisan las tres. La desigualdad estricta descarta los triángulos degenerados (por ejemplo 1 2 3, donde los tres puntos quedan alineados). Se usa long long para que las sumas no desborden, y los resultados 1/0 se imprimen separados por espacio.

Python 3

```
# Hay triángulo si cada lado es menor que la suma de los otros dos.
import sys
d = list(map(int, sys.stdin.read().split()))
```

```

out = []
for k in range(d[0]):
    a, b, c = d[1 + 3 * k:4 + 3 * k]
    out.append("1" if a + b > c and a + c > b and b + c > a else "0")
print(" ".join(out))

```

C++11

```

// Hay triangulo si cada lado es menor que la suma de los otros dos.
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    for (int k = 0; k < n; k++) {
        long long a, b, c; cin >> a >> b >> c;
        cout << (k ? " " : "") << (a + b > c && a + c > b && b + c > a ? 1 : 0);
    }
    cout << "\n";
}

```

47. -. Duel

Evento: Maratón — 26 de abril de 2025 **Tema:** Greedy / dos punteros **Tiempo:** $O(n \log n)$ **Memoria:** $O(n)$ **Archivo:** Duel.py / .cpp **Estado:** local-ok

Solución: Se ordenan las dos manos. Para calcular el máximo de victorias de una mano H1 contra H2 se usa un greedy: recorremos H1 de menor a mayor con un puntero w sobre la carta más pequeña de H2 que aún no se ha vencido; si la carta actual de H1 la supera, sumamos una victoria y avanzamos w. Si no la supera, esa carta se usa para perder contra una carta grande. Así, el máximo de Alice es $\text{wins}(\text{Alice}, \text{Bob})$. Su mínimo es $n - \text{wins}(\text{Bob}, \text{Alice})$, porque cada ronda la gana uno de los dos. El intento manual del usuario solo probaba rotaciones; esta es la idea de Duel_AI.py simplificada.

Python 3

```

# Greedy con dos punteros sobre manos ordenadas: max victorias de Alice = emparejamientos
# donde su carta supera a la de Bob; min de Alice = n - max victorias de Bob.
import sys
d = list(map(int, sys.stdin.read().split()))
n = d[0]
alice = sorted(d[1:n + 1])
s = set(alice)
bob = [x for x in range(1, 2 * n + 1) if x not in s]
def wins(h1, h2):
    w = 0
    for x in h1:
        if x > h2[w]:
            w += 1
    return w
print(n - wins(bob, alice), wins(alice, bob))

```

C++11

```

// Greedy con dos punteros sobre manos ordenadas: max victorias de Alice = emparejamientos
// donde su carta supera a la de Bob; min de Alice = n - max victorias de Bob.
#include <bits/stdc++.h>
using namespace std;
int wins(const vector<int>& h1, const vector<int>& h2) {
    int w = 0;
    for (int x : h1) if (x > h2[w]) w++;
    return w;
}
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<int> a(n), b; vector<bool> has(2 * n + 1);
    for (int &x : a) { cin >> x; has[x] = true; }
    sort(a.begin(), a.end());
    for (int x = 1; x <= 2 * n; x++) if (!has[x]) b.push_back(x);
    cout << n - wins(b, a) << " " << wins(a, b) << "\n";
}

```

48. A. Intentional Blank Page

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** Ad hoc **Tiempo:** O(p) **Memoria:** O(1) **Archivo:** Intentional.py / .cpp **Estado:** local-ok

Solución: Cada problema empieza en una página frontal. Si tiene un número par de páginas termina en una trasera y el siguiente arranca de frente sin desperdicio; si tiene un número impar, su última página queda en el frente y la trasera correspondiente se deja en blanco con el mensaje. Por lo tanto la respuesta es simplemente la cantidad de problemas con número impar de páginas. Se lee la lista y se suma $x \bmod 2$ de cada valor. No hay casos especiales: incluso el último problema cuenta si es impar, como muestra el tercer ejemplo con un solo problema de una página.

Python 3

```
# Cada problema con numero impar de paginas deja una pagina en blanco.
import sys
a = list(map(int, sys.stdin.read().split()))
print(sum(x % 2 for x in a[1:a[0] + 1]))
```

C++11

```
// Cada problema con numero impar de paginas deja una pagina en blanco.
#include <bits/stdc++.h>
using namespace std;
int main() {
    int p, x, c = 0;
    cin >> p;
    while (p-- > 0) { cin >> x; c += x % 2; }
    cout << c << "\n";
}
```

49. B. Full House

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** Ad hoc, conteo **Tiempo:** O(1) **Memoria:** O(1) **Archivo:** Full.py / .cpp **Estado:** local-ok

Solución: Se cuentan las apariciones de cada dígito del número tratado como cadena. Un full house corresponde exactamente a que las frecuencias, ordenadas de menor a mayor, sean [2, 3]: dos valores distintos, uno repetido dos veces y otro tres. Cualquier otra distribución falla: cinco iguales da [5], cuatro y uno da [1, 4], tres y dos distintos da [1, 1, 3], etc. Con un Counter en Python o un map en C++ se obtienen las frecuencias, se ordenan y se comparan contra la lista objetivo. Es O(1) porque siempre hay cinco dígitos.

Python 3

```
# Full house: las frecuencias de los digitos ordenadas son exactamente [2, 3].
from collections import Counter
s = input().strip()
print("YES" if sorted(Counter(s).values()) == [2, 3] else "NO")
```

C++11

```
// Full house: las frecuencias de los digitos ordenadas son exactamente [2, 3].
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s; cin >> s;
    map<char, int> f;
    for (char ch : s) f[ch]++;
    vector<int> v;
    for (auto &e : f) v.push_back(e.second);
    sort(v.begin(), v.end());
    cout << (v == vector<int>{2, 3} ? "YES" : "NO") << "\n";
}
```

50. C. Duck Stress Balls

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** Greedy, conteo **Tiempo:** O(c) **Memoria:** O(1) **Archivo:** Duck.py / .cpp **Estado:** local-ok

Solución: Cada equipo puede recibir a lo sumo un pato de cada color, así que un color con más de n patos hace imposible el reparto. Esa condición también es suficiente: repartiendo siempre un pato de cada uno de los tres colores más abundantes se mantiene la invariante máximo $\leq$ equipos restantes (argumento clásico de intercambio) hasta agotar todo. Por eso basta verificar $3 \cdot \max \leq \text{total}$. Se validó contra una simulación greedy en casos aleatorios. El segundo ejemplo falla porque 59 y 60 superan los 40 equipos.

Python 3

```
# Con n = total/3 equipos, se puede repartir sii ningun color tiene mas de n patos.
import sys
a = list(map(int, sys.stdin.read().split()))
d = a[1:a[0] + 1]
print("YES" if 3 * max(d) <= sum(d) else "NO")
```

C++11

```
// Con n = total/3 equipos, se puede repartir sii ningun color tiene mas de n patos.
#include <bits/stdc++.h>
using namespace std;
int main() {
    int c, x, mx = 0, s = 0;
    cin >> c;
    while (c--) { cin >> x; s += x; mx = max(mx, x); }
    cout << (3 * mx <= s ? "YES" : "NO") << "\n";
}
```

51. D. Twin Numbers

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** Matemáticas, conteo por dígitos **Tiempo:** O(len) **Memoria:** O(len)
Archivo: Twin.py / .cpp **Estado:** local-ok

Solución: Un twin de 2m dígitos queda determinado por su mitad h de m dígitos sin cero inicial. Se define $f(X) = \text{twins} \leq X$. Los de menor longitud suman $9 \cdot (1 + 10 + \dots + 10^{t-1}) = 10^t - 1$, con t la mitad de la mayor longitud par menor que la de X. Si X tiene longitud impar no hay más. Si es par, se recorren los pares de X: mientras ambos dígitos coinciden se copian en h; en el primer par distinto (a, b) se toma a si $a < b$ o $a-1$ si $a > b$ y se completa con nueves. Todo se lleva mod 10007. La respuesta es $f(h_i) - f(h_o) + [\text{lo es twin}]$.

Python 3

```
# Un twin de 2m digitos equivale a su "mitad" de m digitos (sin cero inicial).
# f(X) = twins <= X: todos los de longitud menor ( $10^m - 1$ ) + mitades h con twin(h) <= X.
import sys
M = 10007

def f(x):
    L = len(x)
    full = pow(10, L // 2 - (L % 2 == 0), M) - 1 # twins con menos digitos
    if L % 2:
        return full
    h, ok = 0, True # h = mayor mitad valida (mod M); ok = no se anula
    m = L // 2
    for i in range(m):
        a, b = x[2 * i], x[2 * i + 1]
        if a == b:
            h = (h * 10 + int(a)) % M
            continue
        d = int(a) if a < b else int(a) - 1
        if i == 0 and d == 0:
            ok = False
        h = (h * 10 + d) % M
        h = (h * pow(10, m - i - 1, M) + pow(10, m - i - 1, M) - 1) % M # resto con 9s
        break
    if not ok:
        return full
    return (full + h - pow(10, m - 1, M) + 1) % M

def twin(x):
    return len(x) % 2 == 0 and all(x[i] == x[i + 1] for i in range(0, len(x), 2))

lo, hi = sys.stdin.read().split()
print((f(hi) - f(lo) + twin(lo)) % M)
```

C++11

```
// Un twin de 2m digitos equivale a su "mitad" de m digitos (sin cero inicial).
// f(X) = twins <= X: todos los de longitud menor ( $10^m - 1$ ) + mitades h con twin(h) <= X.
#include <bits/stdc++.h>
using namespace std;
const int M = 10007;
long long pw(long long b, long long e) { long long r = 1; b %= M; while (e) { if (e & 1) r = r * b % M; b = b * b % M; e >>= 1; } return r; }
long long f(const string &x) {
    int L = x.size();
    long long full = (pw(10, L / 2 - (L % 2 == 0)) - 1 + M) % M;
    if (L % 2) return full;
    int m = L / 2;
    long long h = 0;
    for (int i = 0; i < m; i++) {
        char a = x[2 * i], b = x[2 * i + 1];
        if (a == b) { h = (h * 10 + a - '0') % M; continue; }
    }
```

```

        int d = a < b ? a - '0' : a - '0' - 1;
        if (i == 0 && d == 0) return full;
        long long p = pw(10, m - i - 1);
        h = ((h * 10 + d) % M * p + p - 1) % M;
        break;
    }
    return ((full + h - pw(10, m - 1) + 1) % M + M) % M;
}
bool twin(const string &x) {
    if (x.size() % 2) return false;
    for (size_t i = 0; i < x.size(); i += 2) if (x[i] != x[i + 1]) return false;
    return true;
}
int main() {
    string lo, hi; cin >> lo >> hi;
    cout << ((f(hi) - f(lo) + twin(lo)) % M + M) % M << "\n";
}

```

52. E. Simple House

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** Geometría, búsqueda ternaria **Tiempo:** $O(l^2)$, $l = 100$ iteraciones
Memoria: $O(1)$ **Archivo:** Simple.py / .cpp **Estado:** local-ok

Solución: La casa es un rectángulo de lados $w \times t$ con centro (x, y) . Un rectángulo alineado cabe en un polígono convexo si sus cuatro esquinas cumplen cada semiplano; para el lado con normal (a, b) eso equivale a $a \cdot x + b \cdot y + t \cdot (|a|w + |b|h) / 2 \leq c$. Fijado el centro, el mayor t es el mínimo de cuatro funciones lineales, que es cóncavo en (x, y) . Por eso se hace búsqueda ternaria anidada: en x por fuera y en y por dentro, 100 iteraciones cada una. La orientación de cada semiplano se fija usando el centroide de los vértices. El área es $w \cdot h \cdot t^2$.

Python 3

```

# Casa = rectangulo w*t x h*t centrado en (x, y). Para un centro fijo, el t maximo es el
# minimo sobre los 4 lados de una funcion lineal -> concava; ternaria anidada en x e y.
import sys
v = list(map(int, sys.stdin.read().split()))
P = [(v[2 * i], v[2 * i + 1]) for i in range(4)]
w, h = v[8], v[9]
gx = sum(p[0] for p in P) / 4; gy = sum(p[1] for p in P) / 4
E = [] # semiplanos a*x + b*y <= c
for i in range(4):
    (x1, y1), (x2, y2) = P[i], P[(i + 1) % 4]
    a, b = y2 - y1, x1 - x2
    c = a * x1 + b * y1
    if a * gx + b * gy > c:
        a, b, c = -a, -b, -c
    E.append((a, b, c, (abs(a) * w + abs(b) * h) / 2))

def best_t(x, y):
    return min((c - a * x - b * y) / k for a, b, c, k in E)

def tern(lo, hi, g):
    for _ in range(100):
        m1 = lo + (hi - lo) / 3; m2 = hi - (hi - lo) / 3
        if g(m1) < g(m2): lo = m1
        else: hi = m2
    return g((lo + hi) / 2)

xs = [p[0] for p in P]; ys = [p[1] for p in P]
t = tern(min(xs), max(xs), lambda x: tern(min(ys), max(ys), lambda y: best_t(x, y)))
t = max(t, 0.0)
print("%.9f" % (w * h * t * t))

```

C++11

```

// Casa = rectangulo w*t x h*t centrado en (x, y). Para un centro fijo, el t maximo es el
// minimo sobre los 4 lados de una funcion lineal -> concava; ternaria anidada en x e y.
#include <bits/stdc++.h>
using namespace std;
double A[4], B[4], C[4], K[4];
double bestT(double x, double y) {
    double r = 1e18;
    for (int i = 0; i < 4; i++) r = min(r, (C[i] - A[i] * x - B[i] * y) / K[i]);
    return r;
}
template <class F> double tern(double lo, double hi, F g) {
    for (int it = 0; it < 100; it++) {
        double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo) / 3;
        if (g(m1) < g(m2)) lo = m1; else hi = m2;
    }
    return g((lo + hi) / 2);
}

```

```

int main() {
    double x[4], y[4], w, h, gx = 0, gy = 0;
    for (int i = 0; i < 4; i++) { cin >> x[i] >> y[i]; gx += x[i] / 4; gy += y[i] / 4; }
    cin >> w >> h;
    for (int i = 0; i < 4; i++) {
        int j = (i + 1) % 4;
        double a = y[j] - y[i], b = x[i] - x[j], c = a * x[i] + b * y[i];
        if (a * gx + b * gy > c) { a = -a; b = -b; c = -c; }
        A[i] = a; B[i] = b; C[i] = c; K[i] = (fabs(a) * w + fabs(b) * h) / 2;
    }
    double x0 = *min_element(x, x + 4), x1 = *max_element(x, x + 4);
    double y0 = *min_element(y, y + 4), y1 = *max_element(y, y + 4);
    double t = tern(x0, x1, [&](double cx) { return tern(y0, y1, [&](double cy) { return bestT(cx, cy); }); });
    t = max(t, 0.0);
    printf("%.9f\n", w * h * t * t);
}

```

53. F. Pathfinder and Wolves

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** BFS multifuente, DSU (cuello de botella) **Tiempo:** $O(r^2c^2\alpha)$
Memoria: $O(r^2c)$ **Archivo:** Pathfinder.py / .cpp **Estado:** local-ok

Solución: Primero un BFS multifuente desde todos los lobos da la distancia de cada celda al lobo más cercano; la cola del BFS queda ordenada por distancia. Luego se resuelve un problema de camino de cuello de botella máximo: se activan las celdas de mayor a menor distancia (recorriendo la cola al revés) y se unen con sus vecinas ya activas en un DSU. La primera vez que (1,1) y (r,c) quedan en la misma componente, la distancia de la celda recién activada es la respuesta, porque existe un camino que solo usa celdas con al menos esa distancia. Las celdas con lobo nunca se activan.

Python 3

```

# BFS multi-fuente desde los lobos da la distancia de cada celda. Luego se activan celdas
# de mayor a menor distancia uniendolas con DSU; la respuesta es la distancia con la que
# (1,1) y (r,c) quedan conectados (maximo cuello de botella).
import sys
def main():
    data = sys.stdin.buffer.read().split()
    r, c = int(data[0]), int(data[1])
    g = b"".join(data[2:2 + r])
    N = r * c
    dist = [-1] * N
    q = [i for i in range(N) if g[i] == 87] # 'W'
    for i in q: dist[i] = 0
    h = 0
    while h < len(q):
        u = q[h]; h += 1
        d = dist[u] + 1; y, x = divmod(u, c)
        if x > 0 and dist[u - 1] < 0: dist[u - 1] = d; q.append(u - 1)
        if x < c - 1 and dist[u + 1] < 0: dist[u + 1] = d; q.append(u + 1)
        if y > 0 and dist[u - c] < 0: dist[u - c] = d; q.append(u - c)
        if y < r - 1 and dist[u + c] < 0: dist[u + c] = d; q.append(u + c)
    par = list(range(N)); on = [False] * N
    def find(a):
        while par[a] != a:
            par[a] = par[par[a]]; a = par[a]
        return a
    for u in reversed(q):
        if dist[u] == 0: break
        on[u] = True; y, x = divmod(u, c)
        for v, okv in ((u - 1, x > 0), (u + 1, x < c - 1), (u - c, y > 0), (u + c, y < r - 1)):
            if okv and on[v]:
                a, b = find(u), find(v)
                if a != b: par[a] = b
        if on[0] and on[N - 1] and find(0) == find(N - 1):
            print(dist[u]); return
main()

```

C++11

```

// BFS multi-fuente desde los lobos da la distancia de cada celda. Luego se activan celdas
// de mayor a menor distancia uniendolas con DSU; la respuesta es la distancia con la que
// (1,1) y (r,c) quedan conectados (maximo cuello de botella).
#include <bits/stdc++.h>
using namespace std;
vector<int> par;
int find(int a) { while (par[a] != a) a = par[a] = par[par[a]]; return a; }
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int r, c; cin >> r >> c;
    string g, s;
    for (int i = 0; i < r; i++) { cin >> s; g += s; }
    int N = r * c;

```

```

vector<int> dist(N, -1), q;
for (int i = 0; i < N; i++) if (g[i] == 'W') { dist[i] = 0; q.push_back(i); }
for (size_t h = 0; h < q.size(); h++) {
    int u = q[h], y = u / c, x = u % c, d = dist[u] + 1;
    int nb[4] = {x > 0 ? u - 1 : -1, x < c - 1 ? u + 1 : -1, y > 0 ? u - c : -1, y < r - 1 ? u + c : -1};
    for (int v : nb) if (v >= 0 && dist[v] < 0) { dist[v] = d; q.push_back(v); }
}
par.resize(N); iota(par.begin(), par.end(), 0);
vector<char> on(N, 0);
for (int k = N - 1; k >= 0; k--) {
    int u = q[k], y = u / c, x = u % c;
    on[u] = 1;
    int nb[4] = {x > 0 ? u - 1 : -1, x < c - 1 ? u + 1 : -1, y > 0 ? u - c : -1, y < r - 1 ? u + c : -1};
    for (int v : nb) if (v >= 0 && on[v]) par[find(u)] = find(v);
    if (on[0] && on[N - 1] && find(0) == find(N - 1)) { cout << dist[u] << "\n"; return 0; }
}
}
}

```

54. G. Festival of Blossoms

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** Combinatoria sobre cadenas (bordes, Guibas-Odlyzko) **Tiempo:** $O(N \cdot \text{bordes}(S))$ **Memoria:** $O(N)$ **Archivo:** Festival.py / .cpp **Estado:** local-ok

Solución: Se cuentan las que no contienen S y se restan de K^N . Sea $a[n]$ el número de cadenas de largo n sin S y $b[n]$ las que tienen su primera aparición de S terminando justo en n. Extender una cadena sin S con una letra da $K \cdot a[n-1] = a[n] + b[n]$. Pegar S al final de una cadena sin S de largo n-m produce una primera aparición que termina en n-m+j para algún borde j de S (prefijo igual a sufijo, incluido j = m), y el resto queda forzado: $a[n-m] = \text{suma de } b[n-m+j]$. Eso da $b[n]$ y luego $a[n]$. Los bordes se calculan directamente ($m \leq 1000$).

Python 3

```

# a[n] = cadenas de largo n sin S, b[n] = cadenas cuya primera aparicion de S termina en n.
# K*a[n-1] = a[n] + b[n] y a[n-m] = sum_{j borde de S} b[n-m+j] (Guibas-Odlyzko).
# Respuesta: K^N - a[N]. O(N * #bordes).
import sys
MOD = 10**9 + 7
def main():
    t = sys.stdin.read().split()
    N, K, S = int(t[0]), int(t[1]), t[2]
    m = len(S)
    J = [j for j in range(1, m) if S[j:] == S[m - j:]] # bordes propios
    a = [0] * (N + 1); b = [0] * (N + 1)
    a[0] = 1
    for n in range(1, N + 1):
        if n < m:
            a[n] = a[n - 1] * K % MOD
            continue
        o = n - m
        x = (a[o] - sum([b[o + j] for j in J])) % MOD
        b[n] = x
        a[n] = (a[n - 1] * K - x) % MOD
    print((pow(K, N, MOD) - a[N]) % MOD)
main()

```

C++11

```

// a[n] = cadenas de largo n sin S, b[n] = cadenas cuya primera aparicion de S termina en n.
// K*a[n-1] = a[n] + b[n] y a[n-m] = sum_{j borde de S} b[n-m+j] (Guibas-Odlyzko).
// Respuesta: K^N - a[N]. O(N * #bordes).
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;
int main() {
    long long N, K; string S;
    cin >> N >> K >> S;
    int m = S.size();
    vector<int> J;
    for (int j = 1; j < m; j++) if (S.compare(0, j, S, m - j, j) == 0) J.push_back(j);
    vector<long long> a(N + 1), b(N + 1, 0);
    a[0] = 1;
    long long pk = 1;
    for (int n = 1; n <= N; n++) {
        pk = pk * K % MOD;
        if (n < m) { a[n] = a[n - 1] * K % MOD; continue; }
        int o = n - m;
        long long x = a[o];
        for (int j : J) x -= b[o + j];
        x = (x % MOD + MOD) % MOD;
        b[n] = x;
        a[n] = ((a[n - 1] * K - x) % MOD + MOD) % MOD;
    }
}

```

```
    cout << ((pk - a[N]) % MOD + MOD) % MOD << "\n";
}
```

55. H. Birthday Bash

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** Probabilidad (coleccionista de cupones), tabla precalculada
Tiempo: $O(10^6)$ **Memoria:** $O(1000)$ **Archivo:** Birthday.py / .cpp **Estado:** local-ok

Solución: Es el problema del coleccionista de cupones: $E = n * H_n$ con $H_n = 1 + 1/2 + \dots + 1/n$. Como n llega a $2e9$ no se puede sumar en tiempo de ejecución, así que se precalculó offline (en C++, unos segundos) el valor $H_{\{i \cdot 10^6\}} \bmod p$ para $i = 0..1000$ y se incrustó como tabla en el código. En ejecución se suman los menos de 10^6 términos restantes como una sola fracción num/den (num = num*k + den, den = den*k) para usar un único inverso modular. Para $n > p$ se usa $1/k = 1/(k-p)$ y $H_{\{p-1\}} = 0$ (Wolstenholme); $n = p$ da 1.

Python 3

```
# Coleccionista de cupones: E = n * H_n (H_n = 1 + 1/2 + ... + 1/n) mod p.
# T[i] = H_{i*10^6} mod p precalculado offline; se suma el resto (< 10^6 terminos) como
# una fraccion num/den para usar un solo inverso. Si n >= p se usa 1/k = 1/(k-p) y H_{p-1} = 0.
import sys
P = 10**9 + 7
B = 10**6
T = [... 1001 valores precalculados: ver Birthday.py en el repositorio ...]
def H(n):
    q = n // B
    num, den = 0, 1
    for k in range(q * B + 1, n + 1):
        num = (num * k + den) % P
        den = den * k % P
    return (T[q] + num * pow(den, P - 2, P)) % P
n = int(sys.stdin.read())
if n == P:
    print(1)
else:
    print(n % P * H(n if n < P else n - P) % P)
```

C++11

```
// Coleccionista de cupones: E = n * H_n (H_n = 1 + 1/2 + ... + 1/n) mod p.
// T[i] = H_{i*10^6} mod p precalculado offline; se suma el resto (< 10^6 terminos) como
// una fraccion num/den para usar un solo inverso. Si n >= p se usa 1/k = 1/(k-p) y H_{p-1} = 0.
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll P = 1000000007, B = 1000000;
const ll T[] = [... 1001 valores precalculados: ver Birthday.cpp en el repositorio ...];
ll pw(ll b, ll e) { ll r = 1; b %= P; while (e) { if (e & 1) r = r * b % P; b = b * b % P; e >>= 1; } return r; }
ll H(ll n) {
    ll q = n / B, num = 0, den = 1;
    for (ll k = q * B + 1; k <= n; k++) { num = (num * k + den) % P; den = den * k % P; }
    return (T[q] + num * pw(den, P - 2)) % P;
}
int main() {
    ll n; cin >> n;
    if (n == P) { cout << 1 << "\n"; return 0; }
    cout << n % P * H(n < P ? n : n - P) % P << "\n";
}
```

56. I. Dark Things

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** Geometría, búsqueda binaria **Tiempo:** $O(k * l * n)$, $l = 55$
Memoria: $O(n)$ **Archivo:** Dark.py / .cpp **Estado:** local-ok

Solución: Sea $A(\psi)$ el área libre con ángulo entre $-\theta$ y ψ : $R^2/2 * (\psi + \theta)$ menos la parte de cada círculo que queda del lado de ángulo menor que ψ . Como ningún círculo contiene el origen y todo cabe en menos de 180 grados, esa parte es un segmento circular respecto a la recta por el origen con ángulo ψ : con distancia con signo $t = r \sin(\phi - \psi)$, el área es $\rho^2/2 * \arcsin(t/\rho) - t \sqrt{\rho^2 - t^2}$, recortando t a $[-\rho, \rho]$. Los círculos que no cruzan el rayo (intervalo angular $\phi \pm \arcsin(\rho/r)$) se resuelven sin trigonometría. A es creciente, así que cada ángulo de área $j * \text{Total}/k$ se halla por bisección.

Python 3

```
# A(ψ) = area libre con angulo en [-θ, ψ] = R^2/2*(ψ+θ) - parte de cada circulo
# del lado "angulo < ψ" de la recta por el origen (segmento circular). A es creciente:
# biseccion para cada objetivo j*Total/k.
import sys
from math import asin, acos, sin, sqrt, pi, radians, degrees
def main():
    v = list(map(int, sys.stdin.read().split()))
```

```

n, k, R, th = v[0], v[1], v[2], radians(v[3])
C = []
full0 = 0.0
for i in range(n):
    r, f, d = v[4 + 3 * i], radians(v[5 + 3 * i]), v[6 + 3 * i] / 2
    a = asin(d / r)
    C.append((f - a, f + a, r, f, d, pi * d * d))
total = R * R * th - sum(c[5] for c in C)
def area(psi):
    s = R * R / 2 * (psi + th)
    for lo, hi, r, f, d, ar in C:
        if psi >= hi: s -= ar
        elif psi > lo:
            t = r * sin(f - psi)
            t = max(-d, min(d, t))
            s -= d * d * acos(t / d) - t * sqrt(d * d - t * t)
    return s
out = []
lo = -th
for j in range(1, k):
    goal = total * j / k
    a, b = lo, th
    for _ in range(55):
        m = (a + b) / 2
        if area(m) < goal: a = m
        else: b = m
    lo = a
    out.append("%.10f" % (degrees(a) + 0.0 if abs(degrees(a)) > 5e-11 else 0.0))
print("\n".join(out))
main()

```

C++11

```

// A(psi) = area libre con angulo en [-theta, psi] = R^2/2*(psi+theta) - parte de cada circulo
// del lado "angulo < psi" de la recta por el origen (segmento circular). A es creciente:
// biseccion para cada objetivo j*Total/k.
#include <bits/stdc++.h>
using namespace std;
int n, k; double R, th;
vector<double> L0, HI, RR, F, D, AR;
double area(double psi) {
    double s = R * R / 2 * (psi + th);
    for (int i = 0; i < n; i++) {
        if (psi >= HI[i]) s -= AR[i];
        else if (psi > L0[i]) {
            double d = D[i], t = RR[i] * sin(F[i] - psi);
            t = max(-d, min(d, t));
            s -= d * d * acos(t / d) - t * sqrt(d * d - t * t);
        }
    }
    return s;
}
int main() {
    double deg;
    cin >> n >> k >> R >> deg;
    th = deg * M_PI / 180;
    double total = R * R * th;
    for (int i = 0; i < n; i++) {
        double r, f, d; cin >> r >> f >> d;
        f = f * M_PI / 180; d /= 2;
        double a = asin(d / r);
        L0.push_back(f - a); HI.push_back(f + a); RR.push_back(r); F.push_back(f); D.push_back(d);
        AR.push_back(M_PI * d * d); total -= M_PI * d * d;
    }
    double lo = -th;
    for (int j = 1; j < k; j++) {
        double goal = total * j / k, a = lo, b = th;
        for (int it = 0; it < 55; it++) {
            double m = (a + b) / 2;
            if (area(m) < goal) a = m; else b = m;
        }
        lo = a;
        double x = a * 180 / M_PI;
        if (fabs(x) <= 5e-11) x = 0;
        printf("%.10f\n", x);
    }
}

```

57. J. Galactic Voyage

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** Dijkstra sobre residuos, CRT **Tiempo:** $O(m * (S + P^2 K) \log m)$
Memoria: $O(m)$ **Archivo:** Galactic.py / .cpp **Estado:** local-ok

Solución: Con $m = \min(S)$, si x es alcanzable también lo es $x + t*m$, así que basta $d[r] =$ menor posición alcanzable con residuo $r \bmod m$ (idea de Frobenius). Aristas: sumar cualquier s_i , y saltos: desde $x = 0 \pmod{p_j}$ hasta $x + \delta = 0 \pmod{p_k}$, $\delta \leq K$. Para un residuo r con distancia d , el menor $x \geq d$ con $x = r \pmod{m}$ y esas dos congruencias se obtiene por CRT; las combinaciones (j, k, δ) se precálculan. Como el costo resultante es monótono en d , Dijkstra es válido. Si algún residuo queda inalcanzable hay infinitos (respuesta -1); si no, la respuesta es $\max(d) - m$, o -1 si es ≤ 0 .

Python 3

```
# Dijkstra sobre residuos mod m = min(S): d[r] = menor posicion alcanzable = r (mod m); desde ahi
# todo r + t*m tambien lo es. Aristas: +s_i, y saltos de agujero x -> x+delta (delta <= K) con
# x = 0 mod p_j, x+delta = 0 mod p_k: el menor x >= d[r] sale por CRT. Respuesta max(d) - m.
import sys, heapq
from math import gcd
def crt(a1, m1, a2, m2):
    g = gcd(m1, m2)
    if (a2 - a1) % g: return None
    l = m1 // g * m2
    t = (a2 - a1) // g * pow(m1 // g, -1, m2 // g) % (m2 // g)
    return (a1 + m1 * t) % l, l
def main():
    v = list(map(int, sys.stdin.read().split()))
    S, P, K = v[0], v[1], v[2]
    s = v[3:3 + S]; p = v[3 + S:3 + S + P]
    m = min(s)
    combos = set()
    for a in p:
        for b in p:
            for dl in range(1, K + 1):
                c = crt(0, a, -dl % b, b)
                if c: combos.add((c[0], c[1], dl))
    pre = [] # datos para combinar con x = r (mod m)
    for c2, M2, dl in combos:
        g = gcd(m, M2); Mg = M2 // g
        pre.append((c2, g, Mg, pow(m // g, -1, Mg) if Mg > 1 else 0, m * Mg, dl))
    INF = float("inf")
    d = [INF] * m; d[0] = 0
    pq = [(0, 0)]
    while pq:
        du, r = heapq.heappop(pq)
        if du > d[r]: continue
        for st in s:
            y = du + st
            if y < d[y % m]: d[y % m] = y; heapq.heappush(pq, (y, y % m))
        for c2, g, Mg, inv, L, dl in pre:
            diff = c2 - r
            if diff % g: continue
            x = r + m * (diff // g * inv % Mg)
            if x < du: x += (du - x + L - 1) // L * L
            y = x + dl
            if y < d[y % m]: d[y % m] = y; heapq.heappush(pq, (y, y % m))
    mx = max(d)
    print(-1 if mx == INF or mx - m <= 0 else mx - m)
main()
```

C++11

```
// Dijkstra sobre residuos mod m = min(S): d[r] = menor posicion alcanzable = r (mod m); desde ahi
// todo r + t*m tambien lo es. Aristas: +s_i, y saltos de agujero x -> x+delta (delta <= K) con
// x = 0 mod p_j, x+delta = 0 mod p_k: el menor x >= d[r] sale por CRT. Respuesta max(d) - m.
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
ll egcd(ll a, ll b, ll &x, ll &y) { if (!b) { x = 1; y = 0; return a; } ll g = egcd(b, a % b, y, x); y -= a / b * x;
return g; }
ll inv(ll a, ll md) { ll x, y; egcd(a % md, md, x, y); return ((x % md) + md) % md; }
ll md(ll a, ll b) { return (a % b + b) % b; }
struct C { ll c2, g, Mg, iv, L; int dl; };
int main() {
    int S, P, K; cin >> S >> P >> K;
    vector<ll> s(S), p(P);
    for (auto &x : s) cin >> x;
    for (auto &x : p) cin >> x;
    ll m = *min_element(s.begin(), s.end());
    set<array<ll, 3>> seen; vector<C> pre;
    for (ll a : p) for (ll b : p) for (int dl = 1; dl <= K; dl++) {
        ll g = __gcd(a, b), r2 = md(-dl, b); // x = 0 (mod a), x = r2 (mod b)
        if (r2 % g) continue;
```

```

    ll M2 = a / g * b, t = md((r2 / g) * inv(a / g, b / g), b / g), c2 = md(a * t, M2);
    if (!seen.insert({c2, M2, dl}).second) continue;
    ll g2 = __gcd(m, M2), Mg = M2 / g2;
    pre.push_back({c2, g2, Mg, Mg > 1 ? inv(m / g2, Mg) : 0, m * Mg, dl});
}
const ll INF = LLONG_MAX;
vector<ll> d(m, INF); d[0] = 0;
priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, greater<pair<ll, ll>>> pq;
pq.push({0, 0});
auto relax = [&](ll y) { if (y < d[y % m]) { d[y % m] = y; pq.push({y, y % m}); } };
while (!pq.empty()) {
    ll du = pq.top().first, r = pq.top().second; pq.pop();
    if (du > d[r]) continue;
    for (ll st : s) relax(du + st);
    for (auto &c : pre) {
        ll diff = c.c2 - r;
        if (md(diff, c.g)) continue;
        ll x = r + m * md((diff / c.g) % c.Mg * c.iv, c.Mg);
        if (x < du) x += (du - x + c.L - 1) / c.L * c.L;
        relax(x + c.dl);
    }
}
ll mx = *max_element(d.begin(), d.end());
cout << (mx == INF || mx - m <= 0 ? -1 : mx - m) << "\n";
}

```

58. K. Degenerating Walkways

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** Probabilidad en árboles, árbol virtual, rerooting **Tiempo:** $O((n + m) \log n)$ **Memoria:** $O(n \log n)$ **Archivo:** Degenerating.py / .cpp **Estado:** local-ok

Solución: Por linealidad, la respuesta es la suma de las probabilidades de recolectar en cada nodo b ; si x_{L+1} está en b (pos L), hay que llegar a b con nivel L . Esa masa viene de nodos con pos $L-1$ (o de los inicios, para $L = 0$) por caminos sin otros nodos de pos L , y cada nodo intermedio multiplica por $(p/q)^{L+1}/(\deg-1)$. Para cada nivel se construye el árbol virtual de $A_{L-1} \cup A_L$ (aristas comprimidas con prefijos de $1/(\deg-1)$) y se pasan mensajes de subida y bajada; los nodos de A_L absorben. Se guarda la masa por dirección de llegada para no emitirla de vuelta por la misma arista en el siguiente nivel.

Python 3

```

# Recolectar  $x_{L+1}$  en  $b$  (pos( $b$ )= $L$ ) exige llegar a  $b$  con nivel  $L$ : la masa viene de nodos con
# pos  $L-1$  por caminos sin otros nodos de pos  $L$ . Por cada nivel se arma el árbol virtual de
#  $A_{L-1} \cup A_L$  y se pasan mensajes (subida/bajada) con pesos  $\alpha^{L+1}/(\deg-1)$  por nodo
# intermedio; la masa que llega a  $b$  por cada dirección se guarda para el siguiente nivel.
import sys
MOD = 998244353
def main():
    v = list(map(int, sys.stdin.buffer.read().split()))
    n, m, k, p, q = v[:5]
    posof = {x: i for i, x in enumerate(v[5:5 + m])}
    pos = [posof.get(x, -1) for x in v[5 + m:5 + m + n]]
    inv_n = pow(n, MOD - 2, MOD)
    if n == 1:
        print(1 if pos[0] == 0 else 0); return
    adj = [[] for _ in range(n)]
    o = 5 + m + n
    for i in range(n - 1):
        a, b = v[o + 2 * i] - 1, v[o + 2 * i + 1] - 1
        adj[a].append(b); adj[b].append(a)
    al = p * pow(q, MOD - 2, MOD) % MOD
    deg = [len(x) for x in adj]
    inv = [0] * (n + 1); inv[1] = 1
    for i in range(2, n + 1): inv[i] = (MOD - (MOD // i) * inv[MOD % i] % MOD) % MOD
    # árbol enraizado en 0
    par = [-1] * n; dep = [0] * n; order = [0]; tin = [0] * n
    seen = [False] * n; seen[0] = True
    st = [0]; order = []
    while st:
        u = st.pop(); tin[u] = len(order); order.append(u)
        for w in adj[u]:
            if not seen[w]: seen[w] = True; par[w] = u; dep[w] = dep[u] + 1; st.append(w)
    LOG = max(1, (n).bit_length())
    upt = [par[:]]
    upt[0][0] = 0
    for j in range(1, LOG):
        pr = upt[j-1]; upt.append([pr[pr[x]] for x in range(n)])
    def jump(x, d):
        k2 = dep[x] - d; j = 0
        while k2:
            if k2 & 1: x = upt[j][x]
            k2 >>= 1; j += 1
        return x

```

```

def lca(a, b):
    if dep[a] < dep[b]: a, b = b, a
    a = jump(a, dep[b])
    if a == b: return a
    for j in range(LOG - 1, -1, -1):
        if upt[j][a] != upt[j][b]: a = upt[j][a]; b = upt[j][b]
    return par[a]
Q = [1] * n; Qi = [1] * n # productos de 1/(deg-1) y (deg-1) desde la raiz
for u in order[1:]:
    Q[u] = Q[par[u]] * inv[deg[u] - 1] % MOD; Qi[u] = Qi[par[u]] * (deg[u] - 1) % MOD
groups = {}
for x in range(n):
    if pos[x] >= 0: groups.setdefault(pos[x], []).append(x)

def run(nodes, vpar, ew, sink, pw, emit):
    # nodes en preorden; devuelve {sink: [(vecino virtual, masa)]}
    S = {x: 0 for x in nodes}; up = {}; down = {}
    for x in reversed(nodes[1:]):
        val = emit(x, vpar[x])
        if not sink(x): val += pw(x) * S[x]
        up[x] = val % MOD * ew[x] % MOD
        S[vpar[x]] += up[x]
    down[nodes[0]] = 0
    for x in nodes[1:]:
        u = vpar[x]
        val = emit(u, x)
        if not sink(u): val += pw(u) * ((S[u] - up[x] + down[u]) % MOD)
        down[x] = val % MOD * ew[x] % MOD
    res = {}
    for x in nodes:
        if sink(x):
            lst = [(x if False else vpar[x], down[x])] if x != nodes[0] else []
            res[x] = lst
    for x in nodes[1:]:
        if sink(vpar[x]): res[vpar[x]].append((x, up[x]))
    return res

ans = 0
A0 = set(groups.get(0, []))
# nivel 0: todo el arbol, cada nodo fuera de A0 es inicio posible
e0 = [0 if x in A0 else inv_n * al % MOD * inv[deg[x]] % MOD for x in range(n)]
r = run(order, par, [1] * n, lambda x: x in A0, lambda x: al * inv[deg[x] - 1] % MOD,
        lambda x, y: e0[x])
H = {}
for b, lst in r.items():
    H[b] = {"s": inv_n}
    for nb, ms in lst: H[b][nb] = (H[b].get(nb, 0) + ms) % MOD
L = 0
while H:
    ans = (ans + sum(sum(h.values()) for h in H.values())) % MOD
    L += 1
    if L not in groups: break
    f = pow(al, L + 1, MOD)
    sub = {}; tot = {}
    for b, h in H.items():
        sb = {}
        for d, ms in h.items():
            c = deg[b] if d == "s" else deg[b] - 1
            sb[d] = ms * f % MOD * inv[c] % MOD
        sub[b] = sb; tot[b] = sum(sb.values()) % MOD
    sinks = set(groups[L])
    pts = sorted(set(H) | sinks, key=lambda x: tin[x])
    allp = set(pts)
    for i in range(len(pts) - 1): allp.add(lca(pts[i], pts[i + 1]))
    nodes = sorted(allp, key=lambda x: tin[x])
    vpar = {}; ew = {}; stk = []
    for x in nodes:
        while stk and lca(stk[-1], x) != stk[-1]: stk.pop()
        if stk:
            u = stk[-1]; vpar[x] = u
            cnt = dep[x] - dep[u] - 1
            ew[x] = pow(f, cnt, MOD) * Q[par[x]] % MOD * Qi[u] % MOD
        stk.append(x)
    def realdir(u, y):
        return par[u] if vpar.get(u) == y else jump(y, dep[u] + 1)
    def emit(u, y):
        if u not in tot: return 0
        return tot[u] - sub[u].get(realdir(u, y), 0)
    r = run(nodes, vpar, ew, lambda x: x in sinks, lambda x: f * inv[deg[x] - 1] % MOD, emit)
    H = {}
    for b, lst in r.items():
        h = {}
        for nb, ms in lst:
            if ms:

```

```

        d = reldir(b, nb); h[d] = (h.get(d, 0) + ms) % MOD
        if h: H[b] = h
    print(ans % MOD)
main()

```

C++11

```

// Recolectar x_{L+1} en b (pos(b)=L) exige llegar a b con nivel L: la masa viene de nodos con
// pos L-1 por caminos sin otros nodos de pos L. Por cada nivel se arma el arbol virtual de
// A_{L-1} U A_L y se pasan mensajes (subida/bajada) con pesos  $\alpha^{L+1}/(\deg-1)$  por nodo
// intermedio; la masa que llega a b por cada direccion se guarda para el siguiente nivel.
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll MOD = 998244353;
ll pw(ll b, ll e) { ll r = 1; b %= MOD; if (b < 0) b += MOD; while (e) { if (e & 1) r = r * b % MOD; b = b * b % MOD; e >>= 1; } return r; }
int n, LOG;
vector<int> par, dep, tin, deg;
vector<vector<int>> adj, upt;
vector<ll> inv, Q, Qi;
int jump(int x, int d) { int k = dep[x] - d; for (int j = 0; k; j++, k >>= 1) if (k & 1) x = upt[j][x]; return x; }
int lca(int a, int b) {
    if (dep[a] < dep[b]) swap(a, b);
    a = jump(a, dep[b]);
    if (a == b) return a;
    for (int j = LOG - 1; j >= 0; j--) if (upt[j][a] != upt[j][b]) { a = upt[j][a]; b = upt[j][b]; }
    return par[a];
}
// datos por nodo del arbol actual (indexados por nodo real)
vector<ll> S, upm, downm, ew, tot;
vector<int> vpar;
vector<char> isSink;
map<int, vector<pair<int, ll>>> H; // sink -> (direccion real o -1 = inicio, masa)
map<int, map<int, ll>> sub; // fuente -> direccion -> masa emitida por esa entrada
int reldir(int u, int y) { return vpar[u] == y ? par[u] : jump(y, dep[u] + 1); }
int main() {
    ll m, k, p, q;
    cin >> n >> m >> k >> p >> q;
    map<ll, int> posof;
    for (int i = 0; i < m; i++) { ll x; cin >> x; posof[x] = i; }
    vector<int> pos(n);
    for (int i = 0; i < n; i++) { ll x; cin >> x; pos[i] = posof.count(x) ? posof[x] : -1; }
    if (n == 1) { cout << (pos[0] == 0 ? 1 : 0) << "\n"; return 0; }
    adj.assign(n, {});
    for (int i = 0; i < n - 1; i++) { int a, b; cin >> a >> b; a--; b--; adj[a].push_back(b); adj[b].push_back(a); }
    ll al = p % MOD * pw(q, MOD - 2) % MOD, inv_n = pw(n, MOD - 2);
    deg.resize(n); for (int i = 0; i < n; i++) deg[i] = adj[i].size();
    inv.assign(n + 1, 0); inv[1] = 1;
    for (int i = 2; i <= n; i++) inv[i] = (MOD - (MOD / i) * inv[MOD % i] % MOD) % MOD;
    par.assign(n, -1); dep.assign(n, 0); tin.assign(n, 0);
    vector<int> order; vector<char> seen(n, 0);
    vector<int> st = {0}; seen[0] = 1;
    while (!st.empty()) {
        int u = st.back(); st.pop_back(); tin[u] = order.size(); order.push_back(u);
        for (int w : adj[u]) if (!seen[w]) { seen[w] = 1; par[w] = u; dep[w] = dep[u] + 1; st.push_back(w); }
    }
    LOG = 1; while ((1 << LOG) <= n) LOG++;
    upt.assign(LOG, vector<int>(n));
    for (int i = 0; i < n; i++) upt[0][i] = par[i] < 0 ? 0 : par[i];
    for (int j = 1; j < LOG; j++) for (int i = 0; i < n; i++) upt[j][i] = upt[j - 1][upt[j - 1][i]];
    Q.assign(n, 1); Qi.assign(n, 1);
    for (size_t i = 1; i < order.size(); i++) { int u = order[i]; Q[u] = Q[par[u]] * inv[deg[u] - 1] % MOD; Qi[u] =
    Qi[par[u]] * (deg[u] - 1) % MOD; }
    map<int, vector<int>> groups;
    for (int x = 0; x < n; x++) if (pos[x] >= 0) groups[pos[x]].push_back(x);
    S.assign(n, 0); upm.assign(n, 0); downm.assign(n, 0); ew.assign(n, 1); tot.assign(n, 0);
    vpar.assign(n, -1); isSink.assign(n, 0);
    vector<char> isSrc(n, 0);
    ll f = al; int L = 0;
    vector<ll> e0(n, 0);
    // emision de u hacia el vecino virtual y
    auto emit = [&](int u, int y) -> ll {
        if (L == 0) return e0[u];
        if (!isSrc[u]) return 0;
        auto &mp = sub[u]; auto it = mp.find(reldir(u, y));
        return (tot[u] - (it == mp.end() ? 0 : it->second) + MOD) % MOD;
    };
    auto run = [&](const vector<int> &nodes) {
        for (int x : nodes) S[x] = 0;
        for (int i = (int)nodes.size() - 1; i >= 1; i--) {
            int x = nodes[i];
            ll val = emit(x, vpar[x]);
            if (!isSink[x]) val = (val + f * inv[deg[x] - 1] % MOD * S[x]) % MOD;
            upm[x] = val * ew[x] % MOD;
        }
    };
}

```

```

        S[vpar[x]] = (S[vpar[x]] + upm[x]) % MOD;
    }
    downm[nodes[0]] = 0;
    for (size_t i = 1; i < nodes.size(); i++) {
        int x = nodes[i], u = vpar[x];
        ll val = emit(u, x);
        if (!isSink[u]) val = (val + f * inv[deg[u] - 1] % MOD * ((S[u] - upm[x] + downm[u] + MOD) % MOD)) % MOD;
        downm[x] = val * ew[x] % MOD;
    }
    map<int, vector<pair<int, ll>>> res;
    for (size_t i = 0; i < nodes.size(); i++) {
        int x = nodes[i];
        if (isSink[x] && i > 0) res[x].push_back({vpar[x], downm[x]});
        if (i > 0 && isSink[vpar[x]]) res[vpar[x]].push_back({x, upm[x]});
        if (isSink[x] && !res.count(x)) res[x];
    }
    return res;
};

// nivel 0: arbol completo
for (int x : groups[0]) isSink[x] = 1;
for (int x = 0; x < n; x++) { vpar[x] = par[x]; if (!isSink[x]) e0[x] = inv_n * al % MOD * inv[deg[x]] % MOD; }
auto r0 = run(order);
for (auto &e : r0) {
    vector<pair<int, ll>> h = {{-1, inv_n}};
    for (auto &pr : e.second) h.push_back({pr.first, pr.second});
    H[e.first] = h;
}
for (int x : groups[0]) isSink[x] = 0;
ll ans = 0;
while (!H.empty()) {
    for (auto &e : H) for (auto &pr : e.second) ans = (ans + pr.second) % MOD;
    L++;
    if (!groups.count(L)) break;
    f = pw(al, L + 1);
    sub.clear();
    vector<int> pts;
    for (auto &e : H) {
        int b = e.first; isSrc[b] = 1; pts.push_back(b);
        ll t = 0; auto &mp = sub[b];
        for (auto &pr : e.second) {
            int c = pr.first < 0 ? deg[b] : deg[b] - 1;
            ll v = pr.second * f % MOD * inv[c] % MOD;
            mp[pr.first] = (mp[pr.first] + v) % MOD; t += v;
        }
        tot[b] = t % MOD;
    }
    for (int x : groups[L]) { isSink[x] = 1; pts.push_back(x); }
    sort(pts.begin(), pts.end(), [&](int a, int b) { return tin[a] < tin[b]; });
    pts.erase(unique(pts.begin(), pts.end()), pts.end());
    vector<int> nodes = pts;
    for (size_t i = 0; i + 1 < pts.size(); i++) nodes.push_back(lca(pts[i], pts[i + 1]));
    sort(nodes.begin(), nodes.end(), [&](int a, int b) { return tin[a] < tin[b]; });
    nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());
    vector<int> stk;
    for (int x : nodes) {
        while (!stk.empty() && lca(stk.back(), x) != stk.back()) stk.pop_back();
        vpar[x] = -1;
        if (!stk.empty()) {
            int u = stk.back(); vpar[x] = u;
            ew[x] = pw(f, dep[x] - dep[u] - 1) * Q[par[x]] % MOD * Qi[u] % MOD;
        }
        stk.push_back(x);
    }
    auto r = run(nodes);
    map<int, vector<pair<int, ll>>> NH;
    for (auto &e : r) {
        int b = e.first; map<int, ll> h;
        for (auto &pr : e.second) if (pr.second) { int d = reldir(b, pr.first); h[d] = (h[d] + pr.second) % MOD;
        }
        if (!h.empty()) NH[b] = vector<pair<int, ll>>(h.begin(), h.end());
    }
    for (auto &e : H) isSrc[e.first] = 0;
    for (int x : groups[L]) isSink[x] = 0;
    H.swap(NH);
}
cout << ans % MOD << "\n";
}

```

59. L. Copogonia

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** Grafos, fuerza bruta sobre subconjuntos **Tiempo:** $O(2^k * n^2)$
Memoria: $O(k * n^2)$ **Archivo:** Copogonia.py / .cpp **Estado:** local-ok

Solución: Con $k \leq 10$ hay solo 1024 subconjuntos. Se hace DFS decidiendo incluir o no cada vía; al incluir la vía (u, v) de longitud euclidiana w , la matriz de distancias se actualiza en $O(n^2)$ con $d[i][j] = \min(d[i][j], d[i][u] + w + d[v][j], d[i][v] + w + d[u][j])$, que es exacto al agregar una sola arista. La matriz inicial sale de las distancias sobre el ciclo (perímetro acumulado, el menor de los dos sentidos). Se poda cuando el costo ya no mejora la mejor respuesta, y se corta la rama en cuanto el máximo de la matriz es $\leq m$, porque agregar más vías solo cuesta más.

Python 3

```
# k <= 10: se prueban subconjuntos por DFS. La matriz de distancias se actualiza en O(n^2)
# al agregar cada via (u, v, w): d[i][j] = min(d[i][j], d[i][u]+w+d[v][j], d[i][v]+w+d[u][j]).
# Se poda en cuanto todos los pares quedan <= m o el costo supera el mejor.
import sys
from math import hypot

def main():
    t = sys.stdin.read().split()
    n, k, m = int(t[0]), int(t[1]), float(t[2])
    P = [(int(t[3 + 2 * i]), int(t[4 + 2 * i])) for i in range(n)]
    o = 3 + 2 * n
    R = [(int(t[o + 3 * i]) - 1, int(t[o + 3 * i + 1]) - 1, int(t[o + 3 * i + 2])) for i in range(k)]
    pre = [0.0] # perimetro acumulado
    for i in range(n):
        pre.append(pre[-1] + hypot(P[i][0] - P[(i + 1) % n][0], P[i][1] - P[(i + 1) % n][1]))
    per = pre[n]
    d0 = [[min(abs(pre[i] - pre[j]), per - abs(pre[i] - pre[j])) for j in range(n)] for i in range(n)]
    best = [float("inf")]
    def add(d, u, v):
        w = hypot(P[u][0] - P[v][0], P[u][1] - P[v][1])
        du, dv = d[u], d[v]
        res = []
        for row in d:
            a = row[u] + w; b = row[v] + w
            res.append([min(x, a + y, b + z) for x, y, z in zip(row, dv, du)])
        return res
    def dfs(i, d, cost):
        if cost >= best[0]: return
        if max(map(max, d)) <= m + 1e-9:
            best[0] = cost; return
        if i == k: return
        dfs(i + 1, add(d, R[i][0], R[i][1]), cost + R[i][2])
        dfs(i + 1, d, cost)
    dfs(0, d0, 0)
    print(best[0])
main()
```

C++11

```
// k <= 10: se prueban subconjuntos por DFS. La matriz de distancias se actualiza en O(n^2)
// al agregar cada via (u, v, w): d[i][j] = min(d[i][j], d[i][u]+w+d[v][j], d[i][v]+w+d[u][j]).
// Se poda en cuanto todos los pares quedan <= m o el costo supera el mejor.
#include <bits/stdc++.h>
using namespace std;
typedef vector<vector<double>> Mat;
int n, k; double m; long long best = LLONG_MAX;
vector<double> X, Y; vector<int> U, V; vector<long long> Cst;
void dfs(int i, const Mat &d, long long cost) {
    if (cost >= best) return;
    double mx = 0;
    for (auto &r : d) for (double x : r) mx = max(mx, x);
    if (mx <= m + 1e-9) { best = cost; return; }
    if (i == k) return;
    int u = U[i], v = V[i];
    double w = hypot(X[u] - X[v], Y[u] - Y[v]);
    Mat e = d;
    for (int a = 0; a < n; a++) for (int b = 0; b < n; b++)
        e[a][b] = min(d[a][b], min(d[a][u] + w + d[v][b], d[a][v] + w + d[u][b]));
    dfs(i + 1, e, cost + Cst[i]);
    dfs(i + 1, d, cost);
}
int main() {
    cin >> n >> k >> m;
    X.resize(n); Y.resize(n); U.resize(k); V.resize(k); Cst.resize(k);
    for (int i = 0; i < n; i++) cin >> X[i] >> Y[i];
    for (int i = 0; i < k; i++) { cin >> U[i] >> V[i] >> Cst[i]; U[i]--; V[i]--; }
    vector<double> pre(n + 1, 0);
    for (int i = 0; i < n; i++) pre[i + 1] = pre[i] + hypot(X[i] - X[(i + 1) % n], Y[i] - Y[(i + 1) % n]);
    Mat d(n, vector<double>(n));
    for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) {
```

```

        double a = fabs(pre[i] - pre[j]); d[i][j] = min(a, pre[n] - a);
    }
    dfs(0, d, 0);
    cout << best << "\n";
}

```

60. M. Weekend Gardening

Evento: RPC 11 de 2025 (Competitive Programming Network, 11th Activity) **Tema:** Probabilidad, DP **Tiempo:** $O(c*n*e)$ **Memoria:** $O(c*n*e)$
Archivo: Weekend.py / .cpp **Estado:** local-ok

Solución: El estado es (i, j, l) : cuántas baratas, normales y caras se han comprado; con eso se conoce el total y las plantas restantes. Desde un estado con total $< L$, cada tipo se elige con probabilidad (restantes de ese tipo)/(restantes totales). Si la compra lleva el total a $\geq L$ se para: suma a la respuesta si no pasa de H . Si se acaban las plantas sin llegar a L es fracaso. Hay a lo sumo 101^3 estados y se recorren por número de plantas compradas (Python) o en orden lexicográfico (C++), que respeta las transiciones. Se validó contra una recursión exhaustiva.

Python 3

```

# DP de probabilidad sobre (i, j, l) = plantas baratas, normales y caras ya compradas.
# Mientras el total sea < L se sigue comprando (eleccion uniforme entre las restantes);
# al llegar a >= L se para: exito si el total <= H.
import sys
def main():
    L, H, C, N, E, c, n, e = map(int, sys.stdin.read().split())
    ans = 0.0
    cur = {(0, 0, 0): 1.0} # estados por numero de plantas compradas
    for _ in range(c + n + e):
        nxt = {}
        for (i, j, l), pr in cur.items():
            rem = c + n + e - i - j - l
            for di, dj, dl, cnt in ((1, 0, 0, c - i), (0, 1, 0, n - j), (0, 0, 1, e - l)):
                if cnt == 0: continue
                q = pr * cnt / rem
                a, b, g = i + di, j + dj, l + dl
                s = a * C + b * N + g * E
                if s >= L:
                    if s <= H: ans += q
                else:
                    k = (a, b, g); nxt[k] = nxt.get(k, 0.0) + q
        cur = nxt
    print("%.9f" % ans)
main()

```

C++11

```

// DP de probabilidad sobre (i, j, l) = plantas baratas, normales y caras ya compradas.
// Mientras el total sea < L se sigue comprando (eleccion uniforme entre las restantes);
// al llegar a >= L se para: exito si el total <= H.
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
double dp[101][101][101];
int main() {
    ll L, H, C, N, E; int c, n, e;
    cin >> L >> H >> C >> N >> E >> c >> n >> e;
    double ans = 0;
    dp[0][0][0] = 1;
    // el orden lexicografico (i, j, l) respeta las transiciones (solo aumentan indices)
    for (int i = 0; i <= c; i++) for (int j = 0; j <= n; j++) for (int l = 0; l <= e; l++) {
        double pr = dp[i][j][l];
        if (pr == 0) continue;
        if (i * C + j * N + l * E >= L) continue; // estado absorbente (ya contado)
        int rem = c + n + e - i - j - l;
        if (rem == 0) continue;
        int cnt[3] = {c - i, n - j, e - l};
        for (int t = 0; t < 3; t++) {
            if (!cnt[t]) continue;
            int a = i + (t == 0), b = j + (t == 1), g = l + (t == 2);
            double q = pr * cnt[t] / rem;
            ll s = a * C + b * N + g * E;
            if (s >= L) { if (s <= H) ans += q; }
            else dp[a][b][g] += q;
        }
    }
    printf("%.9f\n", ans);
}

```

61. A. Alto Adaptation

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** DP **Tiempo:** $O(n^2)$ **Memoria:** $O(n)$ **Archivo:** Adaptation.py / Adaptation.cpp.py / .cpp **Estado:** local-ok

Solución: Cada nota a admite un rango de octavas k con $l \leq a+12k \leq h$: desde $\text{ceil}((l-a)/12)$ hasta $\text{floor}((h-a)/12)$. Un tramo de notas consecutivas puede cantarse con la misma transposición si la intersección de sus rangos no es vacía. Se hace DP: $\text{dp}[i]$ es el mayor valor posible del tramo más corto al partir las primeras i notas ($\text{dp}[0] = \text{infinito}$). Para cada i se recorre j hacia atrás manteniendo el máximo de los límites inferiores y el mínimo de los superiores; si la intersección se vacía se corta, y si no, $\text{dp}[i] = \max(\text{dp}[i], \min(\text{dp}[j], i-j))$. Con $n \leq 1000$ son a lo sumo medio millón de pasos. La respuesta es $\text{dp}[n]$.

Python 3

(no disponible)

C++11

(no disponible)

62. B. Bottle of New Port

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** Matemáticas / ad hoc **Tiempo:** $O(1)$ **Memoria:** $O(1)$ **Archivo:** Bottle.py / Bottle.cpp.py / .cpp **Estado:** local-ok

Solución: Tras d días el volumen de alcohol es $a - d \cdot da$ y el de otros líquidos $o - d \cdot do$, pero ninguno puede ser negativo: si un líquido se evapora por completo queda en cero. Se calculan ambos con enteros de 64 bits ($d \cdot da$ llega a $1e18$, cabe en long long) y se recorta con $\max(0, .)$. El enunciado garantiza que la botella no queda vacía, así que el denominador es positivo. El porcentaje es $100 \cdot A / (A + O)$, que se imprime con 10 decimales. El juez acepta error $1e-6$, por lo que la salida no tiene que coincidir carácter a carácter con la de los ejemplos (14.2857142857143 frente a 14.2857142857).

Python 3

(no disponible)

C++11

(no disponible)

63. C. Chess

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** Simulación **Tiempo:** $O(N^2 + M)$ **Memoria:** $O(N^2)$ **Archivo:** Chess.py / Chess.cpp.py / .cpp **Estado:** local-ok

Solución: En vez de pintar cada ataque (una reina ataca $O(N)$ casillas y puede haber N^2 piezas) se marcan líneas: las torres y reinas marcan su fila y su columna, y las reinas también sus dos diagonales, identificadas por $r-c$ y $r+c$. Los caballos marcan directamente su propia casilla y las hasta 8 casillas a salto de caballo. Luego se recorre el tablero $N \times N$ y una casilla cuenta como atacada si está marcada su fila, su columna o alguna de sus diagonales, o si la marcó un caballo. Cada pieza se ataca a sí misma, y eso ya queda cubierto. Total $O(N^2 + M)$.

Python 3

(no disponible)

C++11

(no disponible)

64. D. Dralinpome

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** Strings / conteo **Tiempo:** $O(|s|)$ **Memoria:** $O(1)$ **Archivo:** Dralinpome.py / Dralinpome.cpp.py / .cpp **Estado:** local-ok

Solución: Una palabra puede reordenarse en un palíndromo si y solo si a lo sumo una letra aparece un número impar de veces: en un palíndromo las letras se emparejan simétricamente alrededor del centro y solo el carácter central (si la longitud es impar) puede quedar sin pareja. Se cuentan las apariciones de cada una de las 26 letras, se cuenta cuántas tienen frecuencia impar y se responde yes si ese número es 0 o 1, y no en otro caso. Es lineal en la longitud de la palabra y usa memoria constante.

Python 3

(no disponible)

C++11

(no disponible)

65. E. Encrypting

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** Strings / ad hoc **Tiempo:** $O(N)$ **Memoria:** $O(N)$ **Archivo:** Encrypting.py / Encrypting.cpp.py / .cpp **Estado:** local-ok

Solución: Las letras se separan por exactamente una columna vacía (solo puntos en ambas filas) y dentro de una letra nunca hay columnas vacías: la 'v' ocupa 4 columnas y la 'w' 8. Basta recorrer las columnas de izquierda a derecha contando la longitud de cada racha de columnas no vacías. Al encontrar una columna vacía (o el final) se cierra la racha y se emite 'v' si mide 4 o 'w' si mide 8. Se usa un centinela en la posición N para cerrar la última letra. Las filas se leen como tokens porque no contienen espacios.

Python 3

(no disponible)

C++11

(no disponible)

66. F. Flakes

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** Simulación / grillas **Tiempo:** $O(nm(n+m))$ **Memoria:** $O(nm)$ **Archivo:** Flakes.py / Flakes.cpp.py / .cpp **Estado:** local-ok

Solución: El tamaño de un copo centrado en un '+' es el mínimo, sobre las 8 direcciones, de la racha de caracteres correctos que sale del centro: '|' arriba y abajo, '-' a izquierda y derecha, '\' en la diagonal superior izquierda/inferior derecha y '/' en la otra. Para cada '+' de la grilla se camina en cada dirección mientras el carácter coincida y se toma el mínimo de las 8 longitudes. La respuesta es el máximo sobre todos los centros (0 si no hay '+'). Con $n, m \leq 50$ la fuerza bruta es inmediata. Ojo: el PDF muestra el guion como un carácter largo, pero en la entrada es '-'.

Python 3

(no disponible)

C++11

(no disponible)

67. G. Genealogy Gumbo

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** Grafos / alcanzabilidad **Tiempo:** $O(n \log n)$ **Memoria:** $O(n)$ **Archivo:** Genealogy.py / Genealogy.cpp.py / .cpp **Estado:** local-ok

Solución: Como varias personas pueden compartir nombre, cada línea es un hijo distinto, y su padre puede ser cualquier persona con ese nombre o una persona sin padre registrado. Solo puede haber una persona sin padre: la raíz. Se arma el grafo de nombres padre $\rightarrow$ hijo. Si hay dos o más nombres que son padres pero nunca hijos, es imposible. Si hay uno, ese es la raíz; si no hay ninguno, la raíz es una persona nueva con algún nombre. Es posible si y solo si desde la raíz se alcanzan todos los nombres. Sin candidato, se toma el último vértice en terminar un DFS completo: si alguno alcanza a todos, es ese.

Python 3

(no disponible)

C++11

(no disponible)

68. H. Hidden Sequence

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** Greedy / punteros **Tiempo:** O(total) **Memoria:** O(total) **Archivo:** Hidden.py / Hidden.cpp.py / .cpp **Estado:** local-ok

Solución: La lista del jugador i contiene la secuencia real sin las victorias de i . Si el siguiente ganador real es w , su número debe estar al frente de las dos listas en las que aparece (las de los otros dos jugadores). Dos jugadores distintos no pueden cumplir esto a la vez, porque ambos aparecen en la lista del tercero y solo uno puede estar al frente. Se mantiene un puntero por lista y en cada paso se busca el único w que cumple la condición, se avanza en esas dos listas y se agrega w . La longitud total es la suma de las tres longitudes dividida entre 2.

Python 3

(no disponible)

C++11

(no disponible)

69. I. Item Selection

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** Greedy **Tiempo:** O(n) **Memoria:** O(n) **Archivo:** Item.py / Item.cpp.py / .cpp **Estado:** local-ok

Solución: Los clics de selección de cada página son independientes. En cada página con diferencias se elige lo más barato: alternar cada casilla distinta, pulsar Select All y desmarcar los no deseados ($1 + \text{no deseados de la página}$), o pulsar Deselect All y marcar los deseados ($1 + \text{deseados}$). Las páginas sin diferencias no cuestan nada y no hace falta visitarlas. Para navegar, sean L y R la primera y la última página con trabajo: hay que recorrer $[L, R]$ partiendo de s , y lo óptimo es ir primero al extremo más cercano, con costo $(R-L) + \min(|s-L|, |s-R|)$. Si ninguna página tiene trabajo, la respuesta es 0.

Python 3

(no disponible)

C++11

(no disponible)

70. J. Judge Meetings

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** Simulación **Tiempo:** O($n*m$) **Memoria:** O(m) **Archivo:** Judge.py / Judge.cpp.py / .cpp **Estado:** local-ok

Solución: Con a lo sumo 10 jueces, 10 vacaciones cada uno y 120 días, alcanza con simular. Se empieza con $\text{libres}[d] = n$ para cada día y , por cada vacación $[s, e]$ de cada juez, se resta uno a cada día del intervalo. Como las vacaciones de un mismo juez no se solapan, ningún día se descuenta dos veces para el mismo juez. Al final se cuentan los días entre 1 y m con al menos 3 jueces libres. Que las vacaciones vengan desordenadas no importa, porque cada intervalo se procesa por separado. El trabajo total está acotado por $n*10*m$, que es muy pequeño.

Python 3

(no disponible)

C++11

(no disponible)

71. K. Koehandel

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** Ad hoc **Tiempo:** O(1) **Memoria:** O(1) **Archivo:** Koehandel.py / Koehandel.cpp.py / .cpp **Estado:** local-ok

Solución: Si se apuesta x , se termina con $n - x + c$ monedas, y la vaca la gana quien apostó más. Lo primero es la vaca. Si $n > c$, se apuesta el mínimo que gana, $c+1$, y se termina con $n-1$ monedas. Si $n == c$ no se puede ganar, pero empatando con $x = c$ no se pierde la vaca y se conservan las n monedas. Si $n < c$ la vaca se pierde de todos modos, así que conviene apostar 0 y quedarse con $n + c$ monedas. Son tres casos en O(1). Con valores hasta $1e9$, $c+1$ cabe en 32 bits, pero se usa long long por seguridad.

Python 3

(no disponible)

C++11

(no disponible)

72. L. Luminosity

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** DP de intervalos / TSP en la recta **Tiempo:** $O(K^2)$, $K \leq 2N+1$
Memoria: $O(K)$ **Archivo:** Luminosity.py / Luminosity.cpp.py / .cpp **Estado:** local-ok

Solución: Cada cristal x de la unión debe tocarse por última vez en un tiempo $\geq R(x)$, el mayor t entre los rangos que lo contienen. Si se invierte el tiempo, las últimas visitas pasan a ser primeras visitas con plazo, y lo ya visitado es un intervalo que crece alrededor del punto final. Por eso solo importan las coordenadas l_i y r_i : los puntos interiores tienen R menor o igual y se alcanzan antes. R se calcula con un barrido y un heap. Hacia adelante, los pendientes forman un intervalo $[i..j]$ y el guardián está junto a uno de sus extremos; se atiende el extremo izquierdo o el derecho con tiempo $\max(\text{llegada}, R)$. La DP recorre el intervalo por longitud. RIESGO: la versión Python (pura, sin numpy) tarda ~ 28 s con $N=5000$; usar C++ en el juez (~ 0.7 s).

Python 3

(no disponible)

C++11

(no disponible)

73. M. Most Scenic Cycle

Evento: RPC 12 - 2025 (Competitive Programming Network, 12th Activity) **Tema:** Grafos serie-paralelo **Tiempo:** $O(E \log E)$ **Memoria:** $O(E)$
Archivo: Most.py / Most.cpp.py / .cpp **Estado:** local-ok

Solución: Las condiciones (2-conexo, con $E-V+1$ ciclos que forman un árbol) describen un grafo serie-paralelo, parecido a uno outerplanar. Se reduce el grafo: las aristas paralelas se fusionan y los vértices de grado 2 se eliminan uniendo en serie sus dos aristas. Cada arista compuesta $u-v$ guarda P , el mejor camino simple de u a v dentro de lo que representa, y C , el mejor ciclo dentro de ella. En serie: $P = P_1 + P_2$ y $C = \max(C_1, C_2)$. En paralelo: $P = \max(P_1, P_2)$ y $C = \max(C_1, C_2, P_1 + P_2)$, porque el ciclo usa un camino de cada lado. La respuesta es el mayor C . Se validó contra fuerza bruta.

Python 3

(no disponible)

C++11

(no disponible)

74. A. Accidental Arithmetic

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** Probabilidad / linealidad de la esperanza **Tiempo:** $O(L)$ **Memoria:** $O(L)$
Archivo: Accidental.py / Accidental.cpp.py / .cpp **Estado:** local-ok

Solución: Por linealidad de la esperanza, cada término que no es el primero lleva signo $+$ o $-$ con igual probabilidad, así que su aporte esperado es cero. Solo importa el primer término. Si termina en el dígito j (probabilidad $0.1^j \cdot 0.9$, o $0.1^{(L-1)}$ si llega al final), vale el prefijo de $j+1$ dígitos. Al multiplicar ese valor por 0.1^j queda $S_j = \sum_{i \leq j} d_i \cdot 10^{-(i)}$. La respuesta es $0.9 \cdot \sum_{j < L-1} S_j + S_{(L-1)}$. Se recorre el número una sola vez acumulando S_j en punto flotante; las potencias negativas que se vuelven cero no afectan la precisión pedida.

Python 3

(no disponible)

C++11

(no disponible)

75. B. Boggle Sort

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** DP **Tiempo:** O(16·6·26) **Memoria:** O(26) **Archivo:** Boggle.py / Boggle.cpp.py / .cpp **Estado:** local-ok

Solución: Se recorren los dados en orden con una DP: el estado es la última letra colocada y el valor, el mínimo de giros. Para cada dado se prueban sus 6 caras: arriba cuesta 0, cada lateral 1 y la de abajo 2. Una cara c solo se puede poner si la letra anterior es $\leq c$. La cara Q se trata como el par "QU": exige anterior $\leq Q$ y deja U como última letra, así que el siguiente dado debe ser $\geq U$. Se arranca con 'A' como última letra. Si al final no queda ningún estado alcanzable, se imprime impossible. Son 16·6·26 transiciones.

Python 3

(no disponible)

C++11

(no disponible)

76. C. Coherency

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** Geometría + rejilla hash + DSU **Tiempo:** O(n·k), k = vecinos por celda **Memoria:** O(n) **Archivo:** Coherency.py / Coherency.cpp.py / .cpp **Estado:** local-ok

Solución: Dos modelos son vecinos si la distancia entre centros es $\leq (d1+d2)/2 + 50.8$. Multiplicando por 20 y elevando al cuadrado queda la comparación entera $400 \cdot \text{dist}^2 \leq (10 \cdot (d1+d2) + 1016)^2$, sin errores de redondeo. Con diámetro máximo 160, dos vecinos están a menos de 211 mm, así que los centros se agrupan en celdas de 211 mm y solo se comparan celdas adyacentes (la mitad del vecindario, para no repetir pares). Como las bases no se solapan, cada celda tiene pocos modelos. Un DSU cuenta las componentes y, de paso, los grados. Es coherente si hay una sola componente y, con $n \geq 7$, el grado mínimo es ≥ 2 .

Python 3

(no disponible)

C++11

(no disponible)

77. D. Dart Game

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** Implementación **Tiempo:** O(1) **Memoria:** O(1) **Archivo:** Dart.py / Dart.cpp.py / .cpp **Estado:** local-ok

Solución: El anillo de una celda es su distancia de Chebyshev al centro $o = n/2 + 1$, es decir, $\max(|r-o|, |c-o|)$. El centro tiene distancia 0 y vale 100; cada anillo resta 10 puntos. Como $n \leq 21$, el anillo más lejano es el 10 y vale 0. La fórmula $\max(0, 100 - 10 \cdot \text{dist})$ cubre todos los casos, incluso el de no bajar de cero. Es una fórmula directa en O(1). La solución previa del usuario hacía lo mismo con más ramas; aquí queda en una línea.

Python 3

(no disponible)

C++11

(no disponible)

78. E. Excruciating Elevators

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** Residuos modulares + enumeración de fases ancladas **Tiempo:** O(n³ · n · 4) $\approx 1e8$ (C++ 0.6 s) **Memoria:** O(n) **Archivo:** Elevators.py / Elevators.cpp.py / .cpp **Estado:** local-ok (py ~8 s con n=35: usar C++)

Solución: El tiempo total es la suma fija de viajes y trabajos más las esperas. Trabajando módulo $P = 2 \cdot 10^6$ (periodo de un ascensor), cada tramo i tiene un instante ideal c_i y cada ascensor una fase k_j ; la espera es $(c_i + k_j - W) \bmod P$, donde W es la espera acumulada. Fijadas las fases, lo óptimo es tomar siempre el ascensor que pase primero. En una solución óptima cada ascensor tiene algún tramo sin espera, lo que obliga a que $k_j = k_{j'} \pm (c_{\{i-1\}} - c_i)$ respecto a otro ascensor j' . Se enumeran esas fases como un árbol de profundidad ≤ 3 (unos 800 mil conjuntos) y se simula el greedy con poda. Ojo: anclar cada ascensor en su primer uso no es óptimo (contraejemplo en tests/Elevators.8).

Python 3

(no disponible)

C++11

(no disponible)

79. F. Fences Make Good Neighbors

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** DP de intervalos (triangulación de polígono convexo) **Tiempo:** $O(n^3)$ **Memoria:** $O(n^2)$ **Archivo:** Fences.py / Fences.cpp.py / .cpp **Estado:** local-ok (Python lento: usar C++ en el juez)

Solución: Para cada cadena $a \rightarrow b$ del polígono, $C[a][b]$ es el costo mínimo de triangular la región cortada por la diagonal $a-b$. Sale de la recurrencia clásica sobre el vértice v del triángulo (a, v, b) . $A1[a][b]$ y $A2[a][b]$ usan la misma recurrencia, pero exigen que el triángulo (a, v, b) contenga estrictamente al hermano 1 o al 2. Las diagonales que pasan por un hermano valen infinito y los lados del polígono valen 0. Después se prueba cada triángulo central (i, j, k) : sus lados separan tres regiones; se asigna el hermano 1 a una, el 2 a otra y la tercera queda libre (6 asignaciones), sumando las diagonales del triángulo. Si nada es válido, -1 . Python lento: usar C++ en el juez.

Python 3

(no disponible)

C++11

(no disponible)

80. G. Geometry Rush

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** Barrido por columnas / intervalos **Tiempo:** $O(w + n + m)$ **Memoria:** $O(w)$ **Archivo:** Geometry.py / Geometry.cpp.py / .cpp **Estado:** local-ok

Solución: Entre dos curvas monótonas en x la zona libre es un corredor, así que en cada columna las alturas alcanzables forman un intervalo $[lo, hi]$ con la paridad de x . Los vértices tienen coordenadas enteras, de modo que en la franja $[x, x+1]$ cada curva es un segmento recto, más tramos verticales en x y en $x+1$. Una diagonal es válida si va estrictamente por debajo del techo y por encima del piso en ambos extremos. Por eso se precálculan $ceil(\text{mín techo})$ y $floor(\text{máx piso})$ en cada X entero, interpolando con aritmética entera. Las subidas válidas forman un intervalo de y , y las bajadas también. Se intersectan con $[lo, hi]$, se ajusta la paridad y su unión es el nuevo intervalo. Si queda vacío, imposible.

Python 3

(no disponible)

C++11

(no disponible)

81. H. Honest (or dishonest) Lottery

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** Conteo **Tiempo:** $O(n)$ **Memoria:** $O(1)$ **Archivo:** Honest.py / Honest.cpp.py / .cpp **Estado:** local-ok

Solución: Basta un arreglo de 50 contadores. Se leen todos los números de una vez (5 por sorteo, $10n$ sorteos) y se incrementa el contador de cada uno. Al final se recorren los valores del 1 al 50 y se imprimen, separados por espacio, los que tienen contador $> 2n$. Si no hay ninguno, se imprime -1 . Como el recorrido va en orden, la lista ya sale de menor a mayor y no hace falta ordenar. El tiempo es lineal en la entrada y la memoria, constante. Es una versión más corta de la solución previa del usuario.

Python 3

(no disponible)

C++11

(no disponible)

82. I. Intermill Logistics

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** Greedy + ordenamiento **Tiempo:** $O(n \log n)$ **Memoria:** $O(n)$
Archivo: Intermill.py / Intermill.cpp.py / .cpp **Estado:** local-ok

Solución: Para un tiempo final T , el molino i procesa $p_i \cdot \max(0, T - 2t_i)$ kg, y se busca el T mínimo con suma $\geq w$. Esa suma es lineal por tramos, así que basta ordenar los molinos por t . Con los k más cercanos, $T_k = (w + \sum_{i=1}^k 2 \cdot t_i \cdot p_i) / \sum_{i=1}^k p_i$, y es la respuesta en cuanto $T_k \leq 2 \cdot t$ del siguiente molino, porque ese ya no ayudaría. Todo se hace con enteros exactos (enteros grandes en Python, `__int128` en C++, porque $2 \cdot t \cdot \sum p$ llega a unos $2 \cdot 10^{23}$) y solo se divide al imprimir. El costo lo domina el ordenamiento, $O(n \log n)$.

Python 3

(no disponible)

C++11

(no disponible)

83. J. Jump ... Triple Jump

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** Fuerza bruta **Tiempo:** $O(1000)$ **Memoria:** $O(1)$ **Archivo:** Jump.py / Jump.cpp.py / .cpp **Estado:** local-ok

Solución: La menor suma posible es $3a$ y la mayor es $3c$, así que $a = d_{\min}/3$ y $c = d_{\max}/3$ salen directo. Para b se prueba cada valor entre $a+1$ y $c-1$ (como mucho unos 330). Con cada candidato se generan las 10 sumas de multiconjuntos de tamaño 3 de $\{a, b, c\}$ y se compara ese conjunto con la lista dada; el primero que coincide es la respuesta. La solución previa del usuario buscaba b con una sola condición, lo que puede dar falsos positivos. Comparar el conjunto completo evita ese riesgo y sigue siendo trivial en tiempo.

Python 3

(no disponible)

C++11

(no disponible)

84. K. Knowing the Clock

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** Matemática **Tiempo:** $O(1)$ **Memoria:** $O(1)$ **Archivo:** Knowing.py / Knowing.cpp.py / .cpp **Estado:** local-ok

Solución: Si han pasado t minutos desde las 12, la horaria está en $t/2$ grados y el minutero en $6t \bmod 360$. Como h fija el tiempo exacto, $t = 2h$ minutos, y entonces el minutero tiene que estar en $12h \bmod 360$. La respuesta es yes si y solo si $(12 \cdot h) \bmod 360 == m$. Con los ejemplos: $h=32$ da $384 \bmod 360 = 24$, que coincide; $h=60$ da 0, que no es 90. Todo es aritmética entera, sin flotantes, a diferencia de la solución previa, que comparaba contra $m/12$ en punto flotante.

Python 3

(no disponible)

C++11

(no disponible)

85. L. Linguistic Labyrinth

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** Conteo en retícula 3D **Tiempo:** $O(n^3 \cdot (2n)^3)$ **Memoria:** $O(n^3)$
Archivo: Labyrinth.py / Labyrinth.cpp.py / .cpp **Estado:** local-ok (Python lento: usar C++ en el juez)

Solución: Se fija el vector $v = P - A$. Las condiciones quedan $B \cdot v = A \cdot v$ (B está en el plano perpendicular a v que pasa por A) y $C \cdot v = P \cdot v = A \cdot v + |v|^2$. Para cada v (unos $(2n-1)^3$ vectores) se buscan los pares $(A, P = A+v)$, y si no hay ninguno se pasa al siguiente. Si hay, se arman histogramas de $B \cdot v$ y de $C \cdot v$ y por cada par se suma $\text{cntB}[A \cdot v] \cdot \text{cntC}[A \cdot v + |v|^2]$. Los productos escalares están en $[-1400, 1400]$ y caben en un arreglo con desplazamiento. En C++ solo se limpian las casillas usadas y en Python se usan diccionarios como histogramas (Python lento: usar C++ en el juez). El resultado necesita 64 bits.

Python 3

(no disponible)

C++11

(no disponible)

86. M. Marching Orders

Evento: RPC 13 de 2025: Competitive Programming Network, 13th Activity **Tema:** CRT generalizado **Tiempo:** $O(n^2)$ **Memoria:** $O(n)$ **Archivo:** Marching.py / Marching.cpp.py / .cpp **Estado:** local-ok

Solución: Simulando sobre la lista restante, cada paso i fija la congruencia $m \equiv r_i \pmod{n-i}$, donde r_i es la posición de la letra elegida. Queda un sistema con módulos 20, 19, ..., 1, que no son coprimos. Se combina de forma incremental: con la solución actual $a \pmod{M}$ se busca t en $[0, k)$ tal que $a + M \cdot t \equiv r \pmod{k}$. Si no existe, la respuesta es NO; si existe, $a += M \cdot t$ y $M = \text{mcm}(M, k)$. El a final es el menor no negativo y, como $\text{mcm}(1..20) = 232792560 < 10^9$, siempre cumple la cota. Una fuerza bruta sobre m confirmó los resultados.

Python 3

(no disponible)

C++11

(no disponible)

87. -. Borrar

Evento: Maratón Interna de Programación UTP 2025 **Tema:** Cadenas **Tiempo:** $O(|s|)$ **Memoria:** $O(|s|)$ **Archivo:** Borrar.py / .cpp **Estado:** local-ok

Solución: Se recorre el texto una vez. Los caracteres prohibidos (lo que va entre los corchetes de la segunda línea) se guardan en un set para consultarlos en $O(1)$. Cada carácter permitido se agrega solo si es distinto del último carácter del resultado; esto reproduce la variable temp del código original y hace que, al borrar una vocal entre dos espacios, no queden espacios dobles. Al final se quitan los espacios de los extremos (strip). Todo es lineal en la longitud del texto. En C++ también se quitan los $\backslash r$ finales para tolerar entradas con fin de línea de Windows.

Python 3

```
# Quitar los caracteres prohibidos (dados entre corchetes) y no repetir un caracter igual
# al ultimo que quedo en el resultado; al final recortar espacios de los extremos.
import sys
lines = sys.stdin.read().split("\n")
text, banned = lines[0].rstrip("\r"), set(lines[1].strip()[1:-1])
res = []
for ch in text:
    if ch not in banned and (not res or res[-1] != ch):
        res.append(ch)
print("".join(res).strip())
```

C++11

```
// Quitar los caracteres prohibidos (dados entre corchetes) y no repetir un caracter igual
// al ultimo que quedo en el resultado; al final recortar espacios de los extremos.
#include <bits/stdc++.h>
using namespace std;
int main() {
    string text, oc, res;
    getline(cin, text); getline(cin, oc);
    if (!text.empty() && text.back() == '\r') text.pop_back();
    while (!oc.empty() && isspace((unsigned char)oc.back())) oc.pop_back();
    set<char> banned(oc.begin() + 1, oc.end() - 1);
    for (char ch : text)
        if (!banned.count(ch) && (res.empty() || res.back() != ch)) res += ch;
    size_t a = res.find_first_not_of(" \t"), b = res.find_last_not_of(" \t");
    cout << (a == string::npos ? "" : res.substr(a, b - a + 1)) << "\n";
}
```

88. -. Guanex (cola de prioridad)

Evento: Maratón Interna de Programación UTP 2025 **Tema:** Heap **Tiempo:** $O(n \log n)$ **Memoria:** $O(n)$ **Archivo:** Guanex.py / .cpp **Estado:** local-ok

Solución: La versión del usuario buscaba el mínimo con `min()` en una lista, lo que cuesta $O(n)$ por operación y $O(n^2)$ en total, demasiado para N grande. Un min-heap resuelve todo: `heappush` para insertar, `heappop` para borrar el mínimo y `h[0]` para consultarlo, cada una en $O(\log n)$ o menos. En C++ se usa `priority_queue` con `greater<>` para que el tope sea el mínimo. Borrar con el heap vacío se ignora, igual que en el código original. La salida se acumula y se imprime de una vez para que la E/S sea rápida.

Python 3

```
# Cola de prioridad (min-heap): 1 x inserta, 2 elimina el minimo, 3 consulta el minimo.
import sys, heapq
d = sys.stdin.buffer.read().split()
h, out, p = [], [], 1
for _ in range(int(d[0])):
    op = d[p]; p += 1
    if op == b"1":
        heapq.heappush(h, int(d[p])); p += 1
    elif op == b"2":
        if h:
            heapq.heappop(h)
    else:
        out.append(str(h[0]) if h else "Empty!")
sys.stdout.write("\n".join(out) + ("\n" if out else ""))
```

C++11

```
// Cola de prioridad (min-heap): 1 x inserta, 2 elimina el minimo, 3 consulta el minimo.
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    priority_queue<long long, vector<long long>, greater<long long> > h;
    while (n--) {
        int op; cin >> op;
        if (op == 1) { long long x; cin >> x; h.push(x); }
        else if (op == 2) { if (!h.empty()) h.pop(); }
        else if (h.empty()) cout << "Empty!\n";
        else cout << h.top() << "\n";
    }
}
```

89. -. K cuadrado

Evento: Maratón Interna de Programación UTP 2025 **Tema:** Aritmética modular **Tiempo:** $O(q)$ **Memoria:** $O(1)$ **Archivo:** Kcuadrado.py / .cpp **Estado:** local-ok

Solución: La fórmula es directa: la respuesta es $k^2 \bmod M$ con $M = 10^{18} + 3$. El único detalle es el desborde: con k cerca de 10^{18} , k^2 tiene unos 120 bits y no cabe en 64. Python maneja enteros arbitrarios, así que basta $k^2 \% M$. En C++ primero se reduce k módulo M y luego se multiplica en `unsigned __int128`, una extensión de GCC que admiten los jueces habituales, antes de tomar el módulo. Cada consulta cuesta $O(1)$. Se lee todo de una vez y se escribe una línea por consulta con salida rápida.

Python 3

```
# Respuesta directa:  $k^2 \bmod (10^{18} + 3)$ . Python maneja enteros grandes.
import sys
d = sys.stdin.buffer.read().split()
M = 10**18 + 3
sys.stdout.write("".join(str(int(k) ** 2 % M) + "\n" for k in d[1:1 + int(d[0])]))
```

C++11

```
// Respuesta directa:  $k^2 \bmod (10^{18} + 3)$ , usando __int128 para no desbordar.
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    const unsigned long long M = 1000000000000000003ULL;
    int q; cin >> q;
    while (q--) {
        unsigned long long k; cin >> k;
        unsigned __int128 r = (unsigned __int128)(k % M) * (k % M) % M;
        cout << (unsigned long long)r << "\n";
    }
}
```

90. -. Bubble sort

Evento: Práctica Codeforces / LeetCode 2025 **Tema:** Ordenamiento **Tiempo:** $O(n^2)$ **Memoria:** $O(n)$ **Archivo:** BubbleSort.py / .cpp **Estado:** local-ok

Solución: En cada pasada i se comparan pares de elementos vecinos y se intercambian si están desordenados. Así, el mayor de la parte que falta "sube" hasta la posición $n - i - 1$. Por eso el ciclo interno solo llega hasta $n - i - 1$: lo que está más a la derecha ya está en su lugar. Después de $n - 1$ pasadas el arreglo queda ordenado. Son $O(n^2)$ comparaciones, aceptable para n pequeño, aunque en competencia normalmente se usaría sort. Es el mismo código que el usuario tenía en bubbleSort.cpp y problemas.py, ahora leyendo de la entrada estándar.

Python 3

```
# Burbuja: en cada pasada el mayor restante "sube" al final.  $O(n^2)$ .
import sys
d = list(map(int, sys.stdin.read().split()))
n, a = d[0], d[1:1 + d[0]]
for i in range(n - 1):
    for j in range(n - i - 1):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
print(" ".join(map(str, a)))
```

C++11

```
// Burbuja: en cada pasada el mayor restante "sube" al final.  $O(n^2)$ .
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    for (int i = 0; i + 1 < n; i++)
        for (int j = 0; j + 1 < n - i; j++)
            if (a[j] > a[j + 1]) swap(a[j], a[j + 1]);
    for (int i = 0; i < n; i++) cout << (i ? " " : "") << a[i];
    cout << "\n";
}
```

91. -. Maximum subarray

Evento: Práctica Codeforces / LeetCode 2025 **Tema:** Kadane / DP **Tiempo:** $O(n)$ **Memoria:** $O(1)$ **Archivo:** MaximumSubarray.py / .cpp **Estado:** local-ok

Solución: El primer intento del usuario probaba todos los inicios y acumulaba sumas, $O(n^2)$. El algoritmo de Kadane, que el usuario dejó comentado, lo hace en $O(n)$: cur es la mejor suma de un subarreglo que termina en la posición actual. Si cur era negativo conviene empezar de nuevo, así que $cur = \max(cur, 0) + x$, y $best$ guarda el mayor cur visto. Como $best$ empieza en el primer elemento (o en $-\infty$ en C++), funciona aunque todos los números sean negativos: la respuesta es entonces el mayor de ellos. Usa memoria constante.

Python 3

```
# Kadane: la mejor suma que termina en  $i$  es  $\max(x, \text{mejor\_anterior} + x)$ .  $O(n)$ .
import sys
d = list(map(int, sys.stdin.read().split()))
best, cur = d[1], 0
for x in d[1:1 + d[0]]:
    cur = max(cur, 0) + x
    best = max(best, cur)
print(best)
```

C++11

```
// Kadane: la mejor suma que termina en  $i$  es  $\max(x, \text{mejor\_anterior} + x)$ .  $O(n)$ .
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    long long best = LLONG_MIN, cur = 0, x;
    for (int i = 0; i < n; i++) {
        cin >> x;
        cur = max(cur, 0LL) + x;
        best = max(best, cur);
    }
    cout << best << "\n";
}
```

92. A. A Little Leftover Pizza

Evento: RPC 03 - 2026 (Competitive Programming Network, 3rd Activity) **Tema:** Implementación / aritmética **Tiempo:** O(n) **Memoria:** O(1)
Archivo: Pizza.py / Pizza.cpp.py / .cpp **Estado:** local-ok

Solución: Como solo se pueden juntar porciones de pizzas del mismo tamaño, cada tamaño se resuelve por separado. Se suman las porciones sobrantes de las pizzas pequeñas, medianas y grandes, y cada tamaño necesita $\text{ceil}(\text{total}/\text{capacidad})$ cajas, con capacidad 6, 8 y 12. Ninguna caja puede llevar más porciones de las que traía originalmente, y nada impide llenarla hasta esa capacidad, así que redondear la división hacia arriba da exactamente el mínimo. Un tamaño sin porciones aporta cero cajas. La lectura es lineal y la memoria constante: solo se guardan tres acumuladores.

Python 3

(no disponible)

C++11

(no disponible)

93. B. Andor Strikes Again

Evento: RPC 03 - 2026 (Competitive Programming Network, 3rd Activity) **Tema:** DP en árboles **Tiempo:** O(N) **Memoria:** O(N) **Archivo:** Andor.py / Andor.cpp.py / .cpp **Estado:** local-ok

Solución: Para cada nodo se calculan dos costos: volverlo verdadero (cT) y volverlo falso (cF). En una hoja el costo es 0 para su valor actual y 1 para el otro. En un nodo AND, para que sea verdadero todos los hijos deben serlo (cT = suma de los cT de los hijos) y para que sea falso basta un hijo (cF = mínimo de los cF); en un OR es al revés. El árbol llega por niveles: se guardan las filas y se recorren desde la última hasta la raíz, repartiendo en orden los hijos del nivel de abajo entre los nodos internos. La respuesta es el costo del valor contrario al actual de la raíz, $\max(cT, cF)$, porque el valor actual cuesta 0.

Python 3

(no disponible)

C++11

(no disponible)

94. C. AROD

Evento: RPC 03 - 2026 (Competitive Programming Network, 3rd Activity) **Tema:** Conteo / geometría en retícula **Tiempo:** $O(mx \cdot my + V \log V + mx^2 + my^2)$ **Memoria:** $O(mx \cdot my)$ **Archivo:** AROD.py / AROD.cpp.py / .cpp **Estado:** local-ok

Solución: Degenerados: entre dos extremos separados por (dx,dy) hay $\gcd(dx,dy)-1$ puntos intermedios; se multiplica por el número de traslaciones. Los demás se agrupan por caja envolvente $w \times h$, que aparece $(mx-w+1)(my-h+1)$ veces, y siempre tienen un vértice en una esquina. (i) Con dos esquinas opuestas, el tercer punto queda dentro del círculo cuyo diámetro es la diagonal: salen 4 rectos y el resto obtusos, descontando la diagonal. (ii-b) Con dos esquinas adyacentes y un punto en el lado opuesto, es obtuso si $x(w-x) > h^2$, lo que se cuenta con raíz entera. (ii-a) Con una esquina y un punto en cada lado lejano, el ángulo en B es obtuso si $w \cdot x' < y(h-y)$; esa suma ponderada sale de una criba sobre los productos $w \cdot x'$. Agudos = total menos el resto. Comprobado contra fuerza bruta.

Python 3

(no disponible)

C++11

(no disponible)

95. D. But I Want to Win

Evento: RPC 03 - 2026 (Competitive Programming Network, 3rd Activity) **Tema:** Greedy / simulación **Tiempo:** $O(c \log c)$ **Memoria:** $O(c)$ **Archivo:** Butiwant.py / Butiwant.cpp.py / .cpp **Estado:** local-ok

Solución: Si el primero ya tiene más de la mitad, gana en la primera ronda y es imposible. En el mejor caso para el segundo, todos los votos de cada eliminado pasan a él. Los demás candidatos no cambian, así que en cada ronda se elimina el menor de los que quedan (los valores son distintos, no hay empates). Se ordenan los votos y se suman los menores al segundo, contando rondas hasta que supere estrictamente la mitad del total. Si solo quedan dos candidatos sin superarla (o empatados), o si había solo dos desde el principio, se imprime IMPOSSIBLE TO WIN.

Python 3

(no disponible)

C++11

(no disponible)

96. E. Chess Solitaire

Evento: RPC 03 - 2026 (Competitive Programming Network, 3rd Activity) **Tema:** Backtracking con memoización **Tiempo:** $O(\text{estados} \cdot m^2)$ **Memoria:** $O(\text{estados})$ **Archivo:** Chess.py / Chess.cpp.py / .cpp **Estado:** local-ok

Solución: Se hace una búsqueda en profundidad. En cada estado se generan todas las capturas legales: el caballo y el rey saltan, y el alfil, la torre y la dama se deslizan hasta la primera pieza que encuentran. Luego se ordenan por (casilla origen, casilla destino) como texto. Probándolas en ese orden, la primera secuencia completa que aparece es justo la del desempate lexicográfico movimiento a movimiento. Cada captura quita una pieza, así que la profundidad es $m-1 \leq 9$. Los estados (piezas con su posición) que ya se sabe que no tienen solución se guardan en un conjunto para no repetirlos. Si la raíz falla, se imprime No solution.

Python 3

(no disponible)

C++11

(no disponible)

97. F. Fractional Sequence

Evento: RPC 03 - 2026 (Competitive Programming Network, 3rd Activity) **Tema:** Matemáticas **Tiempo:** $O(1)$ **Memoria:** $O(1)$ **Archivo:** Sequence.py / Sequence.cpp.py / .cpp **Estado:** local-ok

Solución: El bloque i tiene exactamente i términos, así que antes de él hay $i(i-1)/2$. Se busca el i con $i(i-1)/2 < n \leq i(i+1)/2$ a partir de una raíz cuadrada aproximada, corregida con un par de pasos para evitar errores de redondeo. La posición dentro del bloque es $k = n - i(i-1)/2 - 1$ y el valor es $i + k/i$. Si $k = 0$ se imprime solo i ; si no, se reduce k/i dividiendo entre su gcd y se imprime ' a/b '. Todo cabe en enteros de 64 bits, porque i es del orden de $9 \cdot 10^4$.

Python 3

(no disponible)

C++11

(no disponible)

98. G. How Many Balls?

Evento: RPC 03 - 2026 (Competitive Programming Network, 3rd Activity) **Tema:** Matemáticas / ecuación cuadrática **Tiempo:** $O(R)$, $R = 10^4$ **Memoria:** $O(1)$ **Archivo:** Balls.py / Balls.cpp.py / .cpp **Estado:** local-ok

Solución: De $2rgq = p(r+g)(r+g-1)$, despejando g , sale la cuadrática $p \cdot g^2 + (p(2r-1) - 2rq) \cdot g + p(r^2 - r) = 0$. Se recorre r de 1 a 10^4 en orden, así que el primero que sirve es el de menor r . Para cada r se calcula el discriminante, que debe ser no negativo y cuadrado perfecto (raíz entera exacta: `isqrt` en Python, `sqrtl` corregido en C++). La raíz mayor, $(-B + \text{raíz})/(2p)$, tiene que ser entera y al menos r . El enunciado garantiza a lo sumo un g válido por cada r . Los valores intermedios llegan a unos $4 \cdot 10^4$, que cabe en long long.

Python 3

(no disponible)

C++11

(no disponible)

99. H. Move It, Slowpoke!

Evento: RPC 03 - 2026 (Competitive Programming Network, 3rd Activity) **Tema:** Dijkstra sobre estados **Tiempo:** $O(m \cdot d \cdot \text{grado} \cdot \log)$ **Memoria:** $O(m \cdot d)$ **Archivo:** Slowpoke.py / Slowpoke.cpp.py / .cpp **Estado:** local-ok

Solución: El estado es la calle dirigida por la que se acaba de llegar ($a \rightarrow b$) junto con el largo c del tramo continuo actual, acotado a $d+1$. Desde ($a \rightarrow b$, c) se puede seguir por cualquier calle $b \rightarrow x$ con $x \neq a$, porque no hay vueltas en U . Si (a, b, x) es un par continuo, el tramo pasa a $c+1$ y solo vale si $c+1 \leq d$; si no lo es, el tramo se reinicia en l (una calle sola sí puede medir más que d). Dijkstra arranca desde todas las calles que salen de s . Las transiciones no continuas no dependen de c , así que solo se relajan la primera vez que la calle sale de la cola. La respuesta es la primera llegada a t .

Python 3

(no disponible)

C++11

(no disponible)

100. I. Number Pyramid

Evento: RPC 03 - 2026 (Competitive Programming Network, 3rd Activity) **Tema:** Propagación de restricciones **Tiempo:** $O(n^2)$ **Memoria:** $O(n^2)$ **Archivo:** Pyramid.py / Pyramid.cpp.py / .cpp **Estado:** local-ok

Solución: Según el enunciado, las tarjetas válidas se resuelven deduciendo casillas a partir de sus vecinas, así que se simula esa propagación con una cola. En cada triángulo (padre, hijo izquierdo, hijo derecho) se cumple $\text{padre} = \text{izq} + \text{der}$. Si se conocen dos valores, se calcula el tercero y se encolan los triángulos que tocan la casilla nueva; si se conocen los tres, se comprueba la igualdad. Una contradicción, o un valor deducido fuera de $[-99, 99]$, da no solution. Si al final queda alguna casilla vacía, hay grados de libertad y la respuesta es ambiguous; si no, solvable y se imprime la pirámide. Cada casilla se llena una sola vez: $O(n^2)$.

Python 3

(no disponible)

C++11

(no disponible)

101. J. Polyomino Tiling

Evento: RPC 03 - 2026 (Competitive Programming Network, 3rd Activity) **Tema:** Cadenas / palíndromos con hueco + bitsets **Tiempo:** $O(k^3/64)$ peor caso teórico; $\sim k^2/8$ arcos $\times k/64$ en la práctica **Memoria:** $O(k^2/64)$ **Archivo:** Polyomino.py / Polyomino.cpp.py / .cpp **Estado:** local-ok

Solución: En ambas formas, cada factor U que empieza en q tiene su reverso complementado en $q+h$, con $h = k/2$. Un arco (q, L) es bueno si $B[q..q+L)$ y $B[q+h..q+h+L)$ son reverso-complemento. Esa condición empareja la posición i con $c+h-i$, donde c es la suma de los extremos; con c fijo, los arcos buenos están anidados, así que se obtienen expandiendo desde cada uno de los k centros. Una factorización es una cadena de 2 o 3 arcos buenos seguidos que suman exactamente h . Para 3 arcos se fija el primero (q_1, a) y, con un AND de bitsets, se cuentan los b para los que (q_1+a, b) y el arco que cierra en q_1+h son buenos. Cada factorización aparece en las rotaciones s y $s+h$, de ahí el $\times 2$.

Python 3

(no disponible)

C++11

(no disponible)

102. K. Triptych

Evento: RPC 03 - 2026 (Competitive Programming Network, 3rd Activity) **Tema:** DP de conteo **Tiempo:** $O(w^3)$ **Memoria:** $O(w^2)$ **Archivo:** Triptych.py / Triptych.cpp.py / .cpp **Estado:** local-ok

Solución: Primero se cuentan todas las secuencias que cumplen las reglas de variedad y equilibrio, y después se restan los palíndromos que también las cumplen. El DP avanza posición a posición con estado (cantidad de A, cantidad de B, último estado entre A, B, BB y C); la cantidad de C sale del largo. Las transiciones prohíben AA, CC y BBB. Al final se suman los estados cuyas tres cantidades difieren a lo sumo en d. Un palíndromo queda fijado por su primera mitad: con w par, la mitad debe terminar en una B sola (el centro queda BB); con w impar, la letra central debe ser distinta de la última de la mitad. Las cantidades totales son el doble de las de la mitad más la del centro. Todo cabe en 64 bits.

Python 3

(no disponible)

C++11

(no disponible)

103. L. Valley Gulls

Evento: RPC 03 - 2026 (Competitive Programming Network, 3rd Activity) **Tema:** Simulación por eventos **Tiempo:** $O(p^2 \cdot (p + m))$ **Memoria:** $O(p + m)$ **Archivo:** Valley.py / Valley.cpp.py / .cpp **Estado:** local-ok

Solución: El terreno se parte en cuencas separadas por los máximos locales; los acantilados cuentan como altura infinita. Cada lago recibe la lluvia que cae sobre todo su ancho (r·ancho de área por hora) más lo que desbordan los lagos llenos que drenan hacia él. Un lago lleno desborda por su borde más bajo y el agua sigue hasta llegar a un lago que todavía se está llenando. En cada fase se calcula cuánto tarda cada lago en llegar a su borde más bajo. Se avanza hasta el evento más próximo, y ese lago queda lleno o se fusiona con el vecino lleno a la misma altura. Antes de avanzar se revisa si algún nido del lago queda cubierto, usando el área exacta bajo el nivel del nido (trapezios y triángulos).

Python 3

(no disponible)

C++11

(no disponible)

104. A. Alejandro, lee por favor

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** Simulación / cadenas **Tiempo:** $O(L)$ **Memoria:** $O(L)$ **Archivo:** Alejandro.py / Alejandro.cpp.py / .cpp **Estado:** local-ok

Solución: El cifrado es determinista y el descifrado puede seguirlo letra a letra en el mismo orden. Se mantiene c y un contador por cada una de las 26 letras. Para cada letra cifrada e, la original es $L = (e - c) \bmod 26$; se escribe L, se incrementa el contador de L y, si queda en múltiplo de K, c aumenta en uno. Como c solo cambia después de procesar la letra, la simulación inversa reproduce exactamente el estado del cifrador. Basta trabajar c módulo 26 en la resta. Se recorre la suma total de longitudes (hasta 10^6) una sola vez, construyendo cada palabra en un buffer y uniendo con espacios al final.

Python 3

(no disponible)

C++11

(no disponible)

105. B. Betito el viajero

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** Grafos / flood fill **Tiempo:** $O(R \cdot C)$ por caso **Memoria:** $O(R \cdot C)$ **Archivo:** Betito.py / Betito.cpp.py / .cpp **Estado:** local-ok

Solución: Es un flood fill clásico. Se rodea la grilla con un borde de '#' para no revisar límites y se aplanan en un arreglo de bytes, de modo que los vecinos de la celda v son $v \pm 1$ y $v \pm W$. Desde la posición del asterisco se hace una búsqueda en profundidad iterativa con una pila: cada celda visitada se marca como '#' para no volver a contarla y se suma uno al contador. Al terminar la pila, el contador es la cantidad de lugares visitables. Cada celda entra a la pila como máximo una vez, así que el costo es lineal en el tamaño del mapa, con hasta 10 casos de 1000×1000 .

Python 3

(no disponible)

C++11

(no disponible)

106. C. Company

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** Árboles / DP **Tiempo:** $O(N)$ **Memoria:** $O(N)$ **Archivo:** Company.py / Company.cpp.py / .cpp **Estado:** local-ok

Solución: Como cada padre tiene índice menor que su hijo, recorrer los nodos de N a 2 procesa siempre los hijos antes que el padre, sin DFS ni recursión. La suma de distancias se obtiene por aristas: la arista (i, P_i) la cruzan $sz(i) \cdot (N - sz(i))$ pares, donde sz es el tamaño del subárbol. Para el diámetro se guarda $h(x)$, la altura del subárbol, y $F(x)$. Al procesar el hijo i de p , el candidato $h(p) + h(i) + 1$ une la rama más alta vista hasta ahora con la nueva; $F(p)$ también hereda $F(i)$. Luego $h(p) = \max(h(p), h(i) + 1)$. Todo es una pasada lineal; la suma cabe en 64 bits.

Python 3

(no disponible)

C++11

(no disponible)

107. D. Dangerous odyssey

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** BFS en grilla / sumas de prefijos **Tiempo:** $O(N \cdot M)$ **Memoria:** $O(N \cdot M)$ **Archivo:** Dangerous.py / Dangerous.cpp.py / .cpp **Estado:** local-ok

Solución: Primero se marcan las celdas peligrosas: una celda lo es si en el cuadrado de radio H centrado en ella hay una S o una B (los cantos se mueven en diagonal, así que es distancia de Chebyshev); con sumas de prefijos 2D cada consulta es $O(1)$. Los refugios válidos son R , Y y A no peligrosos. Un BFS multi-fuente en 4 direcciones desde todos ellos da la distancia de cada celda al refugio más cercano. Finalmente un BFS desde Y por celdas '.', Y o A no peligrosas y con distancia a refugio $\leq D$ da los pasos hasta A ; si no se llega, OSIDEO WILL DIE. Supuesto: la distancia al refugio es de 4 vecinos ignorando obstáculos (los ejemplos no distinguen variantes).

Python 3

(no disponible)

C++11

(no disponible)

108. E. Enarmonía

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** Programación dinámica **Tiempo:** $O(T^2 \cdot K)$ **Memoria:** $O(T \cdot K)$ **Archivo:** Enarmonia.py / Enarmonia.cpp.py / .cpp **Estado:** local-ok

Solución: La lectura queda descrita por cuántos actos se han tomado de cada manuscrito: si se leyeron a del primero y b del segundo, el siguiente acto de cada uno es el índice $a \bmod N_1$ o $b \bmod N_2$, y el último leído depende solo del manuscrito actual. Así el estado es $(t, a, \text{manuscrito actual, cambios usados})$, con $b = t - a$. Desde cada estado hay dos transiciones: seguir en el mismo manuscrito o cambiar (gastando un cambio), sumando 1 si el registro nuevo coincide con el anterior. Se arranca en $(1, 1, \text{primero}, 0)$ y la respuesta es el máximo en $t = T$. En Python puro se usan listas sobre (a, k) .

Python 3

(no disponible)

C++11

(no disponible)

109. F. Flipando colores con la DIAN

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** Greedy / scheduling en árboles (Horn) **Tiempo:** $O(n \log n)$ **Memoria:** $O(n)$ **Archivo:** Flipando.py / Flipando.cpp.py / .cpp **Estado:** local-ok

Solución: Es el problema 1[outtree] Σ wC, resuelto con el algoritmo de Horn. El nodo (o grupo ya fusionado) con mayor razón w/p que no es raíz debe ejecutarse inmediatamente después del grupo de su padre, así que se fusiona con él: el costo aumenta en $P_{\text{padre}} - W_{\text{hijo}}$ (el hijo se retrasa todo lo que dura el grupo padre) y el grupo padre acumula p y w . Se usa un heap de máximos por w/p con borrado perezoso y Union-Find para encontrar el grupo del padre. Una raíz virtual 0 con $p = w = 0$ recibe a las raíces reales. El costo base es la suma de $p_i - w_i$. En C++ se compara $w_1 \cdot p_2$ contra $w_2 \cdot p_1$ de forma exacta.

Python 3

(no disponible)

C++11

(no disponible)

110. G. Guanex y el diámetro con actualizaciones

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** Árboles / LCA (binary lifting) **Tiempo:** $O((n+q) \log V)$ **Memoria:** $O(V \log V)$ **Archivo:** Guanex.py / Guanex.cpp.py / .cpp **Estado:** local-ok

Solución: Si (a, b) son extremos de un diámetro y se agrega una hoja x , el nuevo diámetro es el máximo entre el anterior, $\text{dist}(x, a)$ y $\text{dist}(x, b)$; en ese caso los nuevos extremos incluyen a x . Las distancias se calculan con LCA por binary lifting: $\text{dist} = \text{prof}(x) + \text{prof}(y) - 2 \cdot \text{prof}(\text{lca})$. El árbol inicial se enraíza con un BFS que llena profundidad y la tabla de ancestros; cada hoja nueva solo necesita completar su fila de ancestros en $O(\log)$. El diámetro inicial sale tomando el nodo más profundo a y el más lejano de a . Las etiquetas son menores que $3 \cdot 10^5$, así que se usan arreglos directos.

Python 3

(no disponible)

C++11

(no disponible)

111. H. Humbertov y su taza de café

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** Geometría / fórmula cerrada **Tiempo:** $O(1)$ por caso **Memoria:** $O(1)$ **Archivo:** Humbertov.py / Humbertov.cpp.py / .cpp **Estado:** local-ok

Solución: Si se prolonga el cono truncado hasta su vértice, el volumen hasta una altura x es el de un cono de radio $\rho(x)$ menos el cono de radio r , y el volumen de un cono semejante es proporcional a ρ^3 . Como ρ crece linealmente con x , la condición de mitad de volumen es $\rho^3 - r^3 = (R^3 - r^3)/2$, es decir $\rho = ((r^3 + R^3)/2)^{1/3}$, y la altura es $x = h \cdot (\rho - r)/(R - r)$. Si $r = R$ la taza es un cilindro y la respuesta es $h/2$. Se imprime con 9 decimales. Nota: el ejemplo 2 del enunciado dice 6.509636245 pero el valor exacto es 6.5096362445; el juez acepta error $1e-6$.

Python 3

(no disponible)

C++11

(no disponible)

112. I. Internal triangles

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** Combinatoria / aritmética modular **Tiempo:** O(1) por consulta **Memoria:** O(q) **Archivo:** Internal.py / Internal.cpp.py / .cpp **Estado:** local-ok

Solución: Todo trío de vértices distintos del polígono convexo forma un triángulo distinto, así que la respuesta es $C(n, 3) = n(n-1)(n-2)/6$. En Python se calcula directamente con enteros grandes y luego módulo 10^9+7 ; en C++ se reduce cada factor módulo p y se multiplica por el inverso modular de 6 (166666668). El Internal.py original era correcto pero leía con input() línea por línea y recibió TLE en el juez con 10^5 consultas y límite de 1 segundo; la versión actual lee todo stdin de una vez y escribe todas las respuestas con un único print (0.13 s con el caso máximo).

Python 3

(no disponible)

C++11

(no disponible)

113. J. Juan y sus ovejas

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** Union-Find **Tiempo:** $O((N+P) \alpha(N))$ **Memoria:** O(N) **Archivo:** Juan.py / Juan.cpp.py / .cpp **Estado:** local-ok

Solución: Las razas son las componentes conexas del grafo de pares. Se usa Union-Find con compresión de caminos (por halving) y unión por tamaño: inicialmente hay N componentes de tamaño 1, y cada par que une dos componentes distintas reduce el conteo en uno y suma los tamaños en la nueva raíz. Al terminar el caso, el número de razas es el contador y la mayor raza es el máximo de los tamaños (los tamaños guardados en nodos que dejaron de ser raíz nunca superan al de su raíz, así que el máximo del arreglo es correcto). Cada caso es casi lineal en $N + P$.

Python 3

(no disponible)

C++11

(no disponible)

114. K. K-th shortest path

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** Dijkstra **Tiempo:** $O(K \cdot M \log N)$ **Memoria:** O(N+M) **Archivo:** Kthpath.py / Kthpath.cpp.py / .cpp **Estado:** local-ok

Solución: La definición es iterativa: el primer recorrido es el camino más corto; el segundo es el más corto en el grafo sin las aristas del primero, y así sucesivamente. Por eso basta repetir K veces ($K \leq 10$) un Dijkstra con heap desde S, guardando para cada vértice su predecesor y la arista usada, reconstruir el camino a D y marcar sus aristas como eliminadas (por identificador, para soportar aristas paralelas). El último Dijkstra da la distancia y el camino, que se imprime de S a D separado por ' - '. El enunciado garantiza unicidad, así que el desempate no importa.

Python 3

(no disponible)

C++11

(no disponible)

115. L. Locate Toby's nest

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** Árboles / centro por BFS **Tiempo:** O(H·W) por caso **Memoria:** O(H·W) **Archivo:** Locate.py / Locate.cpp.py / .cpp **Estado:** local-ok

Solución: Como entre dos pasillos hay un único camino, las celdas forman un árbol y la celda buscada es su centro: la de menor excentricidad. El centro de un árbol está en la mitad de cualquier diámetro. Se hace un BFS desde una celda cualquiera para encontrar el extremo a, otro BFS desde a guardando padres para encontrar el extremo b, y se reconstruye el camino de b a a de longitud L. Si L es par el centro es único (posición $L/2$); si es impar hay dos centros adyacentes (posiciones $\lfloor L/2 \rfloor$ y $\lfloor L/2 \rfloor + 1$) y se elige el de menor columna y luego menor fila. Salida en índices desde 1.

Python 3

(no disponible)

C++11

(no disponible)

116. M. Marble tilt maze

Evento: UTP Open 2026 (RPC 09 de 2026) **Tema:** BFS en espacio de estados **Tiempo:** $O((NM)^2)$ **Memoria:** $O((NM)^2)$ **Archivo:** Marble.py / Marble.cpp.py / .cpp **Estado:** local-ok (ejemplo 1 del enunciado inconsistente)

Solución: El estado es el par de posiciones ($\leq 900^2$ estados) y se hace BFS. Para cada dirección se precalcula dónde se detiene una canica sola desde cada celda (o -1 si cae en un agujero o sale del tablero). En un movimiento, si alguna cae el estado se descarta; si ambas terminan en la misma celda, están en la misma línea sin obstáculos entre ellas: la más cercana al tope ocupa la celda y la otra queda justo antes. Así cada transición es $O(1)$. PENDIENTE: con este modelo el ejemplo 1 del enunciado da -1 en lugar de 4 (los ejemplos 2 y 3 sí coinciden), y tampoco lo explican las variantes probadas (orden fijo de canicas, paso a paso); falta entender la regla del ejemplo 1. Auditoría: el modelo reproduce los ejemplos 2 y 3; en el ejemplo 1 ninguna canica puede alcanzar las columnas 4-5 de la fila 2 bajo ninguna regla de deslizamiento, por lo que ese ejemplo parece erróneo.

Python 3

(no disponible)

C++11

(no disponible)